

# Machine Learning Methods

Siger

October 15, 2025

Si Dieu est infini, alors je suis une partie de Dieu sinon je serai sa limite. . .

# Contents

|           |                                                              |           |
|-----------|--------------------------------------------------------------|-----------|
| <b>I</b>  | <b>Collect and Pre-process Data</b>                          | <b>7</b>  |
| <b>1</b>  | <b>Data cleaning</b>                                         | <b>8</b>  |
| 1.1       | Data Quality . . . . .                                       | 8         |
| 1.2       | Validity . . . . .                                           | 8         |
| 1.3       | Accuracy . . . . .                                           | 8         |
| 1.4       | Completeness . . . . .                                       | 8         |
| 1.5       | Consistency . . . . .                                        | 8         |
| 1.6       | Uniformity . . . . .                                         | 8         |
| 1.7       | The workflow . . . . .                                       | 9         |
| 1.8       | Inspection . . . . .                                         | 9         |
| 1.9       | Cleaning . . . . .                                           | 9         |
| 1.10      | Verifying . . . . .                                          | 9         |
| 1.11      | Reporting . . . . .                                          | 9         |
| <b>II</b> | <b>Statistics</b>                                            | <b>10</b> |
| <b>2</b>  | <b>Fundamental probability concepts</b>                      | <b>11</b> |
| 2.1       | Basic probability properties . . . . .                       | 11        |
| 2.1.1     | What is a probability? . . . . .                             | 11        |
| 2.1.2     | Properties . . . . .                                         | 11        |
| 2.1.3     | Moments . . . . .                                            | 12        |
| 2.1.4     | Asymptotic properties . . . . .                              | 12        |
| 2.1.5     | Central Limit Theorem . . . . .                              | 13        |
| 2.1.6     | Convergence in distribution . . . . .                        | 13        |
| 2.1.7     | Lévy Continuity Theorem . . . . .                            | 14        |
| 2.2       | Bivariate case . . . . .                                     | 14        |
| 2.3       | Distribution function . . . . .                              | 15        |
| 2.3.1     | Definition of probability density function (pdf): . . . . .  | 15        |
| 2.3.2     | Definition of cumulative density function (cdf): . . . . .   | 15        |
| 2.3.3     | Percentile for continuous random variables. . . . .          | 15        |
| <b>3</b>  | <b>Distributions</b>                                         | <b>16</b> |
| 3.1       | Discrete distributions with <i>finite</i> support . . . . .  | 16        |
| 3.1.1     | Bernoulli . . . . .                                          | 16        |
| 3.1.2     | Binomial . . . . .                                           | 16        |
| 3.1.3     | Hypergeometric . . . . .                                     | 17        |
| 3.1.4     | Negative Hypergeometric . . . . .                            | 17        |
| 3.1.5     | Benford's law . . . . .                                      | 18        |
| 3.1.6     | Zipf's law . . . . .                                         | 19        |
| 3.2       | Discrete distribution with <i>infinite</i> support . . . . . | 19        |
| 3.2.1     | Beta Negative . . . . .                                      | 19        |
| 3.2.2     | Boltzmann . . . . .                                          | 20        |
| 3.2.3     | Borel . . . . .                                              | 21        |
| 3.2.4     | Discrete phase-type . . . . .                                | 21        |
| 3.2.5     | Extended negative binomial . . . . .                         | 21        |
| 3.2.6     | Geometric . . . . .                                          | 21        |

|          |                                                        |           |
|----------|--------------------------------------------------------|-----------|
| 3.2.7    | Mixed Poisson . . . . .                                | 21        |
| 3.2.8    | Negative binomial . . . . .                            | 21        |
| 3.2.9    | compound Poisson . . . . .                             | 21        |
| 3.2.10   | Poisson . . . . .                                      | 21        |
| 3.2.11   | Skellam . . . . .                                      | 21        |
| 3.2.12   | Zeta . . . . .                                         | 21        |
| <b>4</b> | <b>Bayesian approach</b>                               | <b>22</b> |
| 4.1      | Components . . . . .                                   | 22        |
| 4.1.1    | Bayesian concept learning . . . . .                    | 22        |
| 4.1.2    | Likelihood . . . . .                                   | 22        |
| 4.1.3    | Prior . . . . .                                        | 22        |
| 4.1.4    | Posterior . . . . .                                    | 22        |
| 4.2      | Summarizing posterior distributions . . . . .          | 22        |
| 4.2.1    | MAP (Maximum A Posteriori) estimation . . . . .        | 22        |
| 4.2.2    | Credible intervals . . . . .                           | 23        |
| 4.3      | Bayesian Model Selection . . . . .                     | 23        |
| 4.3.1    | Baysian Occam's razor . . . . .                        | 23        |
| 4.3.2    | Computing the marginal likelihood (evidence) . . . . . | 24        |
| 4.3.3    | Bayes Factors . . . . .                                | 24        |
| 4.3.4    | Jeffreys-Lindley paradox . . . . .                     | 25        |
| 4.4      | Priors . . . . .                                       | 25        |
| 4.4.1    | Uninformative priors . . . . .                         | 25        |
| 4.4.2    | Jeffreys priors . . . . .                              | 25        |
| 4.4.3    | Robust priors . . . . .                                | 26        |
| 4.4.4    | Mixture of conjugate priors . . . . .                  | 26        |
| 4.5      | Hierarchical and Empirical Bayes . . . . .             | 26        |
| 4.5.1    | Hierarchical Bayes . . . . .                           | 26        |
| 4.5.2    | Empirical Bayes . . . . .                              | 26        |
| 4.6      | Bayesian Decision Theory . . . . .                     | 27        |
| 4.6.1    | Bayes estimators for common loss functions . . . . .   | 27        |
| 4.6.2    | Model evaluation metrics . . . . .                     | 28        |
| <b>5</b> | <b>Frequentist approach</b>                            | <b>31</b> |
| 5.1      | Sampling distribution . . . . .                        | 31        |
| 5.1.1    | Sampling Distributions of an estimator . . . . .       | 31        |
| 5.1.2    | Bootstrap . . . . .                                    | 31        |
| 5.2      | Frequentist decision theory . . . . .                  | 31        |
| 5.2.1    | Bayes risk . . . . .                                   | 32        |
| 5.2.2    | Admissible estimators . . . . .                        | 32        |
| 5.3      | Desirable properties of estimators . . . . .           | 32        |
| 5.3.1    | Consistent estimators . . . . .                        | 32        |
| 5.3.2    | Unbiased estimator . . . . .                           | 32        |
| 5.3.3    | Minimum variance estimators . . . . .                  | 33        |
| 5.3.4    | Bias-Variance Trade-off . . . . .                      | 33        |
| 5.4      | Empirical Risk Minimization . . . . .                  | 33        |
| 5.4.1    | Frequentist issue . . . . .                            | 33        |
| 5.4.2    | Regularized risk minimization . . . . .                | 33        |
| 5.5      | Components . . . . .                                   | 33        |
| 5.5.1    | Introduction . . . . .                                 | 33        |
| 5.5.2    | Hypothesis Testing . . . . .                           | 33        |
| 5.5.3    | <i>p-value</i> . . . . .                               | 34        |
| 5.5.4    | Confidence intervals . . . . .                         | 34        |
| 5.5.5    | Multiple comparisons . . . . .                         | 34        |
| 5.6      | Power Analysis . . . . .                               | 34        |
| 5.6.1    | Power of the test . . . . .                            | 35        |
| 5.6.2    | Significant threshold . . . . .                        | 35        |
| 5.6.3    | Effect size . . . . .                                  | 35        |

|            |                                                                      |           |
|------------|----------------------------------------------------------------------|-----------|
| <b>6</b>   | <b>Common statistical tests</b>                                      | <b>36</b> |
| 6.1        | Use of statistical tests . . . . .                                   | 36        |
| 6.1.1      | Terms . . . . .                                                      | 36        |
| 6.1.2      | Table of statistical hypothesis test . . . . .                       | 36        |
| 6.2        | List of common statistical test . . . . .                            | 36        |
| 6.2.1      | Binomial . . . . .                                                   | 36        |
| 6.2.2      | $\chi^2$ test . . . . .                                              | 37        |
| 6.2.3      | Exact test of goodness-of-fit . . . . .                              | 37        |
| 6.2.4      | Fisher's exact test . . . . .                                        | 38        |
| 6.2.5      | G-test . . . . .                                                     | 38        |
| 6.2.6      | Cochran's Q test . . . . .                                           | 39        |
| 6.2.7      | Sign test . . . . .                                                  | 40        |
| 6.2.8      | Contingency coefficients: Cramér's V . . . . .                       | 40        |
| 6.2.9      | Contingency table from a Bayesian perspective . . . . .              | 41        |
| 6.2.10     | Wilconox test . . . . .                                              | 41        |
| 6.2.11     | Mann-Whitney test . . . . .                                          | 42        |
| 6.2.12     | Kruskal-Wallis test . . . . .                                        | 42        |
| 6.2.13     | Friedman test . . . . .                                              | 43        |
| 6.2.14     | Spearman test . . . . .                                              | 43        |
| 6.2.15     | Pearson correlation coefficient . . . . .                            | 44        |
| 6.2.16     | Repeated-measures ANOVA . . . . .                                    | 44        |
| 6.2.17     | 1-way ANOVA . . . . .                                                | 44        |
| 6.2.18     | T-test . . . . .                                                     | 47        |
| <b>7</b>   | <b>Data recovery</b>                                                 | <b>49</b> |
| 7.1        | Sampling methods . . . . .                                           | 49        |
| 7.1.1      | Monte Carlo approximation . . . . .                                  | 49        |
| 7.1.2      | Bootstrap . . . . .                                                  | 49        |
| 7.1.3      | Monte Carlo Inference . . . . .                                      | 50        |
| 7.1.4      | Particle filtering . . . . .                                         | 50        |
| 7.1.5      | Annealing methods . . . . .                                          | 50        |
| 7.2        | Information theory . . . . .                                         | 50        |
| 7.2.1      | Shannon entropy . . . . .                                            | 50        |
| 7.2.2      | Kullback-Leibler (KL) divergence . . . . .                           | 51        |
| 7.2.3      | Mutual information . . . . .                                         | 51        |
| 7.3        | Key Mathematical functions . . . . .                                 | 51        |
| 7.3.1      | Softmax function . . . . .                                           | 51        |
| <b>8</b>   | <b>Markov Models</b>                                                 | <b>53</b> |
| 8.1        | Markov Models and Hidden Markov Models . . . . .                     | 53        |
| 8.1.1      | Markov Models . . . . .                                              | 53        |
| 8.1.2      | Hidden Markov Models . . . . .                                       | 54        |
| 8.1.3      | Inference HMMs . . . . .                                             | 55        |
| 8.1.4      | Learning for HMMs . . . . .                                          | 57        |
| 8.2        | State Model . . . . .                                                | 59        |
| 8.2.1      | Basics . . . . .                                                     | 59        |
| 8.2.2      | Time Series Forecasting . . . . .                                    | 59        |
| 8.2.3      | Inference in LG-SSM (Linear Gaussian - State Space Models) . . . . . | 60        |
| <b>9</b>   | <b>Markov Random Field</b>                                           | <b>61</b> |
| 9.0.1      | Basics . . . . .                                                     | 61        |
| <b>III</b> | <b>Classical Learning</b>                                            | <b>63</b> |
| <b>10</b>  | <b>Supervised Learning</b>                                           | <b>65</b> |
| 10.1       | Classification . . . . .                                             | 65        |
| 10.1.1     | Naive Bayes classifiers . . . . .                                    | 65        |
| 10.1.2     | Linear/Quadratic Discriminant Analysis . . . . .                     | 66        |

|           |                                                                               |           |
|-----------|-------------------------------------------------------------------------------|-----------|
| 10.1.3    | Nearest Shrunken Centroids Classifier . . . . .                               | 68        |
| 10.1.4    | Fisher's Linear Discriminant Analysis (FLDA) . . . . .                        | 68        |
| 10.1.5    | Logistic Regression . . . . .                                                 | 69        |
| 10.2      | Regression . . . . .                                                          | 70        |
| 10.2.1    | Linear Regression . . . . .                                                   | 70        |
| 10.2.2    | Generalized Linear Models (GLMs) . . . . .                                    | 71        |
| 10.2.3    | Learning to rank . . . . .                                                    | 72        |
| 10.2.4    | Supervised PCA . . . . .                                                      | 73        |
| 10.2.5    | Partial Least Squares . . . . .                                               | 73        |
| 10.3      | Classification and Regression . . . . .                                       | 74        |
| 10.3.1    | Multitask . . . . .                                                           | 74        |
| 10.3.2    | Mixture models . . . . .                                                      | 74        |
| 10.3.3    | ARD: Automatic Relevance Determination . . . . .                              | 76        |
| 10.3.4    | Sparse coding . . . . .                                                       | 76        |
| 10.3.5    | Support Vector Machines (SVMs) . . . . .                                      | 77        |
| 10.3.6    | MODEL COMPARISON . . . . .                                                    | 78        |
| 10.3.7    | Gaussian Processes . . . . .                                                  | 78        |
| 10.3.8    | Classification And Regression Trees (CART) . . . . .                          | 79        |
| 10.3.9    | Generalized Additive models . . . . .                                         | 80        |
| 10.3.10   | Multi-layer Perceptron . . . . .                                              | 81        |
| 10.4      | Model Selection . . . . .                                                     | 82        |
| 10.4.1    | Bayesian Variable Selection . . . . .                                         | 82        |
| 10.4.2    | Least Angle Regression Shrinkage . . . . .                                    | 83        |
| 10.5      | Regularization . . . . .                                                      | 84        |
| 10.5.1    | $l_2$ regularization . . . . .                                                | 84        |
| 10.5.2    | $l_1$ regularization . . . . .                                                | 85        |
| 10.5.3    | Noise Robustness . . . . .                                                    | 85        |
| 10.5.4    | Regularization path . . . . .                                                 | 86        |
| 10.5.5    | Coordinate Descent . . . . .                                                  | 86        |
| 10.6      | Ensemble Learning . . . . .                                                   | 87        |
| 10.6.1    | Boosting . . . . .                                                            | 87        |
| 10.6.2    | Bayes Model Averaging (that is not an Ensemble Method) . . . . .              | 88        |
| 10.6.3    | Stacking . . . . .                                                            | 88        |
| 10.6.4    | Bootstrap Aggregating (Bagging) . . . . .                                     | 89        |
| <b>11</b> | <b>Unsupervised Learning</b>                                                  | <b>90</b> |
| 11.1      | Clustering . . . . .                                                          | 90        |
| 11.1.1    | Clustering in general . . . . .                                               | 90        |
| 11.1.2    | <i>K-means</i> algorithm . . . . .                                            | 91        |
| 11.1.3    | Vector Quantization . . . . .                                                 | 91        |
| 11.1.4    | Factor Analysis . . . . .                                                     | 92        |
| 11.1.5    | Mixture of Factor Analyzers . . . . .                                         | 92        |
| 11.1.6    | Canonical Correlation Analysis . . . . .                                      | 92        |
| 11.1.7    | Independent Component Analysis (ICA) . . . . .                                | 93        |
| 11.1.8    | DBSCAN: Density-Based Spatial Clustering of Applications with Noise . . . . . | 93        |
| 11.1.9    | Affinity Propagation . . . . .                                                | 94        |
| 11.1.10   | Dirichlet Process . . . . .                                                   | 94        |
| 11.1.11   | Spectral Clustering . . . . .                                                 | 95        |
| 11.1.12   | Agglomerative Clustering . . . . .                                            | 95        |
| 11.1.13   | Bayesian Hierarchical Clustering . . . . .                                    | 95        |
| 11.2      | Association . . . . .                                                         | 96        |
| 11.2.1    | Directed graphical models (Bayes nets) . . . . .                              | 96        |
| 11.2.2    | Kernels . . . . .                                                             | 96        |
| 11.2.3    | Latent Dirichlet Allocation . . . . .                                         | 98        |
| 11.3      | Dimensionality Reduction . . . . .                                            | 98        |
| 11.3.1    | Principal Components Analysis . . . . .                                       | 98        |
| 11.3.2    | PCA for categorical data . . . . .                                            | 99        |

|                                                                                 |            |
|---------------------------------------------------------------------------------|------------|
| <b>12 Semi-supervised Learning</b>                                              | <b>100</b> |
| <b>13 Reinforcement Learning</b>                                                | <b>101</b> |
| <b>14 Optimization methods</b>                                                  | <b>102</b> |
| 14.1 Optimization of loss functions . . . . .                                   | 102        |
| 14.1.1 Gradient Descent . . . . .                                               | 102        |
| 14.1.2 Newton's method . . . . .                                                | 102        |
| 14.1.3 Online learning and stochastic optimization . . . . .                    | 103        |
| 14.1.4 EM Algorithm . . . . .                                                   | 103        |
| 14.1.5 Backpropagation Algorithm . . . . .                                      | 104        |
| 14.2 Variational Inference . . . . .                                            | 104        |
| 14.2.1 Basics . . . . .                                                         | 104        |
| 14.2.2 Variational Bayes . . . . .                                              | 105        |
| 14.2.3 Variational Bayes EM . . . . .                                           | 105        |
| 14.2.4 Gibbs sampling . . . . .                                                 | 105        |
| 14.3 Numerical Computation . . . . .                                            | 105        |
| 14.3.1 Numerical Computation . . . . .                                          | 105        |
| 14.3.2 Jacobian and Hessian matrices . . . . .                                  | 105        |
| 14.3.3 Optimization technique . . . . .                                         | 105        |
| <b>IV Deep Learning</b>                                                         | <b>106</b> |
| <b>15 Optimization</b>                                                          | <b>107</b> |
| 15.1 Back-propagation . . . . .                                                 | 107        |
| 15.1.1 Purpose . . . . .                                                        | 107        |
| 15.1.2 Modeling . . . . .                                                       | 107        |
| <b>16 Regularization</b>                                                        | <b>109</b> |
| 16.1 Norm penalties . . . . .                                                   | 109        |
| 16.1.1 Data Augmentation . . . . .                                              | 109        |
| 16.1.2 Drop out . . . . .                                                       | 109        |
| <b>17 Types of Neural Networks</b>                                              | <b>110</b> |
| 17.1 Convolutional Neural Networks . . . . .                                    | 110        |
| 17.1.1 Components of CNN . . . . .                                              | 110        |
| 17.2 Recurrent and Recursive Neural Networks . . . . .                          | 112        |
| 17.2.1 Recurrent Neural Networks . . . . .                                      | 112        |
| 17.2.2 Encoder-Decoder Sequence-to-Sequence Architectures . . . . .             | 112        |
| 17.2.3 The Challenge of Long-Term Dependencies . . . . .                        | 113        |
| 17.2.4 Leaky Units . . . . .                                                    | 113        |
| 17.2.5 The Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) . . . . | 114        |
| <b>V Overall Issues</b>                                                         | <b>115</b> |
| 17.3 High Dimensions . . . . .                                                  | 116        |
| 17.3.1 Dimension Increase . . . . .                                             | 116        |
| 17.3.2 Curse of dimensionality . . . . .                                        | 116        |

## Part I

# Collect and Pre-process Data



# Chapter 1

## Data cleaning

[1]

### 1.1 Data Quality

### 1.2 Validity

- **Data-Type Constraints:** for a given column a fixed data-type must be associated with.
- **Range Constraints:** only a range of values should be taken.
- **Mandatory Constraints:** some columns cannot be empty.
- **Unique Constraints:** across a given dataset a field or a combination of variables.
- **Foreign-key constraints:** a foreign key column cannot have a value that does not exist in the primary key.
- **Regular expression patterns:** text fields that have to follow a given alphanumerical pattern.
- **Cross-field validation:** consistency of values, for example considering a given man, his birth date have to be older than his death date.

### 1.3 Accuracy

The degree to which the data is close to the true value.

### 1.4 Completeness

The degree to which the all the required data is known.

### 1.5 Consistency

The degree to which the data is consistent, within the same data set or across multiple data sets.

### 1.6 Uniformity

The degree to which the data is specified using the same unit of measure.

## 1.7 The workflow

## 1.8 Inspection

Detect unexpected behavior in the data.

- **Data profiling:** summary statistics about the data, see *ydata-profiling* in Python.
- **Visualizations:** visualize the data using statistical metrics, see *plotly*
- **Software packages:** to note and check the constraints regarding the data see *pydeequ*

## 1.9 Cleaning

Fix or remove anomalies discovered in the above phase.

- **Irrelevant Data:** ask to the expert what can be the unnecessary columns, check them and remove them if they are not useful.
- **Duplicates**
- **Type conversion:** make sure the appropriate data type is associated with a given column.
- **Syntax errors:** white spaces, pad strings ...
- **Standardize:** same unit across the dataset, same pattern for text.
- **Scaling/Transformation:** in order to compare different scores for example.
- **Normalization:** useful for some statistical methods.
- **Missing values:**
  - Drop: only if the missing values in a column rarely and randomly occur.
  - Impute: many methods,
    - \* *mean* is relevant when data is not skewed
    - \* *median* otherwise
    - \* *hot-deck* imputes from a randomly selected similar record
    - \* *polynomial interpolation* approximate the data with a polynomial
    - \* *linear regression* imputes from prediction
    - \* *k-nearest* imputes depending on a set of closet observations
  - Flag: let the missing value as it is.
- **Outliers:** Remove outliers only if they are harmful for the chosen model.
- **In-record & cross-datasets errors:** fix non-consistent situations like married kids, quantity being different of the one when we compute using other columns.

## 1.10 Verifying

Check correctness of the cleaning phase.

## 1.11 Reporting

Report about changes made, using one of the software summarising the data quality for example.

# **Part II**

# **Statistics**

## Chapter 2

# Fundamental probability concepts

### 2.1 Basic probability properties

#### 2.1.1 What is a probability?

It is a [mathematical measure of the uncertainty](#) of a given event.

**Objectivist interpretation [3]** : assigns numbers describing some objective state, [Frequentist interpretation](#) claiming that the probability of a random event is quantified by the relative frequency in a given experiment.

**Subjectivist interpretation [3]** : assigns numbers quantifying the degree of belief that a given event occurs. *Bayesian* interpretation uses expert knowledge considered as subjective and represented by the prior, as well as experimental data represented by the likelihood. The normalized product of the 2 above quantity is the [posterior probability distribution containing both expert knowledge and experimental data](#).

#### 2.1.2 Properties

**Event and its opposite**  $\mathbb{P}(\{A\}) + \mathbb{P}(\{\bar{A}\}) = 1$

**Not necessary mutually exclusive events**  $\mathbb{P}(\{A \cup B\}) = \mathbb{P}(\{A\}) + \mathbb{P}(\{B\}) - \mathbb{P}(\{A \cap B\})$

**Independent events**  $\mathbb{P}(\{A \cap B\}) = \mathbb{P}(\{A\}) \times \mathbb{P}(\{B\})$

**Conditional Probability**  $\mathbb{P}(A|B) = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)}$ , keep in mind that it comes from the fact that considering case where A occurs knowing that B occurred means reducing probability space  $\Omega$  to  $B$ . In combinatorial we could think about  $|A|B| = \frac{|A \cap B|}{|B|} = \frac{\frac{|A \cap B|}{|\Omega|}}{\frac{|B|}{|\Omega|}} = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)}$

**Law of Total Probability**  $\begin{cases} (B_i)_{1 \leq i \leq n} : \text{partition of a sample } \mathcal{S} \\ \forall i \in \llbracket 1, n \rrbracket, \mathbb{P}(\{B_i\}) \neq 0 \end{cases} \Rightarrow \mathbb{P}(A) = \sum_{i=1}^n \mathbb{P}(B_i) \mathbb{P}(A|B_i)$

**Bayes' Theorem** Using [Law of Total Probability](#):

$$\begin{cases} (B_i)_{1 \leq i \leq n} : \text{partition of a sample } \mathcal{S} \\ \forall i \in \llbracket 1, n \rrbracket, \mathbb{P}(\{B_i\}) \neq 0 \end{cases} \Rightarrow \mathbb{P}(B_i|A) = \frac{\mathbb{P}(A|B_i) \times \mathbb{P}(B_i)}{\sum_{k=1}^n \mathbb{P}(A|B_k) \mathbb{P}(B_k)}$$

It is used to compute the [probability of a cause given its effect](#).

### 2.1.3 Moments

They are certain quantitative measures related to the shape of the function's graph [2].

Recall that in physics to compute moment you need to store for each point the product resulting from the distance between mass center and that point with force applied on that point and finally sum them

each other. ( $\mu = \sum_x d_c(x) \times F(x)$ ).

**$n^{th}$  moments of a random variable:** The  $n^{th}$  moment about the origin of a random variable  $X$  as denoted by  $E(X^n)$ , is defined to be:

$$\mathbb{E}(X^n) = \begin{cases} \sum_{x \in R_X} x^n f(x) & \text{if } X \text{ is discrete} \\ \int_{-\infty}^{\infty} x^n f(x) dx & \text{if } X \text{ is continuous} \end{cases}$$

**Expected value:** The expected value of a random variable  $X$  as denoted by  $E(X)$ , is defined to be:

$$\mathbb{E}(X) = \begin{cases} \sum_{x \in R_X} x f(x) & \text{if } X \text{ is discrete} \\ \int_{-\infty}^{\infty} x f(x) dx & \text{if } X \text{ is continuous} \end{cases}$$

After normalized this moment by total mass we have the [center of mass](#).

**Variance :** Let  $X$  be a random variable with mean  $\mu_X$ . The variance of  $X$  denoted by  $\mathbb{V}(X)$  or  $\sigma_X^2$  is defined by:

$$\mathbb{V}(X) = \mathbb{E}([X - \mu_X]^2)$$

After normalized this moment by total mass we have the [moment of inertia](#).

If  $X$  is a random variable with mean  $\mu_X$  and variance  $\sigma_X^2$  then:

$$\sigma_X^2 = \mathbb{E}(X^2) - \mu_X^2$$

And:

$$\mathbb{V}(aX + b) = a^2 \mathbb{V}(X)$$

### Skewness and Kurtosis

- **Skewness:**  $\mathbb{E}\left(\left[\frac{X - \mu_X}{\sigma_X}\right]^3\right)$ , indicates the direction (negative  $\rightarrow$  left tail is longer, positive  $\rightarrow$  right tail is longer) and relative magnitude of a distribution's [deviation from the normal distribution](#).
- **Kurtosis:**  $\mathbb{E}\left(\left[\frac{X - \mu_X}{\sigma_X}\right]^4\right)$ , [measures the outliers](#), data within one standard deviation will not contribute a lot to the kurtosis values conversely data exceeding one standard deviation will contribute a lot because of the fourth power.

### 2.1.4 Asymptotic properties

**Chebychev inequality** allows to find an estimate of the area between the values  $\mu - k\sigma$  and  $\mu + k\sigma$  for some given  $k \neq 0$ , showing that the area under  $f(x)$  on the interval  $[\mu - k\sigma, \mu + k\sigma]$  is at least  $1 - \frac{1}{k^2}$ . Let  $X$  be a random variable with probability density function  $f(x)$ . If  $\mu$  and  $\sigma > 0$  are the mean and standard deviation of  $X$  then:

$$\mathbb{P}(\{|X - \mu| < k\sigma\}) \geq 1 - \frac{1}{k^2}$$

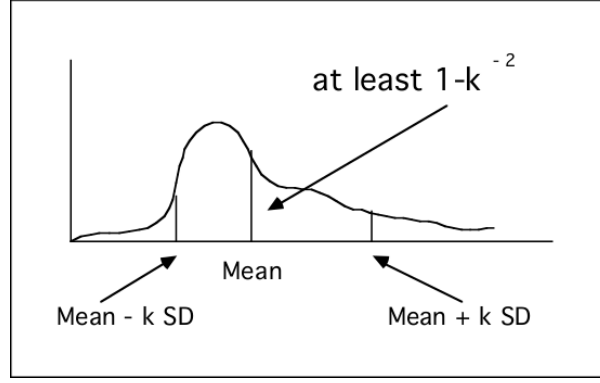


Figure 2.1: Illustration of Chebychev inequality

### Markov inequality

$$X \geq 0 \Rightarrow \mathbb{P}(\{X \geq t\}) \leq \frac{\mathbb{E}(X)}{t}$$

**Theorem weak law of large numbers:** Let  $(X_i)_{1 \leq i \leq n}$ : independent & identically distributed RV

$$\forall \epsilon \in \mathbb{R}_+ : \lim_{n \rightarrow \infty} \mathbb{P}(\{|\bar{S}_n - \mu| \geq \epsilon\}) = 0 \text{ with } \bar{S}_n = \frac{1}{n} \sum_{i=1}^n X_i$$

**Convergence in probability** Suppose  $(X_i)_{1 \leq i \leq n}$  is a sequence of random variables defined on a sample space  $S$ . The sequence “converges in probability” to the random variable  $X$  if, for any  $\epsilon > 0$

$$\lim_{n \rightarrow \infty} \mathbb{P}(\{|X_n - X| < \epsilon\}) = 1$$

**Convergence almost surely** Suppose the RV  $X$  and  $(X_i)_{1 \leq i \leq n}$  is a sequence of random variables defined on a sample space  $S$ . The sequence  $X_n(\omega)$  “converges almost surely” to  $X(\omega)$  if

$$\mathbb{P}\left(\left\{w \in S \mid \lim_{n \rightarrow \infty} X_n(\omega) = X(\omega)\right\}\right) = 1$$

### Properties

- For a Bernoulli distribution,  $\bar{S}_n$  converges in probability to  $p$
- For a Normal distribution,  $\bar{S}_n$  converges almost surely to  $\mu$

### 2.1.5 Central Limit Theorem

The central limit theorem (Lindeberg-Levy Theorem) states that for any population distribution, the distribution of the standardized sample mean is approximately standard normal with better approximations obtained with the larger sample size.

$$\left\{ \begin{array}{l} (X_i)_{1 \leq i \leq n} \\ n \rightarrow \infty \end{array} \right. \overset{n \hookrightarrow ?(\mu, \sigma^2)}{\Rightarrow} \frac{\bar{X} - \mu}{\frac{\sigma}{\sqrt{n}}} \hookrightarrow \mathcal{N}(0, 1)$$

### 2.1.6 Convergence in distribution

Consider  $X$  with its cumulative density function  $F$  and  $(X_i)_{1 \leq i \leq n}$  with their cdf  $(F_i)_{1 \leq i \leq n}$ :

$$\lim_{n \rightarrow \infty} F_n(x) = F(x) \Rightarrow X_n \text{ "converges in distribution" to } X$$

### 2.1.7 Lévy Continuity Theorem

$$\begin{cases} (X_i)_{1 \leq i \leq n} \text{ RV} \\ (F_i)_{1 \leq i \leq n} \text{ distribution functions} \\ (M_{X_i})_{1 \leq i \leq n} \text{ moment generating function} \end{cases} \quad \forall t \in [-h, h] \lim_{n \rightarrow \infty} M_{X_n}(t) = M_X(t) \Rightarrow \lim_{n \rightarrow \infty} F_n(x) = F(x)$$

## 2.2 Bivariate case

**Joint probability density function** Let  $(X, Y) : (\Omega_X, \Omega_Y) \rightarrow (R_X, R_Y)$  and  $f : R_X \times R_Y \rightarrow \mathbb{R}$

$$\forall (x, y) \in R_X \times R_Y, f(x, y) = \mathbb{P}(\{X = x, Y = y\}) \Leftrightarrow$$

$f$  is the joint probability density function for  $X$  and  $Y$

**Marginal probability density function** Let for all  $(x, y) \in R_X \times R_Y$ :  $f(x, y)$  be the joint probability density of  $X$  and  $Y$

$$\begin{cases} f_X(x) = \int_{-\infty}^{\infty} f(x, y) dy \text{ is the marginal probability density of } X \\ f_Y(y) = \int_{-\infty}^{\infty} f(x, y) dx \text{ is the marginal probability density of } Y \end{cases}$$

**Joint cumulative probability distribution function** Let  $F : \mathbb{R}^2 \rightarrow \mathbb{R}$

$$\forall (x, y) \in \mathbb{R}^2, F(x, y) = \mathbb{P}(\{X \leq x, Y \leq y\}) = \int_{-\infty}^y \int_{-\infty}^x f(u, v) du dv \Leftrightarrow$$

$F$  is the joint cumulative probability density function for  $X$  and  $Y$

From the fundamental theorem of calculus:  $f(x, y) = \frac{\partial^2 F(x, y)}{\partial x \partial y}$

**Conditional expectation** The conditional mean of  $X$  given  $Y = y$  is defined as:

$$\mathbb{E}(X|y) = \begin{cases} \sum_{x \in R_X} xg(x/y) \Leftarrow X \text{ discrete} \\ \int_{-\infty}^{\infty} xg(x/y) dx \Leftarrow X \text{ continuous} \end{cases}$$

Properties:

$$\begin{cases} \mathbb{E}_X(\mathbb{E}_{y|x}(Y|X)) = \mathbb{E}_Y(Y) \\ \mathbb{E}(Y|\{X = x\}) = \mu_Y + \rho \frac{\sigma_Y}{\sigma_X}(x - \mu_X) \end{cases}$$

**Conditional Variance**

$$\begin{cases} \mathbb{V}(Y|x) = \mathbb{E}(Y^2|x) - \mathbb{E}(Y|x)^2 \\ \mathbb{E}_x(\mathbb{V}(Y|X)) = (1 - \rho^2)\mathbb{V}(Y) \end{cases}$$

**Conditional Independence**  $X \perp Y|Z \Leftrightarrow \mathbb{P}(X, Y|Z) = \mathbb{P}(X|Z)\mathbb{P}(Y|Z)$

**The chain rule of conditional probabilities** Any joint probability distribution over many random variables may be decomposed into conditional distribution over only one variable:

$$\mathbb{P}(x_{1:T}) = \mathbb{P}(x_1) \prod_{t=2}^T \mathbb{P}(x_t|x_{1:t-1})$$

## 2.3 Distribution function

### 2.3.1 Definition of probability density function (pdf):

Let  $R_X$  be the space of the random variable  $X$ . The function:  $f : R_X \rightarrow \mathbb{R}$  defined by:

$$f(x) = \mathbb{P}(\{X = x\}) \text{ if } X \text{ is discrete.}$$

$$f(x) = \mathbb{P}(\{X \in A\}) = \int_A f(x)dx \text{ if } X \text{ is continuous, with } A \text{ a set of real numbers.}$$

is called probability density function of  $X$ .

### 2.3.2 Definition of cumulative density function (cdf):

Let  $R_X$  be the space of the random variable  $X$ . The function:  $F : R_X \rightarrow \mathbb{R}$  defined by:

$$F(x) = \mathbb{P}(\{X \leq x\}) \text{ if } X \text{ is discrete.}$$

$$F(x) = \mathbb{P}(\{X \leq x\}) = \int_{-\infty}^x f(t)dt \text{ if } X \text{ is continuous, with } A \text{ a set of real numbers.}$$

### 2.3.3 Percentile for continuous random variables.

Let  $p \in [0; 1]$ , a  $100p^{th}$  percentile of the distribution of a random variable  $X$  is  $q \in \mathbb{R}$  satisfying:

$$\mathbb{P}(\{X \leq q\}) \leq p$$

(Recall that the  $F$  is a monotonically increasing function, then it has an inverse  $F^{-1}$ )  
 $q = F^{-1}(p)$

A  $100p^{th}$  is a measure of location for the probability distribution in the sense that  $q$  divides the distribution of the probability mass into 2 parts, one having probability mass  $p$  and other having probability mass  $1 - p$

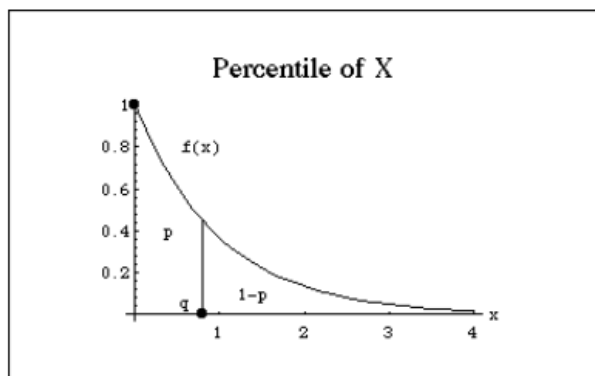


Figure 2.2: Percentile

The  $50^{th}$  percentile of any distribution is called median of the distribution.



# Chapter 3

## Distributions

### 3.1 Discrete distributions with *finite* support

#### 3.1.1 Bernoulli

**Purpose** To estimate the outcome of an experience providing binary outcomes.

**Theory**

**Parameters**  $p$  such that  $\mathbb{P}(X = 1) = p$ .

#### 3.1.2 Binomial

**Purpose** It estimates the number of successes in a sequence of  $n$  independent experiments having binary outcomes.

**Parameters**

- $n$ : number of independent experiments
- $p$ : probability of success,  $\mathbb{P}(X = 1) = p$
- $f(k, n, p) = \mathbb{P}(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}$  the probability mass function giving the probability to get exactly  $k$  successes over  $n$  independent experiments.

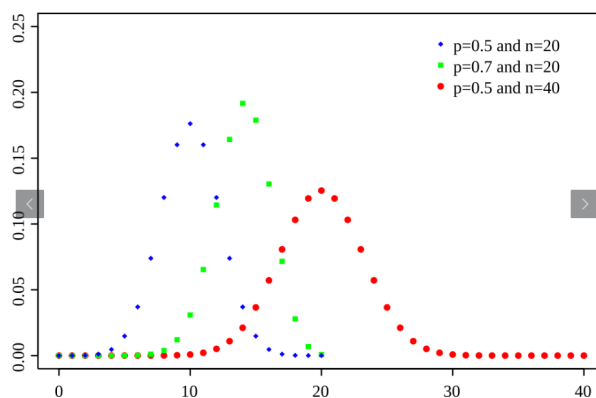


Figure 3.1: Binomial probability mass function

### Approximation

- *Normal distribution*: when  $n$  becomes large enough the *skew* gets lower and a reasonable approximation of  $\mathcal{B}(n, p) \sim \mathcal{N}(np, np(1-p))$
- *Poisson distribution*: when  $n \rightarrow \infty \wedge np \rightarrow C$  ( $n$  should be sufficiently large and  $p$  sufficiently small)  $\mathcal{B}(n, p) \sim \mathcal{P}(np)$
- *Beta distribution*: when  $\alpha = k+1 \wedge \beta = n-k+1$  we have  $\text{Beta}(p, \alpha, \beta) = (n+1)\text{Binomial}(k, n, p)$

### 3.1.3 Hypergeometric

**Purpose** It describes the probability of  $k$  successes (random draws for which an object has a given feature) in  $n$  draws *without replacement* from a finite population of size  $N$  that contains exactly  $K$  objects with that feature.

#### Theory

##### Parameters

- $N$  (population size)  $\in \mathbb{N}$
- $K$  (count of objects having the sought feature)  $\in \llbracket 0, N \rrbracket$
- $n$  (count of draws)  $\in \llbracket 0, N \rrbracket$

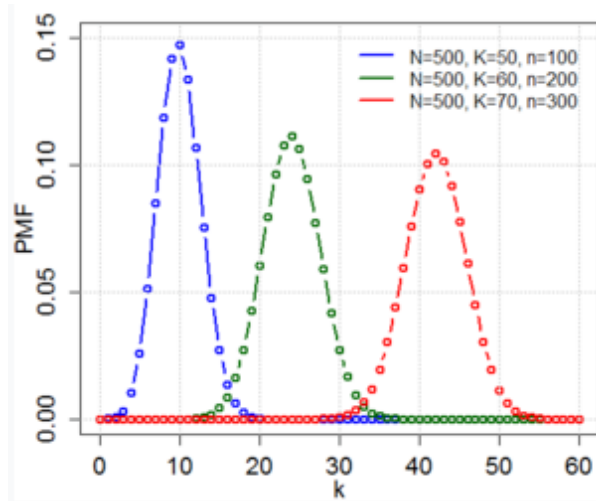


Figure 3.2: Hypergeometric probability mass function

**Probability mass function**  $f(k; N, n, K) = \mathbb{P}(X = k) = \frac{\binom{K}{k} \binom{N-K}{n-k}}{\binom{N}{n}}$

### 3.1.4 Negative Hypergeometric

**Paragraph** It describes the likelihood of  $k$  successes in a sample with exactly  $r$  failures.

#### Theory

### Parameters

- $N$  (population size)  $\in \mathbb{N}$
- $K$  (count of objects having the sought feature)  $\in \llbracket 0, N \rrbracket$
- $r$  (count of failures) *in*  $\llbracket 0, N - K \rrbracket$

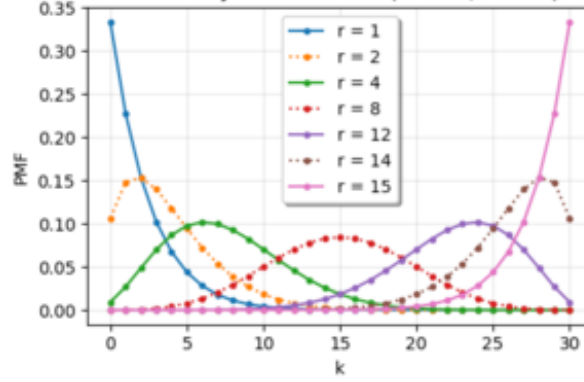


Figure 3.3: Negative Hypergeometric probability mass function

**Probability mass function**  $f(k; N, r, K) = \mathbb{P}(X = k) = \frac{\binom{k+r-1}{k} \binom{N-r-k}{K-k}}{\binom{N}{K}}$

### 3.1.5 Benford's law

**Purpose** It tends to describe the probability that a number, in a real-life set, starts with a given digit (0-9), it follows an observation that in many real-life sets of numerical data the leading digit is likely to be small.

It seems that real-world distributions that span several orders of magnitude rather uniformly like stock-market prices are likely to satisfy Benford's law very accurately, however distribution mostly or entirely within one order of magnitude like IQ scores is unlikely to satisfy Benford's law very accurately.

### Theory

**Parameters**  $d \in \llbracket 1, 9 \rrbracket$  the leading digit

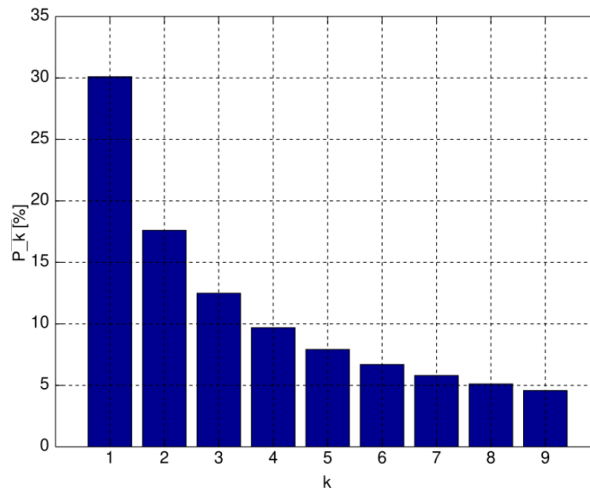


Figure 3.4: Benford's law probability density function

**Probability mass function**  $f(d) = \mathbb{P}(X = d) = \log_{10}(d+1) - \log_{10}(d) = \log_{10}\left(\frac{d+1}{d}\right) = \log_{10}\left(1 + \frac{1}{d}\right)$

### 3.1.6 Zipf's law

**Purpose** This empirical law states that when a list of measured values is sorted in decreasing order the value of the  $n$ -th entry is often approximately inversely proportional to  $n$ .

#### Theory

**Parameters** Let's consider the generalized case where we have an inverse power law with exponent  $s$  instead of 1

- $s \in \mathbb{R}_+$  exponent of generalization
- $N \in \mathbb{N}_*$  count of elements at which we assign a rank  $k$

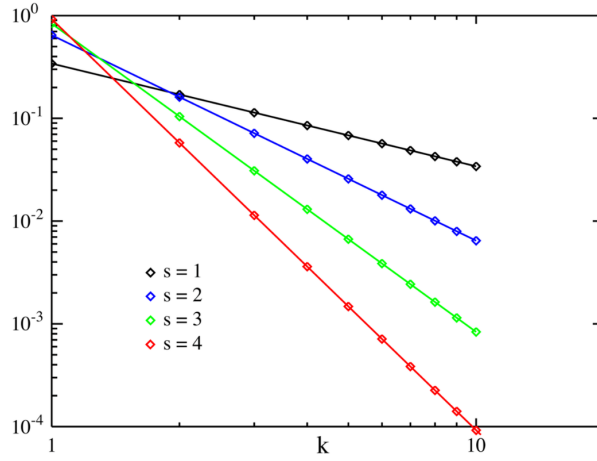


Figure 3.5: Zipf's law probability mass function

Consider  $H_{N,s} = \sum_{k=1}^N \frac{1}{k^s}$  the  $N$ th harmonic number.

$$f(k; N, s) = \begin{cases} \frac{1}{H_{N,s}} \times \frac{1}{k^s} & \Leftrightarrow k \in \llbracket 1, N \rrbracket \\ 0 & \Leftrightarrow (k < 1) \vee (N < k) \end{cases}$$

#### Application

- Natural Language:  $word\_frequency \propto \frac{1}{word\_rank}$

## 3.2 Discrete distribution with *infinite* support

### 3.2.1 Beta Negative

**Purpose** It describes the probability of failures needed to get  $r$  successes in a sequence of independent Bernoulli trials. The probability  $p$  remains constant within a given a Bernoulli experiment however varies across different experiments following a beta distribution.

#### Theory

### Parameters

- $\alpha \in \mathbb{R}_+$  parameter of Beta distribution
- $\beta \in \mathbb{R}_+$  parameter of Beta distribution

**Probability mass function** Consider  $f_{X|p}(k|r, q)$  the negative binomial density and  $f_p(q|\alpha, \beta)$  the beta density we obtain the probability mass function  $f(k|\alpha, \beta, r)$  by marginalisation

$$\begin{aligned}
 f(k|\alpha, \beta, r) &= \int_0^1 f_{X|p}(k|r, q) \times f_p(q|\alpha, \beta) dq \\
 &= \int_0^1 \binom{k+r-1}{k} (1-q)^k q^r \times \frac{q^{\alpha-1} (1-q)^{\beta-1}}{B(\alpha, \beta)} dq \\
 &= \frac{1}{B(\alpha, \beta)} \binom{k+r-1}{k} \int_0^1 q^{\alpha+r-1} (1-q)^{\beta+k-1} dq \\
 &= \frac{1}{B(\alpha, \beta)} \binom{k+r-1}{k} \frac{\Gamma(\alpha+r)\Gamma(\beta+r)}{\Gamma(\alpha+\beta+k+r)} \\
 &= \frac{1}{B(\alpha, \beta)} \binom{k+r-1}{k} B(\alpha+r, \beta+k)
 \end{aligned}$$

More generally  $f(x) = \frac{\Gamma(r+k)}{k!\Gamma(r)} \times \frac{B(\alpha+r, \beta+k)}{B(\alpha, \beta)}$

### 3.2.2 Boltzmann

**Purpose** This distribution known as well as Gibbs distribution describes the probability that a system will be in certain state as a function of that state's energy and the temperature of the system.

#### Theory

##### Parameters

- $\epsilon_i$  the energy of the state  $i$
- $k$  the Boltzmann constant
- $T$  the temperature

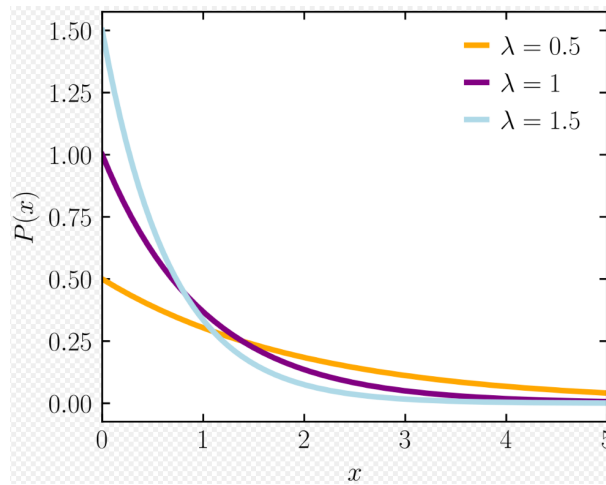


Figure 3.6: Boltzmann probability function, note that  $\lambda = kT$

**Probability mass function** Assume  $p_i$  the probability of the system being in state  $i$ ,  $p_i \propto e^{-\frac{\epsilon_i}{kT}}$ .  
 More precisely we have  $f(i) = \mathbb{P}(X = i) = \frac{e^{-\frac{\epsilon_i}{kT}}}{\sum_{j=1}^M e^{-\frac{\epsilon_j}{kT}}}$

**3.2.3 Borel**

**3.2.4 Discrete phase-type**

**3.2.5 Extended negative binomial**

**3.2.6 Geometric**

**3.2.7 Mixed Poisson**

**3.2.8 Negative binomial**

**3.2.9 compound Poisson**

**3.2.10 Poisson**

**3.2.11 Skellam**

**3.2.12 Zeta**

# Chapter 4

## Bayesian approach

### 4.1 Components

#### 4.1.1 Bayesian concept learning

Let be  $\mathcal{D}$  the data and in

- discrete approach  $h$  the hypothesis taken in account (Binomial, ...)
- in continuous approach:  $m, \theta$  the model (Bernoulli, Gaussian, ...) and its parameters associated with

#### 4.1.2 Likelihood

the probability to get the observed data:

- discrete approach:  $p(\mathcal{D}|h)$
- continuous approach:  $p(\mathcal{D}|\theta, m)$

#### 4.1.3 Prior

the probability of our hypothesis, many prior can be used, and this **subjective** aspect of Bayesian reasoning is a source of much controversy.

- in discrete approach:  $p(h)$
- in continuous approach:  $p(\theta|m)$

#### 4.1.4 Posterior

The posterior is simply the likelihood times the prior, normalized.

- in discrete approach:  $p(h|\mathcal{D}) = \frac{p(\mathcal{D}|h) \times p(h)}{\sum_{h' \in \mathcal{H}} p(\mathcal{D}|h') p(h')}$
- in continuous approach:  $p(\theta|\mathcal{D}, m) = \frac{p(\mathcal{D}|\theta, m) \times p(\theta|m)}{\int_{\theta' \in \Theta} p(\mathcal{D}|\theta', m) p(\theta'|m) d\theta'}$

### 4.2 Summarizing posterior distributions

#### 4.2.1 MAP (Maximum A Posteriori) estimation

Although most appropriate choice for:

- Real valued quantity  $\rightarrow$  *posterior median or mean*
- Discrete  $\rightarrow$  *vector of posterior marginals*

The most popular choice is *posterior mode* aka **MAP**, because it reduces to optimization problems for which efficient algorithms often exist.  
Some point to be aware about MAP:

- No measure of uncertainty
- Plugging in the MAP estimate can result in overfitting
- The mode is an untypical point, unlike the mean or median the mode is a point of measure 0, it does not take the volume of the space into account.
- MAP estimation is not invariant to reparameterization, for example passing from centimeters to inches can break things.)  
The MLE does not suffer from this since the likelihood is a function not a probability density

### 4.2.2 Credible intervals

With point estimates, we want a measure of confidence.

$$C_\alpha(\mathcal{D}) = (l, u) : \mathbb{P}(\{l \leq \theta \leq u | \mathcal{D}\}) \geq 1 - \alpha$$

In general, credible intervals are usually what people want to compute but confidence intervals are usually what they actually compute, because most people are taught frequentist statistics but not Bayesian statistics.

Sometimes with central intervals there might be points be outside the CI which have higher probability density.

More formally  $p^*$  such that:

$$1 - \alpha = \int_{\theta: p(\theta | \mathcal{D}) > p^*} p(\theta | \mathcal{D}) d\theta$$

Then the HPD such that:

$$\mathcal{D} = \{\theta : p(\theta | \mathcal{D}) \geq p^*\}$$

## 4.3 Bayesian Model Selection

A more efficient approach than cross-validation, meaning fitting  $k$  times each model, is **to compute the posterior over models**.

$$p(m | \mathcal{D}) = \frac{p(\mathcal{D} | m)p(m)}{\sum_{m \in \mathcal{M}} p(m, \mathcal{D})}$$

From this we can compute the **MAP model**  $\hat{m} = \arg \max_m p(m | \mathcal{D})$

Then we have the **marginal likelihood**:  $p(\mathcal{D} | \hat{m}) = \int p(\mathcal{D} | \theta)p(\theta | \hat{m})d\theta$ . This equation means that considering the hyper-parameter  $\hat{m}$ , that governs value taken by parameter  $\theta$  we do the weighted sum of likelihood over all possible values of  $\theta$  with a weight of  $p(\theta | \hat{m})$

### 4.3.1 Bayesian Occam's razor

In integrating out the parameters rather than maximizing them we are automatically protected from overfitting: model with more parameters do not necessarily have higher marginal likelihood.

A way to understand the Bayesian Occam's razor effect is to remember that probabilities must sum to one, meaning  $\sum_{\mathcal{D}'} p(\mathcal{D}' | m) = 1$ . Complex models, which can predict many things, must spread their probability mass thinly, and hence will not obtain as large a probability for any given data set as simpler models.



### 4.3.2 Computing the marginal likelihood (evidence)

**Use of conjugate prior** For a fixed model we often write:

$$p(\boldsymbol{\theta}|\mathcal{D}, m) \propto p(\mathcal{D}|\boldsymbol{\theta}, m) \times p(\boldsymbol{\theta}|m)$$

We ignore the normalization constant  $\mathbb{P}(\mathcal{D}|m)$  as it is constant with reference to  $\boldsymbol{\theta}$ . However when comparing models we need to know how to compute the marginal likelihood,  $p(\mathcal{D}|m)$ . In general this can be quite hard, since we have to integrate over all possible parameter values, but when we have a conjugate prior, it is easy to compute.

Let  $p(\boldsymbol{\theta}) = \frac{q(\boldsymbol{\theta})}{Z_0}$  be our prior, where  $q(\boldsymbol{\theta})$  is an **unnormalized distribution**, and  $Z_0$  is the normalization constant of the prior. Let  $p(\mathcal{D}|\boldsymbol{\theta}) = \frac{q(\mathcal{D}|\boldsymbol{\theta})}{Z_l}$  be the likelihood, where  $Z_l$  contains any constant factors in the likelihood. Finally let  $p(\boldsymbol{\theta}|\mathcal{D}) = \frac{q(\boldsymbol{\theta}|\mathcal{D})}{Z_N}$  be our posterior where  $q(\boldsymbol{\theta}|\mathcal{D}) = q(\mathcal{D}|\boldsymbol{\theta})q(\boldsymbol{\theta})$  is the **unnormalized posterior**, and  $Z_N$  is the normalization constant of the posterior.

$$\text{We have: } \begin{cases} p(\boldsymbol{\theta}|\mathcal{D}) &= \frac{p(\mathcal{D}|\boldsymbol{\theta})p(\boldsymbol{\theta})}{p(\mathcal{D})} \\ \frac{q(\boldsymbol{\theta}|\mathcal{D})}{Z_N} &= \frac{\frac{q(\mathcal{D}|\boldsymbol{\theta})q(\boldsymbol{\theta})}{Z_l Z_0 p(\mathcal{D})}}{\frac{Z_N}{Z_0 Z_l}} \\ p(\mathcal{D}) &= \frac{Z_N}{Z_0 Z_l} \end{cases}$$

**BIC and AIC** Simpler approach

- **BIC** In general  $p(\mathcal{D}|m) = \int p(\mathcal{D}|\boldsymbol{\theta})p(\boldsymbol{\theta}|m)d\boldsymbol{\theta}$  can be quite difficult to compute. A popular approximation is:  $BIC \triangleq \log(p(\mathcal{D}|\hat{\boldsymbol{\theta}}_{MLE})) - \frac{\text{dof}(\hat{\boldsymbol{\theta}}_{MLE})}{2} \log(N) \approx \log p(\mathcal{D})$
- **AIC**:  $AIC(m, \mathcal{D}) \triangleq \log(p(\mathcal{D}|\hat{\boldsymbol{\theta}}_{MLE})) - \text{dof}(m)$   
This is derived from Frequentist framework and cannot be interpreted as an approximation to the marginal likelihood. The penalty of AIC is less than BIC, it causes AIC pick more complex models. That **can be better for predictive accuracy**.

**Use of Empirical Bayes** The way to chose the prior is not always clear, for posterior inference it is not an issue as the likelihood often overwhelm the prior, but when computing the marginal likelihood the prior plays a much more important role. Indeed, we are averaging likelihood over all possible parameter settings as weighted by the prior.

If the prior is unknown the correct Bayesian procedure is to to **put a prior on the prior** (hierarchical Bayes).

In the case of a linear regression we can use a prior of the form  $\mathbb{P}(\mathbf{w}) = \mathcal{N}(\mathbf{0}, \alpha^{-1}\mathbf{I})$ , then

$$\mathbb{P}(\mathcal{D}|m) = \int \int \mathbb{P}(\mathcal{D}|\mathbf{w}) \mathbb{P}(\mathbf{w}|\alpha, m) \mathbb{P}(\alpha|m) d\mathbf{w} d\alpha$$

**The higher up we go in the Bayesian hierarchy the less sensitive are the results to the prior settings.** A computational shortcut is to optimize  $\alpha$  rather than integrating it out. That is, we use

$p(\mathcal{D}|m) \approx \int p(\mathcal{D}|\mathbf{w})p(\mathbf{w}|\hat{\alpha}, m)d\mathbf{w}$ . where  $\hat{\alpha} = \arg \max_{\alpha} p(\mathcal{D}|\alpha, m) = \arg \max_{\alpha} \int p(\mathcal{D}|\mathbf{w})p(\mathbf{w}|\alpha, m)d\mathbf{w}$  This is Empirical Bayes.

### 4.3.3 Bayes Factors

When prior on models is uniform, then model selection is equivalent to picking the model with the highest marginal likelihood. Now suppose we just have two models we are considering, call them the null hypothesis,  $M_0$  and the alternative hypothesis,  $M_1$ .

$$BF_{1,0} \triangleq \frac{p(\mathcal{D}|M_1)}{p(\mathcal{D}|M_0)} = \frac{\frac{p(M_1|\mathcal{D})}{p(M_0|\mathcal{D})}}{\frac{p(M_1)}{p(M_0)}} \frac{\text{Posterior odds}}{\text{Prior odds}}$$

- *Posterior odds*: quantifies relative plausibility of the rival hypotheses **after** having seen the data.
- *Bayes Factor*,  $BF_{1,0}$ , quantifies the evidence provided by the data, this is like a likelihood ratio, except we integrate out the parameters, which allows us to compare models of different complexity.
- *Prior odds*: quantifies relative plausibility of the rival hypotheses **before** seeing the data.

| Bayes Factor $BF(1, 0)$           | Interpretation              |
|-----------------------------------|-----------------------------|
| $BF < \frac{1}{100}$              | Decisive evidence for $M_0$ |
| $BF < \frac{1}{10}$               | Strong evidence for $M_0$   |
| $\frac{1}{10} < BF < \frac{1}{3}$ | Modest evidence for $M_0$   |
| $\frac{1}{3} < BF < 1$            | Weak evidence for $M_0$     |
| $1 < BF < 3$                      | Weak evidence for $M_1$     |
| $3 < BF < 10$                     | Modest evidence for $M_1$   |
| $BF > 10$                         | Strong evidence for $M_1$   |
| $BF > 100$                        | Decisive evidence for $M_1$ |

#### 4.3.4 Jeffreys-Lindley paradox

Problems can arise when we use improper priors (i.e. priors that do not integrate to 1) for model selection/hypothesis testing, even though such priors may be acceptable for other purposes. In particular the Bayes Factor will always favor the simplest model since the probability of the observed data under a complex model with a very diffuse prior will be very small. Thus it is important to use proper priors when doing model selection.

### 4.4 Priors

The most controversial aspect of Bayesian statistics is its reliance on priors

#### 4.4.1 Uninformative priors

If we do not have strong evidence on what  $\theta$  should be, it is common to use an uninformative priors, to "let the data speak for itself".

One might think that, for Bernoulli parameter  $\theta$  the most uninformative prior would be the uniform distribution:  $Beta(1, 1)$ , but the posterior would then be:  $\mathbb{E}(\theta|\mathcal{D}) = \frac{N_1 + 1}{N_1 + N_0 + 2}$ , whereas the MLE is  $\frac{N_1}{N_1 + N_0}$ .

As by decreasing the magnitude of the pseudo counts, we can lessen the impact of the prior, we can argue that the most non-informative prior is:

$$\lim_{\epsilon \rightarrow 0} Beta(\epsilon, \epsilon) = Beta(0, 0)$$

Called the *Haldane prior*, it is an improper prior.

In general it is advisable to perform a some kind of **sensitivity analysis**, in which one checks how much one's conclusions or prediction change in response to change in the modelling assumptions which includes the choice of the prior and the likelihood as well. If the conclusion are relatively insensitive to the modelling assumption, one can have more confidence in the results.

#### 4.4.2 Jeffreys priors

Harold Jeffreys designed a general purpose technique for creating non-informative priors. The key observation is that if  $p(\phi)$  is non-informative then any re-parametrization of the prior, such as  $\theta = h(\phi)$  for some function  $h$  should also be non-informative.

- Start with a variable change:  $p_\theta(\theta) = p_\phi(\phi) \left| \frac{d\phi}{d\theta} \right|$

- Consider the following constraint:  $p_\phi(\phi) \propto \sqrt{\mathcal{I}(\phi)}$ , where  $\mathcal{I}(\phi)$  is the Fisher information.  
 $\mathcal{I}(\phi) \triangleq -\mathbb{E} \left( 2 \times \frac{d \log(p(X|\phi))}{d\phi} \right)$ . This is a measure of the curvature of the expected negative log likelihood and hence a [measure of stability of the MLE](#).
- Now  $\frac{d \log(p(x|\theta))}{d\theta} = \frac{d \log(p(X|\phi))}{d\phi} \frac{d\phi}{d\theta}$
- $\mathcal{I}(\theta) = \mathcal{I}(\phi) \left( \frac{d\phi}{d\theta} \right)^2$
- $\sqrt{\mathcal{I}(\theta)} = \sqrt{\mathcal{I}(\phi)} \left| \frac{d\phi}{d\theta} \right|$
- Finally  $p_\theta(\theta) = p_\phi(\phi) \left| \frac{d\phi}{d\theta} \right| \propto \sqrt{\mathcal{I}(\phi)} \left| \frac{d\phi}{d\theta} \right| = \sqrt{\mathcal{I}(\theta)}$

#### 4.4.3 Robust priors

To prevent an undue influence on the result, we build [priors having heavy tails](#), which avoids forcing things to be too close to the prior mean.

#### 4.4.4 Mixture of conjugate priors

Conjugate priors simplify the computation of robust priors, but are often not robust, and not flexible enough to encode our prior knowledge. However [it turns out that a mixture \(linear combination\) of conjugate priors is also conjugate, and seem to be a good compromise](#).

We can represent a mixture by [introducing a latent indicator variable  \$z\$](#) , where  $z = k$  means that  $\theta$  comes from mixture component  $k$ .

### 4.5 Hierarchical and Empirical Bayes

#### 4.5.1 Hierarchical Bayes

A key requirement for computing the posterior  $p(\theta|\mathcal{D})$  is the specification of a prior  $p(\theta|\eta)$  where  $\eta$  are the hyper-parameters. A Bayesian approach is to [put a prior on our priors](#). This is an example of a **hierarchical Bayesian Model**.

#### 4.5.2 Empirical Bayes

In hierarchical Bayesian models, we need to compute the posterior on multiple levels of latent variables. For example, in a two-level model, we need to compute:  $p(\eta, \theta|\mathcal{D}) \propto p(\mathcal{D}|\theta)p(\theta|\eta)p(\eta)$

[We can approximate the posterior on the hyper-parameters with a point-estimate,  \$p\(\eta|\mathcal{D}\) \approx \delta\_{\hat{\eta}}\(\eta\)\$  where  \$\hat{\eta} = \arg \max\_{\eta} p\(\eta|\mathcal{D}\)\$ . Since  \$\eta\$  is typically much smaller than  \$\theta\$  in dimensionality, it is less prone to overfitting, so we can safely use a uniform prior on  \$\eta\$ . Then the estimate becomes:](#)

$$\hat{\eta} = \arg \max_{\eta} p(\mathcal{D}|\eta) = \arg \max_{\eta} \int p(\mathcal{D}|\theta)p(\theta|\eta)d\theta$$

This overall approach is called **Empirical Bayes**

Empirical Bayes violates the principle that the prior should be chosen independently of the data. However, we can just view it as a computationally cheap approximation to inference in a hierarchical Bayesian model, just as we viewed MAP estimation as an approximation to inference in the one level model  $\theta \rightarrow \mathcal{D}$ . In fact, we can construct a hierarchy in which the more integrals one performs, the "more Bayesian" one becomes:

| Method                  | Definition                                                                                                                           |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Maximum likelihood      | $\hat{\theta} = \arg \max_{\theta} p(\mathcal{D} \theta)$                                                                            |
| MAP estimation          | $\hat{\theta} = \arg \max_{\theta} p(\mathcal{D} \theta)p(\theta \eta)$                                                              |
| ML-II (Empirical Bayes) | $\hat{\eta} = \arg \max_{\eta} \int p(\mathcal{D} \theta)p(\theta \eta)d\theta = \arg \max_{\eta} p(\mathcal{D} \eta)$               |
| MAP-II                  | $\hat{\eta} = \arg \max_{\eta} \int p(\mathcal{D} \theta)p(\theta \eta)p(\eta)d\theta = \arg \max_{\eta} p(\mathcal{D} \eta)p(\eta)$ |
| Full Bayes              | $p(\theta, \eta \mathcal{D}) \approx p(\mathcal{D} \theta)p(\theta \eta)p(\eta)$                                                     |

## 4.6 Bayesian Decision Theory

We can formalize any given [statistical decision problem](#) as a [game against nature](#) (as opposed to a game against other strategic players, which is the topic of game theory). In this game, nature picks a state or parameter  $\theta$  or [label](#),  $y \in \mathcal{Y}$ , [unknown to us](#), and then [generates an observation](#),  $\mathbf{x} \in \mathcal{X}$  which we get to see. We then have to make a decision, that is, [we have to choose an action](#)  $a$  from some **action space**  $\mathcal{A}$ . Finally we incur some **loss**,  $L(y, a)$ , which [measures how compatible our action  \$a\$  is with nature's hidden state  \$y\$](#) .

Our goal is [to derive a decision procedure or policy](#),  $\delta : \mathcal{X} \rightarrow \mathcal{A}$  which [specifies the optimal action for each possible input](#) which specifies the optimal action for each possible input, meaning the [action that minimizes the expected loss](#):

$$\delta(\mathbf{x}) = \arg \min_{a \in \mathcal{A}} \mathbb{E}(L(y, a))$$

In the Bayesian vision, we mean the expected value of  $y$  given the data we have seen so far, whereas in the frequentist vision we mean the expected value refers to  $x$  and  $y$  that we expect to see in the future. In the Bayesian vision the optimal action having observed  $\mathbf{x}$  is defined as [the action  \$a\$  that minimizes the posterior expected loss](#):

$$\rho(a|\mathbf{x}) \triangleq \mathbb{E}_{p(y|\mathbf{x})}(L(y, a)) = \sum_y L(y, a)p(y|\mathbf{x})$$

Hence the [Bayes estimator also called Bayes decision rule](#) is given by:

$$\delta(\mathbf{x}) = \arg \min_{a \in \mathcal{A}} \rho(a|\mathbf{x})$$

Note that here [we adopted a predictive point of view](#), as we took  $y$  as reference otherwise it would have been a inferential point of view in taking  $\theta$ .

### 4.6.1 Bayes estimators for common loss functions

- **MAP** estimate minimizes 0-1 loss:  $L(y, a) = \mathbb{1}_{\{y \neq a\}} \begin{cases} 0 & \text{if } a = y \\ 1 & \text{else} \end{cases}$
- **Reject option**, in classification problems where  $p(y|\mathbf{x})$  is very uncertain we may prefer to choose a reject action, in which we refuse to classify the example as any of the specified classes. Let choosing  $a = C + 1$  correspond to picking the reject action, and choosing  $a \in \{1, \dots, C\}$  correspond to picking one of the classes.

$$L(y = j, a = i) = \begin{cases} 0 & \text{if } i = j \text{ and } i, j \in \{1, \dots, C\} \\ \lambda_r & \text{if } i = C + 1 \\ \lambda_s & \text{otherwise} \end{cases}$$

where  $\lambda_r$  is the cost of the reject action, and  $\lambda_s$  is the cost of a substitution error.

- **Squared Error** ( $l_2$ ) for a continuous parameters.  $L(y, a) = (y - a)^2$
- **Absolute Error** ( $l_1$ ) more robust against outliers.  $L(y, a) = |y - a|$ . The optimal point is the median.

- **Supervised learning** considering a prediction function  $\delta : \mathcal{X} \rightarrow \mathcal{Y}$  and some cost function  $l(y, \delta(x))$ . Then the loss incurred by taking action  $\delta$  when the unknown state of nature is  $\theta$  (the parameters of the data generating the mechanism).  $L(\theta, \delta) \triangleq \mathbb{E}_{(\mathbf{x}, y) \sim p(\mathbf{x}, y|\theta)} (l(y, \delta(\mathbf{x}))) = \sum_{\mathbf{x}} \sum_y L(y, \delta(\mathbf{x}) p(\mathbf{x}, y|\theta))$

#### 4.6.2 Model evaluation metrics

- **False positive vs False negative trade-off** for binary decision problems there are 2 types of errors:
  1. false positive (false alarm) if  $\hat{y} = 1 \wedge y = 0$
  2. false negative (missed detection) if  $\hat{y} = 0 \wedge y = 1$

We can consider the loss matrix:

| Headers       | $y = 1$  | $y = 0$  |
|---------------|----------|----------|
| $\hat{y} = 1$ | 0        | $L_{FP}$ |
| $\hat{y} = 0$ | $L_{FN}$ | 0        |

where  $L_{FN}$  is the cost of a false negative and  $L_{FP}$  the cost of a false positive.

- **Precision recall curves** When trying to detect a rare event the number of negatives is very large, hence comparing *sensitivity* and *the error of type I* is not very informative. We would then like to use a measure that only talks about positives.

$$\text{precision} = \frac{TP}{\hat{N}_+}$$

$$\text{recall} = \frac{TP}{N_+}$$

A **precision recall curve** is a plot of *precision* vs *recall*.

- **F-scores** is the *harmonic mean of precision and recall*:

$$F_1 \triangleq \frac{2}{\frac{1}{\text{precision}} + \frac{1}{\text{recall}}}$$

**Receiver Operating Characteristic (ROC) curve** From the below table

| Headers  | Truth           |                 | Count                                   |
|----------|-----------------|-----------------|-----------------------------------------|
| Estimate | 1               | $TP$            | $\hat{N}_+ = TP + FP$                   |
|          | 0               | $FN$            | $\hat{N}_- = FN + TN$                   |
| Count    | $N_+ = TP + FN$ | $N_- = FP + TN$ | $N = N_+ + N_- = \hat{N}_+ + \hat{N}_-$ |

we can generate the *confusion matrix* is the below table

| Headers       | $y = 1$                                          | $y = 0$                                    |
|---------------|--------------------------------------------------|--------------------------------------------|
| $\hat{y} = 1$ | $\frac{TP}{N}$ (sensitivity/recall)              | $\frac{FP}{N}$ (error type I/ false alarm) |
| $\hat{y} = 0$ | $\frac{FN}{N}$ (error type II/ missed detection) | $\frac{TN}{N}$ (specificity)               |

Consider  $f(\mathbf{x})$  a *measure of our confidence that  $y = 1$*  like  $f = \sigma$ , however it does not need to be probabilistic simply being monotonically related to  $\mathbb{P}(y = 1|\mathbf{x})$ . Let  $\tau$  be *some threshold parameter*, then our classification rule is  $\mathbb{1}_{\{f(\mathbf{x}) > \tau\}}$ . We can then compute TPR(True Positive Rate) also known as sensibility recall by using  $TPR = \frac{TP}{N_+} \approx \mathbb{P}(\hat{y} = 1|y = 1)$ . As well we are able to compute FPR(False Positive Rate) by using  $FPR = \frac{FP}{N_-} \approx \mathbb{P}(\hat{y} = 1|y = 0)$

ROC is achieved by computing the above TPR and FPR for different value of  $\tau$  Note that

- classifying everything as negative ( $\tau = 1$ ) leads to  $TPR = 0$  and  $FPR = 0$
- classifying everything as positive ( $\tau = 0$ ) leads to  $TPR = 1$  and  $FPR = 1$
- a system working randomly will be close to the diagonal line  $TPR = FPR$

AUC (Area Under the Curve) summarize the ROC curve

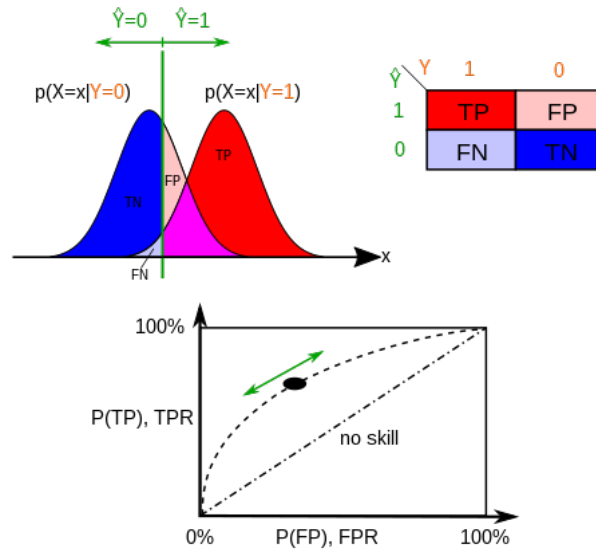


Figure 4.1: Receiver Operating Characteristic curve with distribution illustration

In binary classification the class prediction for each instance is often made based on a continuous random variable  $X$ , which is the score computed for the instance (the estimated probability in logistic regression).

$X$  follows a probability density  $f_1(x)$  if the instance belongs to the class "positive" and  $f_0(x)$  otherwise.

Therefore  $\begin{cases} TPR(\tau) = \int_{\tau}^{\infty} f_1(x)dx \\ FPR(\tau) = \int_{\tau}^{\infty} f_0(x)dx \end{cases}$  and ROC curve plots parametrically  $TPR(\tau)$  versus  $FPR(\tau)$  with  $\tau$  a varying parameter.

Note that this  $\tau$  is fundamentally the same that the one above which was squash in the interval  $[0, 1]$

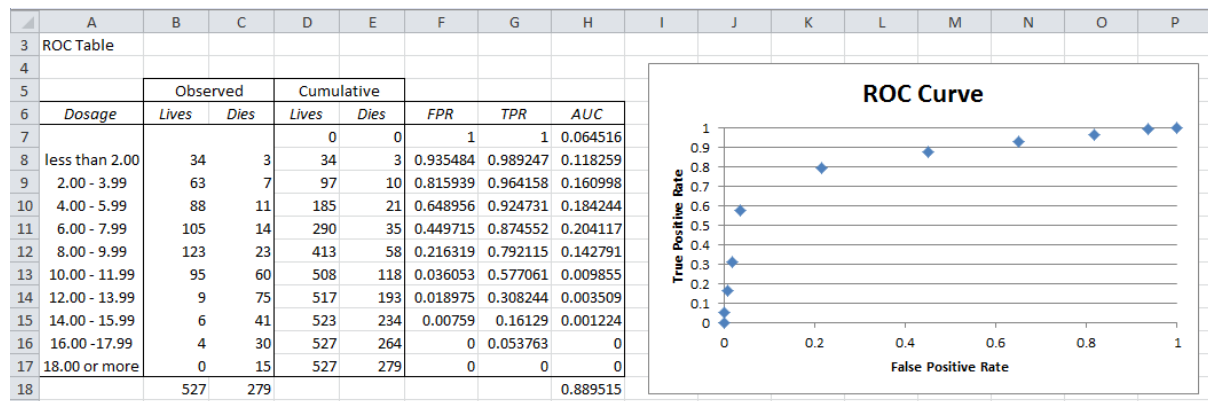


Figure 4.2: Receiver Operating Characteristic curve

In the above table we have the following formula :

- $D9 : SUM(\$B\$7 : B9) \rightarrow$  Cumulative Lives
- $E9 : SUM(\$C\$7 : C9) \rightarrow$  Cumulative Dies
- $F9 : 1 - D9/\$D\$17 \rightarrow$  False Positive Rate  $(1 - \frac{TN}{N_+})$
- $G9 : 1 - E9/\$E\$17 \rightarrow$  True Positive Rate  $(1 - \frac{FN}{N_-})$

- $H9: (F10 - F9) * G9 \rightarrow \text{Area Under the Curve}$

# Chapter 5

## Frequentist approach

### 5.1 Sampling distribution

#### 5.1.1 Sampling Distributions of an estimator

In frequentist statistic a parameter estimate  $\hat{\theta}$  is computed by applying an estimator  $\delta$  to some data  $\mathcal{D}$ , so  $\hat{\theta} = \delta(\mathcal{D})$ . The uncertainty in the parameter estimate can be measured by computing the *sampling distribution of the estimator*. Imagine sampling many different datasets  $\mathcal{D}^{(s)}$  from some true model  $p(\cdot|\theta^*)$  meaning  $\mathcal{D}^{(s)} = \{x_i^{(s)} \hookrightarrow p(\cdot|\theta^*)\}_{1 \leq i \leq N}$  for  $1 \leq s \leq S$  and  $\theta^*$  is the true parameter. Now apply the estimator  $\hat{\theta}(\cdot)$  to each  $\mathcal{D}^{(s)}$  to get a set of estimates  $\{\delta(\mathcal{D}^{(s)})\}_{1 \leq s \leq S}$ . As we let  $S \rightarrow \infty$ , the distribution induced on  $\hat{\theta}(\cdot)$  is the *sampling distribution of the estimator*.

#### 5.1.2 Bootstrap

It is a *simple Monte Carlo technique to approximate the sampling distribution*. The idea is that if we knew the true parameters  $\theta^*$ , we could generate  $S$  fake datasets of size  $N$ , from the true distribution. We could then compute our estimator from each sample, and use the empirical distribution of the resulting samples as our estimate of the sampling distribution.

Since  $\theta^*$  is unknown, the idea of the *parametric bootstrap* is to generate the samples using  $\hat{\theta}(\mathcal{D})$  instead. An alternative, called *non-parametric bootstrap* is to sample the  $x_i^s$  (with replacement) from the original data  $\mathcal{D}$  and then compute the induced distribution as before.

### 5.2 Frequentist decision theory

In Frequentist decision theory there is a loss function and a likelihood, but there is no prior and hence no posterior or posterior expected loss. Thus there is no automatic way of deriving an optimal estimator, unlike the Bayesian case.

Instead, we are free to choose any estimator or decision procedure  $\delta : \mathcal{X} \rightarrow \mathcal{A}$  we want.

Having chosen an estimator, we define its *expected loss or risk* as follows

$$R(\theta^*, \delta) \triangleq \mathbb{E}_{p(\tilde{\mathcal{D}}|\theta^*)} (L(\theta^*, \delta(\tilde{\mathcal{D}}))) = \int L(\theta^*, \delta(\tilde{\mathcal{D}})) p(\tilde{\mathcal{D}}|\theta^*) d\tilde{\mathcal{D}}$$

where  $\tilde{\mathcal{D}}$  is data sampled from 'nature's distribution' represented by parameter  $\theta^*$ .

Whereas the *Bayesian posterior expected loss*:

$$p(a|\mathcal{D}, \pi) \triangleq \mathbb{E}_{p(\theta|\mathcal{D}, \pi)} (L(\theta, a)) = \int_{\Theta} L(\theta, a) p(\theta|\mathcal{D}, \pi) d\theta$$

We see that the Bayesian approach averages over  $\theta$ , which is unknown, and conditions on  $\mathcal{D}$  which is known. Unlike the frequentist approach averages over  $\tilde{\mathcal{D}}$ , thus ignoring the observed data, and conditions on  $\theta^*$  which is unknown.



### 5.2.1 Bayes risk

How to choose amongst the estimators? We need some way to convert  $R(\theta^*, \delta)$  into single measure of quality,  $R(\delta)$  which does not depend on knowing  $\theta^*$ . One approach is to put a prior on  $\theta^*$  and then to define **Bayes risk** of an estimator as follows:

$$R_B(\delta) \triangleq \mathbb{E}_{p(\theta^*)} (R(\theta^*, \delta)) = \int R(\theta^*, \delta) p(\theta^*) d\theta^*$$

A **Bayes estimator** or **Bayes decision rule** is one which minimizes the expected risk:  $\delta_B \triangleq \arg \min_{\delta} R_B(\delta)$

**Connection Bayesian and Frequentist approaches to decision theory.**

- *Theorem 1* A Bayes estimator can be obtained by minimizing the posterior expected loss for each  $x$
- *Theorem 2* Every admissible frequentist decision rule is a Bayes decision rule with respect to some possibly improper prior distribution.

**Minimax risk** Some frequentist statistic users avoid using Bayes risk since it requires the choice of a prior, although this is only in the evaluation of the estimator, not necessarily as part of its construction. An alternative approach is as follows:

1. Define the maximum risk of an estimator as:

$$R_{\max}(\delta) \triangleq \max_{\theta^*} R(\theta^*, \delta)$$

2. A **minimax rule** is one which minimizes the maximum risk:  $\delta_{MM} \triangleq \arg \min_{\delta} R_{\max}(\delta)$

Minimax estimators have a certain appeal, however computing them can be hard and furthermore they are very pessimistic. In most statistical situations, excluding games theoretic ones, assuming nature is an adversary is not a reasonable assumption.

### 5.2.2 Admissible estimators

The basic problem with frequentist decision theory is that it relies on knowing the true distribution  $p(\cdot|\theta^*)$  in order to evaluate the risk. However it might be the case that some estimators are worse than others regardless of the value of  $\theta^*$ .

In particular if for  $\theta \in \Theta$ ,  $R(\theta, \delta_1) \leq R(\theta, \delta_2)$  then we say that  $\delta_1$  **dominates**  $\delta_2$ .

An estimator is said to be **admissible** if it is not strictly dominated by any other estimator.

**Admissibility is not enough**

## 5.3 Desirable properties of estimators

### 5.3.1 Consistent estimators

An estimator is said to be **consistent** if it eventually recovers the true parameters that generated the data as the sample size goes to infinity.

### 5.3.2 Unbiased estimator

The **bias** of an estimator is defined as

$$\text{bias}(\hat{\theta}(\cdot)) = \mathbb{E}_{p(\mathcal{D}|\theta^*)} (\hat{\theta}(\mathcal{D}) - \theta^*)$$

The estimator is **unbiased** when the bias is equal to 0.

### 5.3.3 Minimum variance estimators

A famous result called the **Cramerè-Rao lower bound** provides a lower bound on the variance of any unbiased estimator. More precisely: Let  $(X_j)_{1 \leq j \leq p} \hookrightarrow p(X|\theta_0)$  and  $\hat{\theta}(\cdot)$  an unbiased estimator of  $\theta^*$ . Then, under various smoothness assumptions on  $p(X|\theta_0)$  we have

$$\mathbb{V}(\hat{\theta}) \geq \frac{1}{nI(\theta^*)}$$

where  $I(\theta^*)$  is the Fisher information matrix.

### 5.3.4 Bias-Variance Trade-off

As  $MSE = variance + bias^2$

It might be wise to use a biased estimator, so long as it reduces our variance, assuming our goal is to minimize squared error.

## 5.4 Empirical Risk Minimization

### 5.4.1 Frequentist issue

Frequentist decision theory suffers from the fundamental problem that one cannot actually compute the risk function, since it relies on knowing the true data distribution. By contrast, the Bayesian posterior expected loss can always be computed since it conditions on the data rather than on  $\theta^*$ .

However there is one setting which avoids this problem, it is when the task is to predict observable quantities, as opposed to estimating hidden variables or parameters.

Instead of looking at loss functions of the form  $L(\theta^*, \delta(\mathcal{D}))$  let us look at loss functions of the form  $L(y, \delta(\mathbf{x}))$ .

Then the risk becomes:  $R(p_*, \delta) \triangleq \mathbb{E}_{(\mathbf{x}, y) \hookrightarrow p_*} (L(y, \delta(\mathbf{x}))) = \sum_{\mathbf{x}} \sum_y L(y, \delta(\mathbf{x})) p_*(\mathbf{x}, y)$  Where  $p_*$  represents "nature's distribution", indeed this distribution is unknown, but a simple approach is to use the empirical distribution, derived from some training data to approximate  $p_*(x, y) \approx p_{emp}(x, y) \triangleq$

$\frac{1}{N} \sum_{i=1}^N \delta_{x_i}(\mathbf{x}) \delta_{y_i}(y)$  We define the empirical risk as follows:

$$R_{emp}(\mathcal{D}, \delta) \triangleq R(p_{emp}, \delta) = \frac{1}{N} \sum_{i=1}^N L(y_i, \delta(\mathbf{x}_i))$$

### 5.4.2 Regularized risk minimization

$$R'(\mathcal{D}, \delta) = R_{emp}(\mathcal{D}, \delta) + \lambda C(\delta)$$

where  $C(\delta)$  measures the complexity of the prediction function  $\delta(\mathbf{x})$  and  $\lambda$  controls the strength of the complexity penalty. This approach is known as **regularized risk minimization**.

## 5.5 Components

### 5.5.1 Introduction

Avoid treating parameters as random variables. The notion of variation across repeated trials forms the basis for modelling uncertainty.

### 5.5.2 Hypothesis Testing

A frequentist statistics, probabilities represent the frequencies at which particular events happen.

### 5.5.3 *p-value*

It is the heart of frequentist hypothesis testing, it tells us the [probability of getting a particular test statistic  \$t\$  as big as the one we have or bigger under the null hypothesis](#) (that there is actually no effect). By convention we usually conclude an effect is *statistically significant* if the *p-value* is less than a threshold  $\alpha$ .

### 5.5.4 Confidence intervals

When we fit a model to our data we look for the *maximum of likelihood* parameters, meaning the parameters that are most consistent with our data. For each parameter we will be able to construct 95% interval namely [95 of the 100 intervals generated will contain the true value of the parameter](#).

If  $H_0 : \beta = 0$  is true, the probability of getting a 95% confidence interval that does not include 0 is less than 0.05. In other words, if the 95% confidence does not include 0,  $p < 0.05$ .

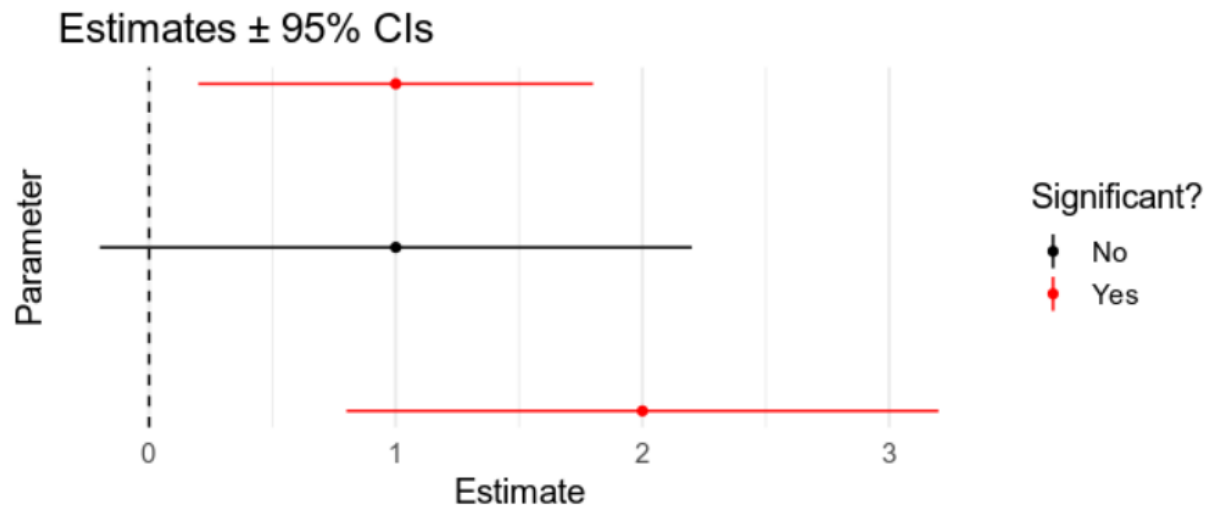


Figure 5.1: Confidence interval

### 5.5.5 Multiple comparisons

The more tests we run the more likely it is to we'll find at least one that is significant even though the null hypothesis is true. We can then apply a Bonferroni correction.

Let's say we are running  $k$  tests, we can either adjust:

- the threshold  $\alpha_{adj} = \frac{\alpha}{k}$  OR
- the *p-value*  $p_{adj} = k \times p$

## 5.6 Power Analysis

It has as general purpose [to find the right sample number](#).

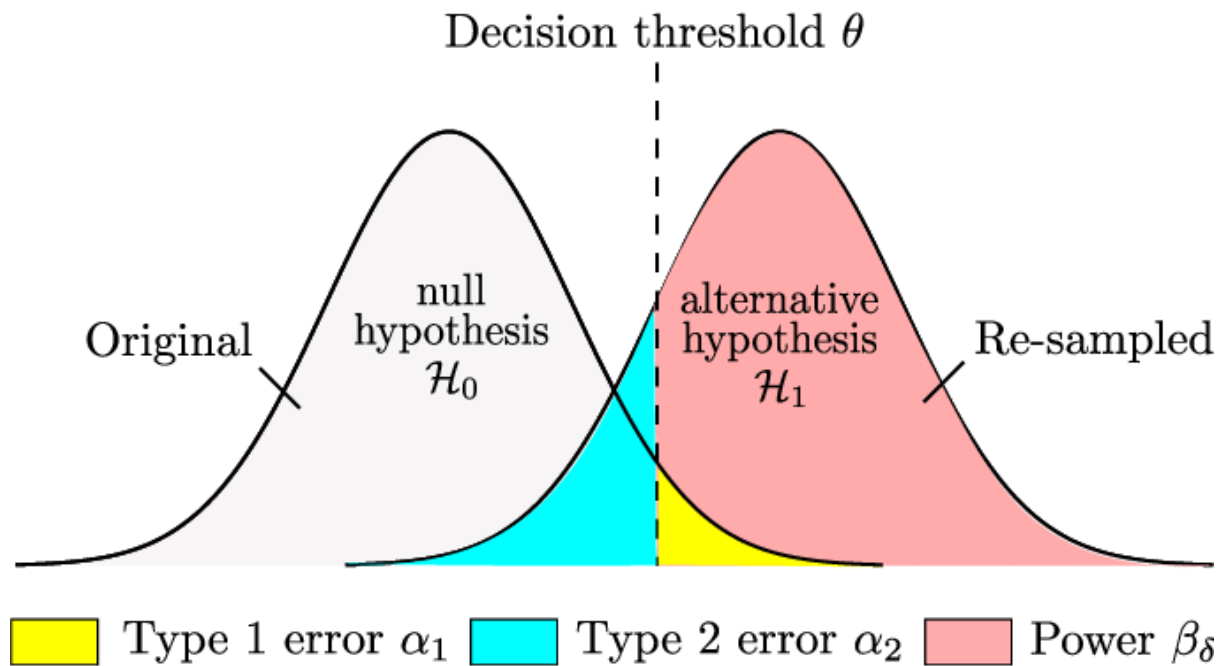


Figure 5.2: caption

|                     | $H_0$ is True              | $H_1$ is True              |
|---------------------|----------------------------|----------------------------|
| Do not reject $H_0$ | Right decision             | Type II Error $\sim \beta$ |
| Reject $H_0$        | Type I Error $\sim \alpha$ | Right decision             |

### 5.6.1 Power of the test

Start by defining:  $Power = 1 - \beta$ , considering  $H_1$  true it is the probability to correctly reject  $H_0$

### 5.6.2 Significant threshold

Then propose  $\alpha$ , the probability to wrongly reject  $H_0$ . It will be the reference to which the  $p$ -value will be compared, the statistical test will be significant ( $H_0$  rejected) if  $p\text{-value} \leq \alpha$ .

### 5.6.3 Effect size

It quantifies how meaningful the relationship between variables or the difference between group is, it indicates a practical significance.

While statistical significance ( $p$ -value) shows the existence of an effect, practical significance ( $effect\ size$ ) shows if this effect is large enough to be meaningful in the real world.

There are dozens of measures for effect sizes, and the most common are *Cohen's d* and *Pearson's r*.

# Chapter 6

## Common statistical tests

### 6.1 Use of statistical tests

#### 6.1.1 Terms

- *Paired* samples: one-to-one correspondence between data in the first and second set.
- *Matched* samples: every subject in one group with an equivalent in another.

#### 6.1.2 Table of statistical hypothesis test

Statistical method table.

|                                                               | Binomial/Discrete                                    | Continuous, from Normal distribution            | Continuous measurement (Score/Rank), from non-Normal distribution |
|---------------------------------------------------------------|------------------------------------------------------|-------------------------------------------------|-------------------------------------------------------------------|
| <b>Example of data sample</b>                                 | List of patients recovering or not after a treatment | Reading of heart pressure from several patients | Ranking of several treatment efficiency                           |
| <b>Describe one data sample</b>                               | Proportions                                          | Mean, Standard Deviation                        | Median                                                            |
| <i>Compare one data sample to a hypothetical distribution</i> | $\chi^2$ / <i>G</i> -test or Binomial test           | 1-sample t-test                                 | Sign test or Wilconox test                                        |
| <i>Compare 2 paired samples</i>                               | Sign test                                            | Paired t-test                                   | Sign test or Wilconox test                                        |
| <i>Compare 2 unpaired samples</i>                             | $\chi^2$ / <i>G</i> -test or Fisher's extract test   | Unpaired t-test                                 | Mann-Whitney test                                                 |
| <i>Compare 3 or more unmatched samples</i>                    | $\chi^2$ / <i>G</i> -test                            | 1-way ANOVA                                     | Kruskal-Wallis test                                               |
| <i>Compare 3 or more matched samples</i>                      | Cochrane Q test                                      | Repeated-measures ANOVA                         | Friedman test                                                     |
| <i>Quantify association between 2 paired samples</i>          | Contingency coefficients                             | Pearson correlation                             | Spearman correlation                                              |

### 6.2 List of common statistical test

#### 6.2.1 Binomial

To check if the deviations from a theoretically expected distribution of observations into 2 categories.

#### Assumptions

- Sample items are independent.

- Items are dichotomous and nominal.
- The sample size is significantly less than the population size
- The sample is a fair representation of the population

**Frequentist** Let define a user-defined probability  $p_0$ , with  $H_0 : p = p_0$  and 
$$\begin{cases} H_1 : p \neq p_0: \text{two-tailed test} \\ H_1 : p < p_0: \text{left-tailed test} \\ H_1 : p > p_0: \text{right-tailed test} \end{cases}$$

**Bayesian** Define the prior distribution with a  $Beta(a, b)$  distribution

*Return to the table.*

### 6.2.2 $\chi^2$ test

Either [used to test goodness-of-fit or independence between 2 variables](#). It checks either if there is a significant difference between the expected and observed frequencies.

- *goodness-of-fit*: expected frequencies are computed with a theoretical relationship between observed frequencies
- *independence*: expected frequencies are computed with observed frequencies from the other sample

#### Assumptions

- simple random sample
- sample with a sufficiently large size is assumed, for small sample size see Cash test
- [expected cell count has to be adequate](#), a rule of thumb is at least 5 for 2-by-2 table and 5 or more in 80% of cells in larger tables.
- Independence of the observations

**Frequentist** 
$$\chi^2 = \sum_{i=1}^n \frac{\left(\frac{O_i}{N} - p_i\right)^2}{p_i} : \begin{cases} O_i: \text{number of observations of type } i \\ N: \text{total number of observations} \\ n: \text{number of cells in the table.} \\ p_i: \text{expected proportions of the fraction of type } i \text{ in the population.} \end{cases}$$

**Bayesian** Does not exist, see *contingency table*  
*Return to the table.*

### 6.2.3 Exact test of goodness-of-fit

Unlike the conventional statistical tests, [there is no test statistic, we directly compute the p-value under the null hypothesis](#). The most common use are for dichotomous nominal variables or multinomial variables.

#### Assumptions

- [Observations are independent](#).
- Small sample size  $\lesssim 1000$

**Frequentist** Let us define the list of, respectively, expected counts for each modality  $i$ ,  $(E_i)_{1 \leq i \leq m}$ , and observed counts  $(O_i)_{1 \leq i \leq m}$ . Then 
$$\begin{cases} H_0 : \forall i \in \llbracket 1, m \rrbracket, O_i = E_i \\ H_1 : \exists i \in \llbracket 1, m \rrbracket, O_i \neq E_i: \text{two-tailed test} \end{cases}$$

### 6.2.4 Fisher's exact test

To check the significance of the contingency between 2 kinds of classification of a given object, initially Fisher used this test to distinguish drink in which the tea has been put before the milk or vice-versa. For large sample use *G-test*

#### Assumptions

- In practice, small sample size  $\lesssim 1000$

**Frequentist** For example let's divide a population into male and female and for each persons indicating if this person is currently studying or not. We want to test if the proportion of studying students is higher among the women than among the men.

|              | Men     | Women   | Row Total           |
|--------------|---------|---------|---------------------|
| Studying     | $a$     | $b$     | $a + b$             |
| Non-Studying | $c$     | $d$     | $c + d$             |
| Column Total | $a + c$ | $b + d$ | $a + b + c + d = n$ |

The conditional on the margins of the table is distributed as *Hypergeometric*( $a + c, a + b, c + d$ ) meaning  $a + c$  draws from a population with  $a + b$  success and  $c + d$  failures. The probability of obtaining such set of values is given by

$$p = \frac{\binom{a+b}{a} \times \binom{c+d}{c}}{\binom{n}{a+c}} = \frac{\binom{a+b}{b} \times \binom{c+d}{d}}{\binom{n}{b+d}}$$

**Bayesian** Does not exist, see *contingency table*

*Return to the table.*

### 6.2.5 G-test

It's a likelihood-ratio or a maximum likelihood statistical significance test. Either used to test goodness-of-fit and independence between 2 variables. It checks either if there is a significant difference between the expected and observed frequencies. This test tends to replace  $\chi^2$ -test

- *goodness-of-fit*: expected frequencies are computed with a theoretical relationship between observed frequencies
- *independence*: expected frequencies are computed with observed frequencies from the other sample
- *repeated tests*: first variable is analysed with a goodness-of-fit and the second one represents the repetition of the experiments multiple times. Thus it allows to assess the goodness-of-fit on a large sample instead of multiple lower samples. Expected frequencies is a theoretical relationship between the observed frequencies segmented in groups by the modalities of the second variable.

#### Assumptions

- Expected count must not be small in any modality.

#### Strengths

- Approximation to the theoretical  $\chi^2$  distribution is better attained with *G-test* than  $\chi^2$  test.
- Cases where  $O_i > 2 \times E_i$ , *G-test* is always better than  $\chi^2$  test.

#### Weaknesses

- in test of independence, for a small sample size use rather Fisher's extract test.

**Frequentist** We compare the observed counts in each modality with their expected counts. Let us define the list of, respectively, expected counts for each modality  $i$ ,  $(E_i)_{1 \leq i \leq m}$ , and observed counts  $(O_i)_{1 \leq i \leq m}$ . Then  $\begin{cases} H_0 : \forall i \in \llbracket 1, m \rrbracket, O_i = E_i \\ H_1 : \exists i \in \llbracket 1, m \rrbracket, O_i \neq E_i: \text{two-tailed test} \end{cases}$

$$G = 2 \sum_{i=1}^m O_i \times \ln \left( \frac{O_i}{E_i} \right)$$

$$\ln \left( \frac{L(\tilde{\theta}|x)}{L(\hat{\theta}|x)} \right) = \ln \left( \frac{\prod_{i=1}^m \tilde{\theta}^{x_i}}{\prod_{i=1}^m \hat{\theta}^{x_i}} \right) = \ln \left( \frac{\prod_{i=1}^m \left( \frac{x_i}{n} \right)^{x_i}}{\prod_{i=1}^m \left( \frac{e_i}{n} \right)^{x_i}} \right) = \ln \left( \prod_{i=1}^m \left( \frac{x_i}{e_i} \right)^{x_i} \right) = \sum_{i=1}^m x_i \ln \left( \frac{x_i}{e_i} \right)$$

Then we multiply by  $-2$  to get  $G$ -test that is asymptotically equivalent to the *Pearson's*  $\chi^2$  formula.

### 6.2.6 Cochran's Q test

It checks if  $k$  treatments have identical effect, the response can take only 2 possible outcomes and a second variable segments the treatments.

|          | Treatment 1 | Treatment 2 | ...      | Treatment k |
|----------|-------------|-------------|----------|-------------|
| Block 1  | $x_{11}$    | $x_{12}$    | ...      | $x_{1k}$    |
| Block 2  | $x_{21}$    | $x_{22}$    | ...      | $x_{2k}$    |
| Block 3  | $x_{31}$    | $x_{32}$    | ...      | $x_{3k}$    |
| $\vdots$ | $\vdots$    | $\vdots$    | $\ddots$ | $\vdots$    |
| Block b  | $x_{b1}$    | $x_{b2}$    | ...      | $x_{bk}$    |

And  $\forall (i, j) \in \llbracket 1, b \rrbracket \times \llbracket 1, k \rrbracket, x_{ij} \in \{0, 1\}$

#### Assumptions

- The blocks are randomly selected from the population of all possible blocks.
- Outcome of the treatments are dichotomous, and should be coded in a standard way

**Frequentist** For example if  $b$  respondents in a survey had each been asked  $k$  *Yes/No* questions the  $Q$ -test could be use to test the null hypothesis that all questions were equally likely to elicit the answer "Yes".

We have  $\begin{cases} H_0: \text{the treatments are equally effective} \\ H_a: \text{the treatments are not equally effective} \end{cases}$

$$T = k(k-1) \frac{\sum_{j=1}^k (x_{.j} - \frac{N}{k})^2}{\sum_{i=1}^b x_{i.} (k - x_{i.})} \begin{cases} k: \text{number of treatments} \\ x_{.j}: \text{column total for the } j\text{th treatment} \\ b: \text{number of blocks} \\ X_{i.}: \text{row total for the } i\text{th block} \\ N: \text{grand total} \end{cases}$$

For significance level  $\alpha$ , the asymptotic critical region is  $T > \chi_{1-\alpha, k-1}^2$  which is the  $(a - \alpha)$  quantile of the  $\chi^2$  distribution with  $K - 1$  degrees of freedom.

**Bayesian** Does not exist, see *contingency table*



### 6.2.7 Sign test

It is a statistical method to test for consistent differences between pairs of observations, such as the weight of subjects before and after treatment. For comparisons of paired observations  $(x, y)$  the *sign-test* is [most useful if comparison can only be expressed as  \$x > y\$ ,  \$x = y\$ , or  \$x < y\$](#) . If instead the differences can be expressed in numeric quantities it is worthy to use *t-test* or [Wilcoxon signed-rank test](#) will usually have greater power than the sign test to detect consistent differences.

**Frequentist** Let  $p = \mathbb{P}(\{X > Y\})$ , then

$$\begin{cases} H_0 : p = 0.5 \text{ meaning that given } (x_i, y_i) \text{ each element is equally likely to be larger than the other} \\ H_a : p \neq 0.5 \end{cases}$$

Pairs are omitted for which there is no differences so that there is a potential reduced sample of  $m$  pairs. The statistics  $W$  is defined as follow:

$$W = \mathbf{1}_{\{x_i > y_i\}} \hookrightarrow \mathcal{B}(m, 0.5)$$

**Assumptions** Let  $\forall i \in \llbracket 1, n \rrbracket$ ,  $Z_i = X_i - Y_i$

- $Z_i$  are assumed independent.
- Each  $Z_i$  comes from the same continuous population.
- The values  $X_i$  and  $Y_i$  are ordered.

**Strengths**

- A fewer assumptions need to be made than for parametrical test

**Weaknesses**

- The power of test is lower than for a parametrical test

### 6.2.8 Contingency coefficients: Cramér's V

To [quantify associations between 2 paired samples in a contingency table](#), it is based on  $\chi^2$  and varies from 0 (no association) to 1 (complete association).

**Frequentist** Let a sample of size  $n$  of the simultaneously distributed variable  $A$  and  $B$ .  $\forall (i, j) \in$

$$\llbracket 1, r \rrbracket \times \llbracket 1, c \rrbracket, n_{ij} = \text{Card}(\{A_i, B_j\}). \text{ Then } \chi^2 = \sum_{(i,j) \in \llbracket 1, r \rrbracket \times \llbracket 1, c \rrbracket} \frac{\left(n_{ij} - \frac{n_{i.} \times n_{.j}}{n}\right)^2}{\frac{n_{i.} \times n_{.j}}{n}}$$

Finally the Cramér's V with bias correction is:

$$V = \sqrt{\frac{\max\left(0, \frac{\chi^2}{n} - \frac{(r-1)(c-1)}{n}\right)}{\min\left(r - \frac{(r-1)^2}{n-1} - 1, c - \frac{(c-1)^2}{n-1} - 1\right)}}$$

**Assumptions**

- The both variables have to be nominal.

**Strengths**

- Good analog of the  $R^2$  for categorical variables.

**Weaknesses**

- Can tend to overestimate the strength of association.

### 6.2.9 Contingency table from a Bayesian perspective

To test the independence hypothesis between 2 variables.

**Bayesian** Let's consider 4 sampling plans, depending on which sampling plan is chosen the Bayes factor formula will change.

- *Poisson* sampling scheme: Each cell count is considered as random and so is the grand total, the cells are Poisson distributed. This design often occurs in purely observational work.
- *Joint multinomial* sampling scheme: same as above except that now, the grand total is fixed.
- *Independent multinomial* sampling scheme: either all row margins or all column margins are fixed, this scheme is frequently used in psychological studies.
- *Hypergeometric* sampling scheme: here both row margins and column margins are fixed. Practical use of this scheme is rare!

Bayes factors are often difficult to compute, as they are obtained by integrating out over the entire parameter space, a process that is non-trivial when the integrals are high-dimensional and intractable. Then we will use the 4 Bayes Factor developed by Gunnell and Dickey in 1974, because they only require computation of common functions such as gamma functions, for which numerical approximation are already available.

Here the logic: the Bayes Factor  $BF_{01}^{i+1}$  computed at the observation  $i + 1$ , contains the information up to the step  $i$  with the extra information of the step  $i + 1$ . We can then see  $BF_{01}^{i+1}$  as the Bayes factor of the observation  $i + 1$  conditioned on the observation  $i$ .

Finally thanks to the successive conditionalization the Bayes Factor are easy to compute.

#### Assumptions

- We need to be consider data providing from one of the following sampling scheme: *Poisson*, *Joint multinomial*, *Independent multinomial* or *Hypergeometric*

#### Strengths

- Bayesian approach, then no issue to assess the significance
- Implemented in R

#### Weaknesses

- Restricted to the above sampling scheme today.

### 6.2.10 Wilconox test

Non parametric test, used to test the location of a population based on a data sample or to compare the locations of two populations using two matching samples.

It is a good alternative of the *t-test* when the mean is not of interest for the studied population.

**Frequentist** Let  $Y$  and  $X$  be 2 random variables, and  $(x_i, y_i)_{1 \leq i \leq n}$  a paired sample.

1.  $\forall i \in \llbracket 1, n \rrbracket, |x_i|$
2. Sort the  $(|x_i|)_{1 \leq i \leq n}$  and assign a rank  $(R_i)_{1 \leq i \leq n}$
3. The test statistic  $T = \sum_{i=1}^n \text{sgn}(X_i) R_i$
4. Produce a *p-v* by computing T to its distribution under the null hypothesis.

We will provide the logic for a one-sample test, the two-sample follows the same logic but with 2 variables.

Assume the data consists of independent and identically distributed (IID) samples from a distribution  $F$  then consider 2 variables  $(X_1, X_2) \hookrightarrow IID(F)$  Define  $p_2 = \mathbb{P}\left(\left\{\frac{X_1 + X_2}{2} > 0\right\}\right) = 1 - F^{(2)}(0)$  Then

Wilcoxon signed-rank  $sum \rightarrow H_0 : p_2 = \frac{1}{2}$  In restricting the distributions of interest we can reach more interpretable null and alternative hypotheses. On mildly restrictive is that  $F^{(2)}$  has a unique median  $\mu$ . This median is called pseudo median of  $F$  Then we have  $H_0 : \mu = 0$

•

### Assumptions

- Distribution  $F$  is symmetric

### Strengths

Weaknesses TO COMPLETE

## 6.2.11 Mann-Whitney test

Test for a randomly selected values  $x$  and  $y$  from 2 populations,  $\mathbb{P}(\{x \leq y\}) = \mathbb{P}(\{x > y\})$

**Frequentist** Let  $(n_1, n_2)\mathcal{N}_*^2$  and  $(x_i)_{1 \leq i \leq n_1}$  and  $(y_i)_{1 \leq i \leq n_2}$  both samples independent of each other.

Then  $\begin{cases} U_1 = n_1 n_2 + \frac{n_1(n_1+1)}{2} - R_1 \\ U_2 = n_1 n_2 + \frac{n_2(n_2+1)}{2} - R_2 \end{cases}$   $R_1, R_2$  being the sum of the ranks in groups 1 and 2.

Note that  $AUC_1 = \frac{U_1}{n_1 n_2}$  meaning  $U$ -statistics is related to the area under the receiver operating characteristic.

### Assumptions

- All observation from both groups are independent
- Values are at least ordinal
- $H_0$ : the distribution of both population is identical
- $H_1$ : the 2 distribution of population are different

### Strengths

Weaknesses

## 6.2.12 Kruksal-Wallis test

Non-parametrical to [test if samples originate from the same distribution](#).

**Frequentist** Let  $N$  be the number of observations across all groups,  $g$  number of groups,  $n_i$  the number of observation in the group  $i$ ,  $r_{ij}$  the rank of observation  $j$  from group  $i$ .

$$\text{And } \begin{cases} \bar{r}_{i\cdot} = \frac{\sum_{j=1}^{n_i} r_{ij}}{n_i} \\ \bar{r} = \frac{N+1}{2} \end{cases}$$

- Rank all data from all groups together

- $(N-1) \frac{\sum_{i=1}^g n_i (\bar{r}_{i.} - \bar{r})^2}{\sum_{i=1}^g \sum_{j=1}^g n_i (r_{ij} - \bar{r})^2}$  To actually check the stochastic differences.
- A correction can be brought for large number of ties.

### Assumptions

- Independence
- All groups should have the same distributions

### Strengths

- Non-parametrical test, no need of the normally distributed assumption.

### Weaknesses

#### 6.2.13 Friedman test

Non-parametric statistical test, [analog of the repeated-measures ANOVA](#). It is used to detect differences in treatments across multiple test attempts.

#### Frequentist

- Consider a matrix of  $n$  rows (the blocks) and  $k$  columns (the treatments) and a single observation at the intersection of each block and treatment. Then calculate the ranks.
- $\bar{r}_{.j} = \frac{1}{n} \sum_{i=1}^n r_{ij}$
- The test statistic is given by  $Q = \frac{12n}{k(k+1)} \sum_{j=1}^k \left( \bar{r}_{.j} - \frac{k+1}{2} \right)^2$ . Note that the value of  $Q$  does need to be adjusted for tied values in data.
- Finally when  $n$  or  $k$  is large ( $n > 15$  or  $k > 4$ ) the probability distribution of  $Q$  can be approximated by a  $\chi^2$  distribution.

### Assumptions

- Independence

### Strengths

### Weaknesses

#### 6.2.14 Spearman test

It [assesses how well the relationship between 2 variables can be described using a monotonic function](#). Let  $X, Y$  be 2 random variables, and  $R$  the function transforming the realization of a random variable.

**Frequentist**  $r_s = \rho_{R(X), R(Y)} = \frac{Cov(R(X), R(Y))}{\sigma_{R(X)} \sigma_{R(Y)}} \begin{cases} \rho : \text{Pearson correlation coefficient applied to the rank variables} \\ Cov(R(X), R(Y)) \end{cases}$

### Assumptions

### Strengths

## Weaknesses

### 6.2.15 Pearson correlation coefficient

This coefficient is essentially a normalized measurement of the covariance such that the result has a value  $-1$  and  $1$

**Frequentist**  $\rho_{X,Y} = \frac{Cov(X,Y)}{\sigma_X \sigma_Y}$

## Assumptions

## Strengths

## Weaknesses

- Not robust

### 6.2.16 Repeated-measures ANOVA

Used in repeated measure design, meaning when we measure multiple time the same variable taken on the same or matched subject either under different conditions or at different periods.

**Frequentist** Here  $F = \frac{\frac{SS_{treatment}}{df_{treatment}}}{\frac{SS_{error}}{df_{error}}}$  In a *between-subjects* design there is a element of variance due to individual difference that is combined with the treatment and error term, meaning:  $SS_{total} = SS_{treatment} + SS_{error}$  In a *repeated-measure* design it is possible to partition subject variability from the treatment and error term, meaning  $SS_{total} = SS_{treatment(excluding\ individual\ differences)} + SS_{subjects} + SS_{error}$

## Assumptions

- *Normality*: for each level of the within-subject factor, the dependent variable must have a normal distribution.
- *Sphericity*: difference scores computed between 2 levels of a within subject factor must have the same variance for the comparison of any 2 levels.
- *Randomness*: cases should be derived from a random sample.

## Strengths

- ability to partition out variability due to individual differences.

## Weaknesses

- Vulnerable to missing values, imputation, unequivalent time points between subjects and violation of Sphericity.

### 6.2.17 1-way ANOVA

Analysis of Variance describes the partition of the response variable sum of squares in a linear model into “explained” and “unexplained” components.

- Single categorical (or less common numerical) explanatory variable corresponds to One-Way ANOVA
- 2 factors to Two-Way ANOVA
- 3 factors to Three-Way ANOVA

The term “analysis of variance” is a bit of misnomer, we use variance-like quantities to study the equality or non-equality of population means, so we are analyzing means, not variances.

**Frequentist** examines equality of population means for a quantitative outcome and a single categorical explanatory variable with any number of levels.

The term “one-way” indicates that there is a single explanatory variable (“treatment”) with 2 or more levels and only one level of treatment is applied at any time for a given subject.

And  $H_0 : \forall (i, j) \in \llbracket 1, k \rrbracket^2 \mu_i = \mu_j$

In ANOVA we work with variances and also “variance-like quantities” which are not really the variance of anything, but are still calculated as  $\frac{SS}{df}$  all of these quantities are called “mean squares”.

The deviation for subject  $j$  of group  $i$  in the figure above is mathematically equal to  $Y_{ij} - \bar{Y}_i$  where  $Y_{ij}$  is the observed value for subject  $j$  of group  $i$  and  $\bar{Y}_i$  is the sample mean for group  $i$ .

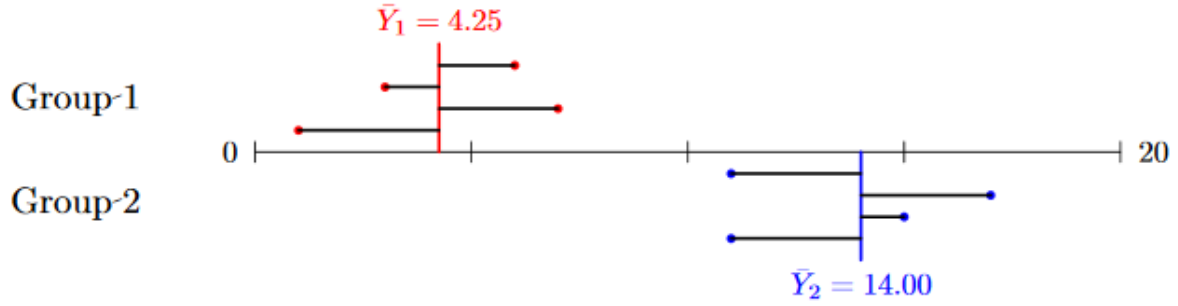


Figure 6.1: Deviations for within-group of squares

$$MS_{within} = \frac{SS_{within}}{df_{within}} \begin{cases} SS_{within} = \sum_{j=1}^k SS_j = \sum_{j=1}^k \sum_{i=1}^{n_j} (Y_{ij} - \bar{Y}_{\bullet j})^2 \\ df_{within} = df_j = \sum_{j=1}^k (n_j - 1) = N - k \end{cases}$$

$MS_{within}$  is a good estimate of  $\sigma^2$  from our model regardless of the truth of  $H_0$ . This is due to the way  $SS_{within}$  is defined.

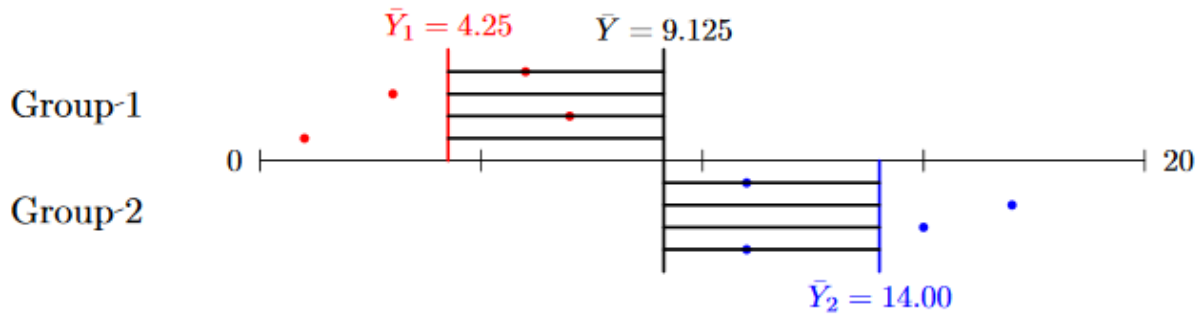


Figure 6.2: Deviations for between-group sum of squares

$SS_{between}$  is the sum of the  $N$  squared between-group deviations, where the deviation is the same for all subjects in the same group. The formula is :

$$MS_{Between} = \frac{SS_{Between}}{df_{between}} \begin{cases} SS_{between} = \sum_{j=1}^k n_j (\bar{Y}_{\bullet j} - \bar{Y})^2 \\ df_{between} = k - 1 \end{cases}$$

Because of the way  $SS_{between}$  is defined,  $MS_{between}$  is a good estimate of  $\sigma^2$  only if  $H_0$  is true. Otherwise it tends to be larger.

The  $F$ -statistic defined by  $F = \frac{MS_{between}}{MS_{within}}$  tends to be larger if the alternative hypothesis is true than if the null hypothesis is true.

We can quantify “large” for the  $F$ -statistic, by comparing it to its null sampling distribution which is the specific  $F$ -distribution which has degrees of freedom matching the numerator and denominator of the  $F$ -statistic.

Concerning inferences to build the confidence interval we need the standard error (the standard deviation of the means) that is  $\sqrt{\frac{MS_{within}}{n_i}}$

Let's detail the case where  $p = 2$  meaning comparison of 2 samples: with means  $\mu_1$  and  $\mu_2$ , and the same variance  $\sigma^2$  and finally  $n = n_1 + n_2$  observations. Model:

$$\forall (j, i) \in \llbracket 1, 2 \rrbracket \times \llbracket 1, n_j \rrbracket y_{ij} = \mu_j + \epsilon_{ij} = \mu + \alpha_j + \epsilon_{ij}$$

$\alpha_j = \mu_j - \mu$  is called (treatment-) effect

Decomposition:

$$\begin{aligned} SS_{total} &= \sum_{j=1}^{n_1} (y_{1j} - \bar{y})^2 + \sum_{j=1}^{n_2} (y_{2j} - \bar{y})^2 \\ &= \sum_{j=1}^{n_1} (y_{1j} - \bar{y}_1 + \bar{y}_1 - \bar{y})^2 + \sum_{j=1}^{n_2} (y_{2j} - \bar{y}_2 + \bar{y}_2 - \bar{y})^2 \\ &= \underbrace{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}_{SS_{within}} + \underbrace{n_1(\bar{y}_1 - \bar{y})^2 + n_2(\bar{y}_2 - \bar{y})^2}_{SS_{between}} \end{aligned}$$

$SS_{between}$  corresponds to squared enumerator  $(\bar{y}_1 - \bar{y}_2)^2$  of the statistic:

$$\begin{aligned} SS_{between} &= n_1(\bar{y}_1 - \bar{y})^2 + n_2(\bar{y}_2 - \bar{y})^2 \\ &= n_1 \left( \bar{y}_1 - \frac{n_1\bar{y}_1 + n_2\bar{y}_2}{n_1 + n_2} \right)^2 + n_2 \left( \bar{y}_2 - \frac{n_1\bar{y}_1 + n_2\bar{y}_2}{n_1 + n_2} \right)^2 \\ &= \frac{n_1 n_2}{n_1 + n_2} (\bar{y}_1 - \bar{y}_2)^2 \end{aligned}$$

$SS_{within}$  corresponds to denominator of  $t$ -statistic:  $s = \sqrt{\frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2}}$

Pooled variance that is an estimate of the fixed common variance  $\sigma^2$  underlying various populations that have different means.  $\hat{\sigma} = \frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{(n_1 - 1) + (n_2 - 1)}$

Null hypothesis  $H_0 : \mu_1 = \mu_2$  or  $\alpha_1 = \alpha_2 = 0$

$F$ -test  $(\bar{Y}_1 - \bar{Y}_2) \hookrightarrow \mathcal{N}\left(\mu_1 - \mu_2, \left(\frac{1}{n_1} + \frac{1}{n_2}\right)\sigma^2\right)$

$$\mathbb{E}\left([\bar{Y}_1 - \bar{Y}_2]^2\right) = \left(\frac{1}{n_1} + \frac{1}{n_2}\right)\sigma^2 + (\mu_1 - \mu_2)^2$$

$$\mathbb{E}(MS_{between}) = \mathbb{E}\left(\frac{n_1 n_2}{n_1 + n_2} [\bar{Y}_1 - \bar{Y}_2]^2\right) = \sigma^2 + \frac{n_1 n_2}{n_1 + n_2} (\mu_1 - \mu_2)^2$$

$$\mathbb{E}(MS_{within}) = \sigma^2$$

$$F = \frac{MS_{between}}{MS_{within}}$$

Here  $F = t^2$

Degrees of freedom  $= n - 1$

$$= \underbrace{(n - m)}_{df_{within}} + \underbrace{(m - 1)}_{df_{between}}$$

- $SS_{within}$  and  $SS_{between}$  are independent
- under  $H_0$   $\mathbb{E}(MS_{between}) = \mathbb{E}(MS_{within}) = \sigma^2$
- under  $H_a$   $\mathbb{E}(MS_{between}) > \sigma^2$  and  $\mathbb{E}(MS_{within}) = \sigma^2$

Hence

$$F = \frac{MS_{between}}{MS_{within}} \hookrightarrow F_{m-1, n-m}$$

In the case of 2 groups (“*t-test*”) we received:

$$\bar{y}_1 - \bar{y}_2 \pm t_{n-2, 1-\frac{\alpha}{2}} s \sqrt{\frac{1}{n_1} + \frac{1}{n_2}}$$

**Assumptions** The statistical model for which one-way ANOVA is appropriate is that the

- (Quantitative) Outcomes for each group are normally distributed
- Outcome variances are all equal to ( $\sigma^2$ )
- The errors are assumed to be independent.

**Strengths**

**Weaknesses**

### 6.2.18 T-test

It is commonly used when the test statistic would follow a normal distribution if the value of a scaling term in the test statistic were known.

When the scaling term is estimated from the data, under certain conditions, the test statistic follows a *Student's t-test*.

Most test statistics have the form  $t = \frac{Z}{s}$ ,  $Z$  may be sensitive to the alternative hypothesis, whereas  $s$  is a scaling parameter allowing to determine the distribution  $t$ .

**One-sample**

$$t = \frac{\bar{x} - \mu_0}{\frac{s}{\sqrt{n}}}, \begin{cases} \bar{x}: \text{sample mean} \\ s: \text{sample standard deviation} \\ n: \text{sample size} \end{cases}$$

By the central limit theorem, if the observations are independent and the second moment exist, then  $t$  will approximately follow the distribution  $\mathcal{N}(0, 1)$

**Assumptions** Although the parent population does not need to be normally distributed, the distribution of the population sample means  $(\bar{x}_s)_{1 \leq s \leq S}$ .

**Strengths**

**Weaknesses**

**Slope of a regression line** Suppose one is fitting:  $Y = \alpha + \beta x + \epsilon$ , where  $x$  is known and  $\alpha$  and  $\beta$  are unknown and finally  $\epsilon \hookrightarrow \mathcal{N}(0, \infty)$ . Symbols with hat will refer to estimators.

$$t_{score} = \frac{\hat{\beta} - \beta_0}{SE_{\hat{\beta}}} \hookrightarrow \mathcal{T}_{n-2}$$

The standard error of the slope coefficient: 
$$\begin{cases} SE_{\hat{\beta}} = \frac{\sqrt{\frac{1}{n-2} \sum_{i=1}^n (y_i - \hat{y}_i)^2}}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2}} \\ SSR = \sum_{i=1}^n \hat{\epsilon}_i^2 = \sum_{i=1}^n (y_i - \hat{y}_i)^2 \end{cases}$$



## Independent 2-sample t-test

$$\text{Equal sample sizes and variance} \quad \begin{cases} t = \frac{\bar{X}_1 - \bar{X}_2}{s_p \sqrt{\frac{2}{n}}} \\ s_p = \sqrt{\frac{s_{X_1}^2 + s_{X_2}^2}{2}} \end{cases}$$

$$\text{Unequal sample sizes and similar variances } \left( \frac{1}{2} < \frac{s_{X_1}}{s_{X_2}} < 2 \right) : \begin{cases} t = \frac{\bar{X}_1 - \bar{X}_2}{s_p \sqrt{\frac{1}{n_1} + \frac{1}{n_2}}} \\ s_p = \sqrt{\frac{(n_1 - 1)s_{X_1}^2 + (n_2 - 1)s_{X_2}^2}{n_1 + n_2 - 2}} \end{cases}$$

$$\text{Unequal sample sizes and unequal variances } (s_{X_1} > 2s_{X_2} \text{ or } s_{X_2} > 2s_{X_1}) : \begin{cases} t = \frac{\bar{X}_1 - \bar{X}_2}{s_{\Delta}} \\ s_{\Delta} = \sqrt{\frac{s_{X_1}^2}{n_1} + \frac{s_{X_2}^2}{n_2}} \end{cases}$$

$$\text{Dependent t-test for paired samples} \quad t = \frac{\bar{X}_D - \mu_0}{\frac{s_D}{\sqrt{n}}}$$

Assumptions

Strengths

Weaknesses

# Chapter 7

## Data recovery

### 7.1 Sampling methods

#### 7.1.1 Monte Carlo approximation

**Purpose** computing the distribution of a random variable's function using the change of variables formula can be difficult.

As simple and powerful alternative is to generate  $S$  samples  $(x_s)_{1 \leq s \leq S}$  from the distribution.

**Theory** Given the samples we can approximate the distribution of  $f(X)$  by using the empirical distribution of  $\{f(x_s)\}_{1 \leq s \leq S}$

We can use Monte Carlo to approximate the expected value of any function of a random variable:

$$\mathbb{E}(f(X)) = \int f(x)p(x)dx \approx \frac{1}{S} \sum_{s=1}^S f(x_s)$$

where  $x_s \hookrightarrow p(X)$ . This called Monte Carlo integration

**Accuracy** It increases with sample size.

In denoting  $\mu = \mathbb{E}(f(X))$  the exact mean and  $\hat{\mu}$  the MC approximation, one can show that

$$(\hat{\mu} - \mu) \hookrightarrow \mathcal{N}\left(0, \frac{\sigma^2}{S}\right)$$

where  $\sigma^2 = \mathbb{V}(f(X))$  this is a consequence of central-limit theorem, of course  $\sigma^2$  is unknown.

$\sqrt{\frac{\hat{\sigma}^2}{S}}$  is called the *standard error* and is an estimate of our uncertainty about our estimate  $\mu$

#### Strengths

- function only evaluated in places where there is non-negligible probability: advantage over numerical integration

#### Examples

- estimating  $\pi$

#### 7.1.2 Bootstrap

**Purpose** In practice bootstrap seems to be a good compromise between speed and accuracy. It is a Monte Carlo technique to approximate the sampling distribution.

**Theory** If we knew the true parameters  $\theta^*$  we could generate  $S$  fake datasets of size  $N$  from the distribution  $\forall(i, s) \in \llbracket 1, n \rrbracket \times \llbracket 1, s \rrbracket, x_i^s \hookrightarrow p(\cdot | \theta^*)$

We could then compute our estimator from each sample:  $\hat{\theta}^s = f(x^s)$ .

Then 2 approaches:

- *parametric*: generate the samples using  $\hat{\theta}(\mathcal{D})$  as  $\theta$  is unknown
- *non-parametric*: sample the  $(x^s)_{1 \leq s \leq S}$  with replacement from the original  $\mathcal{D}$  and then compute induced distribution

**Connection between  $\hat{\theta}^s = \hat{\theta}(x^s)$  and  $\theta^s \hookrightarrow p(\cdot | \mathcal{D})$**  Conceptually quite different, but in the common case the prior is not very strong they can be quite similar. One can think of the [bootstrap distribution](#) as a "poor man's" posterior.

## Strengths

- Useful [when the estimator is a complex function of the true parameter](#)

### 7.1.3 Monte Carlo Inference

**Sampling from a Gaussian (Box-Muller method)** The idea is we sample uniformly from a unit radius circle and then use the change of variables to derive samples from spherical 2d Gaussian.

## Rejection sampling

**Basic idea** we create a *proposal distribution*  $q(x)$  which satisfies

## Importance sampling

**Basic idea** The idea is to draw samples  $\mathbf{x}$  in regions which have high probability  $\mathbb{P}(\mathbf{x})$

### 7.1.4 Particle filtering

it is a Monte Carlo algorithm for recursive Bayesian inference

### 7.1.5 Annealing methods

COMPLETE

## 7.2 Information theory

Originally developed [to study sending messages from discrete alphabets over a noisy channel](#) (like communication via radio transmission). The basic intuition behind information theory is that learning that an unlikely event has occurred is more informative than learning that a likely event has occurred.

### 7.2.1 Shannon entropy

**Purpose** It is a [measure of the uncertainty in a random variable](#).

**Theory** We have self-information

$$I(x) = -\log(\mathbb{P}(x))$$

Depending of the basis of the used logarithm, the unit can be expressed in *nats* if the base is  $e$  or *bits* if the basis is 2.

One *nat* is the amount of information gained by observing an event of probability  $\frac{1}{e}$

$$\mathbb{H}(X) \triangleq -\mathbb{E}_{X \hookrightarrow P}(I(x)) = -\sum_{k=1}^K \mathbb{P}(X = k) \log(\mathbb{P}(X = k))$$

It gives a lower bound on the number of bits needed on average to encode symbols drawn from a distribution  $P$ . The discrete distribution with maximum entropy is the uniform distribution, conversely the one with minimum entropy is any delta-function that puts all its mass on one state.

## 7.2.2 Kullback-Leibler (KL) divergence

**Purpose** It is a measure of dissimilarity of 2 probability distribution  $p$  and  $q$ .

**Theory**

$$\begin{aligned}\mathbb{KL}(p||q) &= \sum_{k=1}^K p_k \log\left(\frac{p_k}{q_k}\right) \\ &= \sum_{k=1}^K p_k \log(p_k) - \sum_{k=1}^K p_k \log(q_k) \\ &= -\mathbb{H}(p) + \mathbb{H}(p, q)\end{aligned}$$

It gives the extra amount of information needed to send a message containing symbols drawn from probability distribution  $P$  when we use a code that was designed to minimize the length of messages drawn from probability distribution  $Q$ . Where  $\mathbb{H}(p, q)$  is called *cross entropy*, being the average number of bits needed to encode data coming from a source with distribution  $p$  when we use model  $q$  to define our codebook.

## 7.2.3 Mutual information

**Purpose** Correlation coefficient is quite restrictive, a more general approach is to determine how similar the joint distribution  $p(X, Y)$  is to the factorized distribution  $p(X)p(Y)$

**Theory**

**Discrete**

$$\mathbb{I}(X, Y) \triangleq \mathbb{KL}(p(X, Y)||p(X)p(Y)) = \mathbb{H}(X) - \mathbb{H}(X|Y) = \mathbb{H}(Y) - \mathbb{H}(Y|X)$$

where  $\mathbb{H}(Y|X)$  is the *conditional entropy* defined as  $\mathbb{H}(Y|X) = \sum_x p(x) \mathbb{H}(Y|X=x)$ . It is the extra amount of information needed to send a message containing symbols drawn from probability distribution  $P$  when use a code that was designed to minimize the length of messages drawn from probability distribution  $Q$ . Thus we can interpret the *Mutual Information between  $X$  and  $Y$  as the reduction in uncertainty about  $X$  after observing  $Y$ , or, by symmetry about  $Y$  after observing  $X$* .

**Continuous** Quantizing can have significant impact on the results, we could then try to estimate many different bin sizes and locations to finally compute the maximum MI achieved called *maximal information coefficient*:

$$m(x, y) = \frac{\max_{G \in \mathcal{G}(x, y)} \mathbb{I}(X(G), Y(G))}{\log(\min(x, y))}$$

where  $\mathcal{G}(x, y)$  is the set of 2d grids of size  $x \times y$  and  $X(G), Y(G)$  represents a discretization of the variable onto the grid.  $MIC \triangleq \max_{x, y: xy < B} m(x, y)$

## 7.3 Key Mathematical functions

### 7.3.1 Softmax function

**Purpose** The softmax function takes as input a vector  $z$  of  $K$  real numbers, and normalizes it into a probability distribution consisting of  $K$  probabilities proportional to the exponentials of the input numbers.

$$\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

**Interpretation** it is rather a smooth approximation of the *argmax* function, meaning the function returning the index of the maximum value of a given vector.

# Chapter 8

## Markov Models

### 8.1 Markov Models and Hidden Markov Models

#### 8.1.1 Markov Models

**Definition** It assumes that  $X_t$  captures all the relevant information for predicting the future. If we assume discrete time steps the joint distribution is:

$$\mathbb{P}(X_{1:T}) = \mathbb{P}(X_1) \prod_{t=2}^T \mathbb{P}(X_t|X_{t-1})$$

if the transition function  $\mathbb{P}(X_t|X_{t-1})$  is independent of time then the chain is called *homogeneous, stationary* or *time-invariant*.

This assumption allows us to model an arbitrary of variables using a fixed number of parameters, such models are called **stochastic process**.

**Assumptions**  $x_{t+1} \perp x_{1:t-1}|x_t$ , meaning the future is independent of the past given the present, is called the *first order Markov assumption*.

**Transition matrix** When  $X_t$  is discrete the conditional distribution  $\mathbb{P}(X_t|X_{t-1})$  can be written as  $K \times K$  matrix known as **transition matrix  $A$** , where  $A_{ij} = \mathbb{P}(X_t = j|X_{t-1} = i)$  being the probability of going from state  $i$  to  $j$ . Each row of the matrix sums to one  $\sum_j A_{ij} = 1$  so it is called *stochastic matrix*.

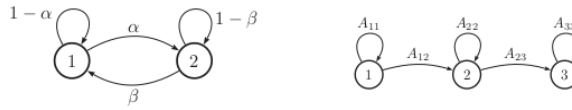


Figure 8.1: State transition diagrams for, 2-state chain and 3-state chain.

The  $n$ -step transition matrix  $A(n)$  is defined as  $A_{ij}(n) = \mathbb{P}(X_{t+n} = j|X_t = i)$  For a steps  $m + n$ ,  $A(m + n) = A(m) + A(n)$

#### Application

- Language modeling
  - Sentence completion
  - Data compression
  - Text classification
  - Automatic essay writing

**MLE for Markov language models** The [probability of any particular sequence of length T](#) is given by:

$$\mathbb{P}(x_{1:T}|\boldsymbol{\theta}) = \pi(x_1) \prod_{t=2}^T A(x_{t-1}, x_t) = \prod_{i=1}^K \pi_i^{\mathbb{1}_{\{x_1=i\}}} \prod_{t=2}^T \prod_{j=1}^K \prod_{i=1}^K A_{ij}^{\mathbb{1}_{\{x_t=j, x_{t-1}=i\}}}$$

Then the log-likelihood of a set of sequences  $\mathcal{D} = \{\mathbf{x}_i\}_{1 \leq i \leq n}$  :

$$\log(\mathbb{P}(\mathcal{D}|\boldsymbol{\theta})) = \sum_{n=1}^N \log(\mathbb{P}(\mathbf{x}_n|\boldsymbol{\theta})) = \sum_i N_i^{(1)} \log(\pi_i) + \sum_i \sum_j N_{ij} \log(a_{ij})$$

where  $\hat{\pi}_i = \frac{N_i^{(1)}}{\sum_{i'} N_{i'}^{(1)}}$  and  $\hat{a}_{ij} = \frac{N_{ij}}{\sum_{j'} N_{ij'}}$

**Stationary distribution of a Markov chain** We can interpret Markov models as stochastic dynamical systems, where we hop from one state to another at each time step, we are then often interested in the [long term distribution over states, called stationary distribution](#).

Considering the transition matrix  $\mathbf{A}$  defined by  $A_{ij} = \mathbb{P}(X_t = j | X_{t-1} = i)$  and the probability to be at a given state at time t  $\boldsymbol{\pi}$  defined by  $\pi_t(j) = \mathbb{P}(X_t = j)$ .

[If we ever reach a stage where  \$\boldsymbol{\pi} = \boldsymbol{\pi}\mathbf{A}\$  then we have reached the stationary distribution](#).

## 8.1.2 Hidden Markov Models

**Definition** It consists of a discrete-time, [discrete-state Markov chain, with hidden states](#)  $z_t \in \llbracket 1, K \rrbracket$ . The corresponding joint distribution has the form:

$$\mathbb{P}(\mathbf{z}_{1:T}, \mathbf{x}_{1:T}) = \mathbb{P}(\mathbf{z}_{1:T}) \mathbb{P}(\mathbf{x}_{1:T}|\mathbf{z}_{1:T}) = \left[ \mathbb{P}(z_1) \prod_{t=2}^T \mathbb{P}(z_t|z_{t-1}) \right] \prod_{t=1}^T \mathbb{P}(\mathbf{x}_t|z_t)$$

It's common to have for:

- *discrete* observations:  $\mathbb{P}(\mathbf{x}_t = l | z_t = k, \boldsymbol{\theta}) = \text{Beta}(k, l)$
- *continuous* observations:  $\mathbb{P}(\mathbf{x}_t | z_t = k, \boldsymbol{\theta}) = \mathcal{N}(\mathbf{x}_t | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$

### Strengths

- can represent long-range dependencies between observations compared to Markov models
- useful for time series prediction
- can define class-conditional densities inside a generative classifier

### Examples

- Automatic speech recognition:  $\mathbf{x}_t$  represents features extracted from the speech signals and  $z_t$  represents the word that is being spoken
- Activity recognition:  $\mathbf{x}_t$  represents feature extracted from a video frame and  $z_t$  is the class of activity the person is engaged (running, waking, sitting)

## Types of inferences

- **Filtering:** computing the *belief state*  $\mathbb{P}(z_t|\mathbf{x}_{1:t})$  online or recursively as the data streams in. The name "filtering" is due to the noise reduction induced with the hidden state estimating using the current  $\mathbb{P}(z_t|\mathbf{x}_t)$
- **Smoothing:** computing  $\mathbb{P}(z_t|\mathbf{x}_{1:T})$  offline given all the evidence
- **Fixed Lag Smoothing:** computing  $\mathbb{P}(z_{t-l}|\mathbf{x}_{1:t})$  that is an interesting compromise between on-line and offline estimation. It provides better performance than filtering but incurs a slight delay. Changing the lag size, one can trade off *accuracy vs delay*
- **Prediction:** computing  $\mathbb{P}(z_{t+h}|\mathbf{x}_{1:t})$  where  $h > 0$  is called the *prediction horizon*. For example  $h = 2$  implies  $\mathbb{P}(z_{t+2}|\mathbf{x}_{1:t}) = \sum_{z_{t+1}} \sum_{z_t} \mathbb{P}(z_{t+2}|z_{t+1}) \mathbb{P}(z_{t+1}|z_t) \mathbb{P}(z_t|\mathbf{x}_{1:t})$
- **MAP estimation:** computing  $\arg \min_{(z_k)_{1 \leq k \leq t}}$  which is most probable state sequence.
- **Posterior samples:** if there is more than one plausible interpretation of the data, it can be useful to sample from the posterior  $\mathbf{z}_{1:t} \hookrightarrow \mathbb{P}(\mathbf{z}_{1:t}|\mathbf{x}_{1:t})$
- **Probability of the evidence:** compute  $\mathbb{P}(\mathbf{x}_{1:t}) = \sum_{\mathbf{z}_{1:t}} \mathbb{P}(\mathbf{z}_{1:t}, \mathbf{x}_{1:t})$  meaning summing up over all hidden paths. Can be used to classify sequences for model-based clustering, anomaly detection.

### 8.1.3 Inference HMMs

#### The forward algorithm

**Purpose** recursively computing the *filtered marginals*: for  $j \in \llbracket 1, K \rrbracket$   $\mathbb{P}(z_t = j|\mathbf{x}_{1:t})$  in a hidden Markov model.

#### Steps

1. *Prediction step:* computing the *one-step-ahead predictive density* acting as the new prior for  $t$ :  

$$\mathbb{P}(z_t = j|\mathbf{x}_{1:t-1}) = \sum_i \mathbb{P}(z_t = j|z_{t-1} = i) \mathbb{P}(z_{t-1} = i|\mathbf{x}_{1:t-1})$$
2. *Update step:* we absorb the observed data from  $t$  using Bayes rule:  

$$\alpha_t(j) \triangleq \mathbb{P}(z_t = j|\mathbf{x}_{1:t}) = \mathbb{P}(z_t = j|\mathbf{x}_t, \mathbf{x}_{1:t-1}) = \frac{1}{Z_t} \mathbb{P}(\mathbf{x}_t|z_t = j) \mathbb{P}(z_t = j|\mathbf{x}_{1:t-1})$$
where the normalization constant is  $Z_t \triangleq \mathbb{P}(\mathbf{x}_t|\mathbf{x}_{1:t-1}) = \sum_j \mathbb{P}(\mathbf{x}_t|z_t = j) \mathbb{P}(z_t = j|\mathbf{x}_{1:t-1})$

Hint: consider  $\mathbf{x}_t \perp \mathbf{x}_{1:t-1} | z_t$

The distribution  $\mathbb{P}(z_t|\mathbf{x}_{1:t})$  is called the *(filtered) belief state at time t*, it is a vector of  $K$  numbers denoted  $\alpha_t$ . Considering  $\odot$  being the *Hadamard product* representing element-wise vector multiplication.

$$\alpha_t \propto \psi_t \odot \Psi^T \alpha_{t-1}$$

where  $\psi_t(j) = \mathbb{P}(\mathbf{x}_t|z_t = j)$  being the local evidence at time  $t$  and  $\Psi(i, j) = \mathbb{P}(z_t = j|z_{t-1} = i)$  is the transition matrix.

#### Forwards-Backwards algorithm

**Purpose** Computing smoothed marginals  $\mathbb{P}(z_t = j|\mathbf{x}_{1:T})$  using offline inference.



**Basic idea** The key decomposition relies on the fact that we can break the chain into two parts: the past and the future by conditioning on  $z_t$

Let  $\alpha_t(j) \triangleq \mathbb{P}(z_t = j | \mathbf{x}_{1:t})$  be the filtered belief state as before. Define as well  $\beta_t(j) \triangleq \mathbb{P}(\mathbf{x}_{t+1:T} | z_t = j)$  being the conditional likelihood of the future evidence given that the hidden state at time  $t$  is  $j$ .

Finally define  $\gamma_t(j) \triangleq \mathbb{P}(z_t = j | \mathbf{x}_{1:T})$  as the desired smoothed posterior marginal.

We previously described how to recursively compute the  $\alpha$ 's in a *left-to-right* fashion. We now describe how to recursively compute the  $\beta$ 's in a *right-to-left* fashion.

$$\begin{aligned}
 \beta_{t-1}(i) &= \mathbb{P}(\mathbf{x}_{t:T} | z_{t-1} = i) \\
 &= \sum_j \mathbb{P}(z_t = j, \mathbf{x}_t, \mathbf{x}_{t+1:T} | z_{t-1} = i) \\
 &= \sum_j \mathbb{P}(\mathbf{x}_{t+1:T} | z_t = j, \cancel{z_{t-1} = i}, \mathbf{x}_t) \mathbb{P}(z_t = j, \mathbf{x}_t | z_{t-1} = i) \\
 &= \sum_j \mathbb{P}(\mathbf{x}_{t+1:T}, z_t = j) \mathbb{P}(\mathbf{x}_t | z_t = j, \cancel{z_{t-1} = i}) \mathbb{P}(z_t = j | z_{t-1} = i) \\
 &= \sum_j \beta_t(j) \psi_t(j) \Psi(i, j)
 \end{aligned}$$

In matrix-vector form:  $\beta = \Psi(\psi_t \odot \beta_t)$  We can think of this algorithm as passing "messages" from left to right and then from right to left and then combining them at each node.

**Complexity** It takes  $O(K^2T)$  time since at each step we perform a  $K \times K$  We can think of this algorithm as passing "messages" from left to right and then from right to left, and then combining them at each node. Finally

$$\gamma_t(j) \propto \alpha_t(j) \beta_t(j)$$

## The Viterbi Algorithm

**Purpose** Compute the most probable sequence of states in a chain-structured graphical model:  $\mathbf{z}^* = \arg \max_{\mathbf{z}_{1:T}} \mathbb{P}(\mathbf{z}_{1:T} | \mathbf{x}_{1:T})$ .

This is equivalent to computing a shortest path through *trellis diagram* where the nodes are possible states at each time step and the node and edge weights are log probabilities.

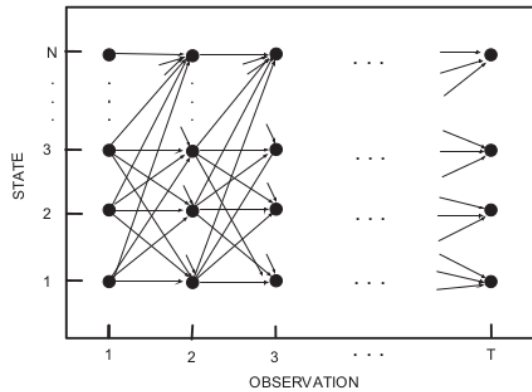


Figure 8.2: Trellis diagram: states vs time for a Markov chain.

**MAP vs MPE** The (*jointly*) most probable sequence of states:  $\mathbf{z}^*$  is not necessarily the same as the sequence of (*marginally*) most probable states, *Maximize of the Posterior Marginals* (MPM):  $\hat{\mathbf{z}} = \left( \arg \max_z \mathbb{P}(z | \mathbf{x}_{1:T}) \right)_{z_1 \leq z \leq z_T}$ .

In one hand the joint MAP estimate is that is always globally consistent on the other hand the MPM estimate can be more robust

**Detail of the algorithm** Although is tempting to implement Viterbi by replacing the sum-operator with max-operator in the forward-backwards, it can lead to incorrect results if there are multiple equally probably joint assignments. Therefore Viterbi uses max-product in the forward pass and a traceback procedure in the backwards pass.

Let's define the probability of ending up in state  $j$  at time  $t$  given that we take the most probable path:

$$\delta_t(j) \triangleq \max_{\mathbf{z}_{1:t-1}} \mathbb{P}(\mathbf{z}_{1:t-1}, z_t = j | \mathbf{x}_{1:t})$$

The key insight is that the most probable path to state  $j$  at time  $t$  must consist of the most probable path to some other state  $i$  at time  $t-1$ , followed by a transition from  $i$  to  $j$ .

1. Hence  $\delta_t(j) = \max_i \delta_{t-1}(i) \Psi(i, j) \psi_t(j)$
2. Keeping track of the most likely previous state for each possible state that we end up:  $a_t(j) = \arg \max_i \delta_{t-1}(i) \Psi(i, j) \psi_t(j)$ . It tells us the most likely previous state on the most probable path to  $z_t = j$ .
3. initialize  $\delta_1(j) = \pi_j \psi_1(j)$
4. compute the most probable final state  $z_T^* = \arg \max_j \delta_T(j)$
5. using traceback we can compute the most probable sequence of states:  $z_t^* = a_{t+1}(z_{t+1}^*)$

**Complexity** It is  $O(K^2T)$

## Forwards filtering, backwards sampling

**Purpose** Sample paths from posterior:  $\mathbf{z}_{1:T}^s \hookrightarrow \mathbb{P}(\mathbf{z}_{1:T} | \mathbf{x}_{1:T})$  Without being obliged to perform forward-backwards pass and then an additional forward sampling pass.

**Steps** The key insight is that to do forward pass and then perform sampling in the backward pass we can write the joint from right to left using  $\mathbb{P}(\mathbf{z}_{1:T} | \mathbf{x}_{1:T}) = \mathbb{P}(z_T | \mathbf{x}_{1:T}) \prod_{t=T-1}^1 \mathbb{P}(z_t | z_{t+1}, \mathbf{x}_{1:t})$

Then we can sample  $z_t$  given future sampled states using:

$$z_t^s \hookrightarrow \mathbb{P}(z_t | z_{t+1:T}, \mathbf{x}_{1:T}) = \mathbb{P}(z_t | z_{t+1}, \underline{\mathbf{z}_{t+2:T}}, \underline{\mathbf{x}_{t+1:T}}) = \mathbb{P}(z_{t+1} | z_{t+1}^s, \mathbf{x}_{1:t})$$

The sampling distribution is given by:

$$\begin{aligned} \mathbb{P}(z_t = i | z_{t+1} = j, \mathbf{x}_{1:t}) &= \mathbb{P}(z_t | z_{t+1}, \mathbf{x}_{1:t}, \underline{\mathbf{x}_{t+1:T}}) \\ &= \frac{\mathbb{P}(z_{t+1}, z_t | \mathbf{x}_{1:t+1})}{\mathbb{P}(z_{t+1} | \mathbf{x}_{1:t+1})} \\ &\propto \frac{\mathbb{P}(\mathbf{x}_{t+1} | z_{t+1}, \underline{\mathbf{z}_{t+2:T}}, \underline{\mathbf{x}_{t+1:T}}) \mathbb{P}(z_{t+1}, z_t | \mathbf{x}_{1:t})}{\mathbb{P}(z_{t+1}, z_t | \mathbf{x}_{1:t+1})} \\ &= \frac{\mathbb{P}(\mathbf{x}_{t+1} | z_{t+1}) \mathbb{P}(z_{t+1} | z_t, \mathbf{x}_{1:t}) \mathbb{P}(z_t | \mathbf{x}_{1:t})}{\mathbb{P}(z_{t+1}, z_t | \mathbf{x}_{1:t+1})} \\ &= \frac{\phi_{t+1}(j) \psi(i, j) \alpha_t(i)}{\alpha_{t+1}(j)} \end{aligned}$$

### 8.1.4 Learning for HMMs

We want to estimate the parameters  $\theta = (\pi, \mathbf{A}, \mathbf{B})$  where  $\pi(i) = \mathbb{P}(z_i = i)$  is the initial state distribution,  $\mathbf{A}(i, j) = \mathbb{P}(z_t = j | z_{t-1} = i)$  the transition matrix and  $\mathbf{B}$  the parameters of the class-conditional densities  $\mathbb{P}(\mathbf{x}_t | z_t = j)$

**Training with fully observed data** If we observe the hidden state sequences, we can compute the MLEs for  $\mathbf{A}$  and  $\boldsymbol{\pi}$  exactly.

In using a conjugate prior we can easily compute the posterior.

The details on how to [estimate  \$B\$  depend on the form of the observation model](#).

**EM for HMMs (Baum-Welch Algorithm)** If  $z_t$  are not observed we are in a situation analogous to fitting a mixture model. The most common approach is to [use EM algorithm to find the MLE or MAP parameters](#).

**E step** It is straightforward to show that the expected complete data log likelihood is given by:

$$Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{old}) = \sum_{k=1}^K \mathbb{E}(N_k^1) \log(\pi_k) + \sum_{j=1}^K \sum_{k=1}^K \mathbb{E}(N_{jk}) \log(A_{jk}) + \sum_{i=1}^N \sum_{t=1}^{T_i} \sum_{k=1}^K \mathbb{P}(z_t = k | \mathbf{x}_i, \boldsymbol{\theta}^{old}) \log(\mathbb{P}(\mathbf{x}_i | \phi_k))$$

$$\text{where } \begin{cases} \mathbb{E}(N_k^1) &= \sum_{i=1}^N \mathbb{P}(z_{i1} = k | \mathbf{x}_i, \boldsymbol{\theta}^{old}) \\ \mathbb{E}(N_{jk}) &= \sum_{i=1}^N \sum_{t=1}^{T_i} \mathbb{P}(z_{i,t-1} = j, z_{i,t} = k | \mathbf{x}_i, \boldsymbol{\theta}^{old}) \\ \mathbb{E}(N_j) &= \sum_{i=1}^N \sum_{t=1}^{T_i} \mathbb{P}(z_{i,t} = k | \mathbf{x}_i, \boldsymbol{\theta}^{old}) \end{cases}$$

puted by running the forward-backward algorithm on each sequence.

In particular, this algorithm computes the following smoothed node and edge marginals:

$$\begin{cases} \gamma_{i,t}(j) \triangleq \mathbb{P}(z_t = j | \mathbf{x}_{i,1:T_i}, \boldsymbol{\theta}) \\ \xi_{i,t}(j, k) \triangleq \mathbb{P}(z_{t-1} = j, z_t = k | \mathbf{x}_{i,1:T}, \boldsymbol{\theta}) \end{cases}$$

**M step** From results in mixture of Multinoulli:  $\hat{A}_{jk} = \frac{\mathbb{E}(N_k)}{\sum_{k'} \mathbb{E}(N_{jk'})}$ ,  $\hat{\pi}_k = \frac{\mathbb{E}(N_k^1)}{N}$

For a Multinoulli observation model the expected sufficient statistics are:  $\mathbb{E}(M_{jl}) = \sum_{i=1}^N \sum_{t=1}^{T_i} \gamma_{i,t}(j) \mathbb{1}_{\{x_{i,t}=l\}} =$

$$\sum_{i=1}^N \sum_{t: x_{i,t}=l} \gamma_{i,t}(j).$$

The M step has the form  $\hat{B}_{jl} = \frac{\mathbb{E}(M_{jl})}{\mathbb{E}(N_j)}$ .

For a Gaussian observation model: 
$$\begin{cases} \mathbb{E}(\bar{x}_k) = \sum_{i=1}^N \sum_{t=1}^{T_i} \gamma_{i,t}(k) \mathbf{x}_{i,t} \\ \mathbb{E}((\bar{x}_k)^T) = \sum_{i=1}^N \sum_{t=1}^{T_i} \gamma_{i,t}(k) \mathbf{x}_{i,t} \mathbf{x}_{i,t}^T \end{cases}$$

The M step becomes :

$$\begin{cases} \hat{\boldsymbol{\mu}}_k = \frac{\mathbb{E}(\bar{x}_k)}{\mathbb{E}(N_k)} \\ \hat{\boldsymbol{\Sigma}}_k = \frac{\mathbb{E}((\bar{x}_k)^T) - \mathbb{E}(N_k) \hat{\boldsymbol{\mu}}_k \hat{\boldsymbol{\mu}}_k^T}{\mathbb{E}(N_k)} \end{cases}$$

**Bayesian methods for 'fitting' HMMs** We can use variational Bayes EM method, the E step uses forwards-backwards but where we plug in the posterior mean parameters instead of the MAP estimates. The M step updates the parameters of the conjugate posteriors instead of updating the parameters themselves.

**Model selection**

### Choosing the number of hidden states

- using grid-search over range of  $K$ 's
- Variational Bayes to "extinguish" unwanted components
- Use an "infinite HMM" which is based on the hierarchical Dirichlet process

## 8.2 State Model

### 8.2.1 Basics

**Definition** It is an hidden Markov models, except the **hidden states are continuous**. The model can

be written in the following generic form: 
$$\begin{cases} \mathbf{z}_t = g(\mathbf{u}_t, \mathbf{z}_{t-1}, \boldsymbol{\epsilon}_t) \\ \mathbf{y}_t = h(\mathbf{z}_t, \mathbf{u}_t, \boldsymbol{\delta}_t) \end{cases}$$
 where  $\mathbf{z}_t$  is the hidden state  $\mathbf{u}_t$  is an optimal input or control signal,  $\mathbf{y}_t$  is the observation,  $g$  is the *transition model*,  $h$  is the *observation model* and  $\boldsymbol{\epsilon}_t$  is the system noise at time  $t$ .

- *transition model*:  $\mathbf{z}_t = \mathbf{A}_t \mathbf{z}_{t-1} + \mathbf{B}_t \mathbf{u}_t + \boldsymbol{\epsilon}_t$
- *observation model*  $\mathbf{y}_t = \mathbf{C}_t \mathbf{z}_t + \mathbf{D}_t \mathbf{u}_t + \boldsymbol{\delta}_t$
- *system noise*:  $\boldsymbol{\epsilon}_t \hookrightarrow \mathcal{N}(\mathbf{0}, \mathbf{Q}_t)$
- *observation noise*:  $\boldsymbol{\delta}_t \hookrightarrow \mathcal{N}(\mathbf{0}, \mathbf{R}_t)$

This model is called a Linear-Gaussian State Space Model (LG-SSM), if the parameters  $\boldsymbol{\theta}_t = (\mathbf{A}_t, \mathbf{B}_t, \mathbf{C}_t, \mathbf{D}_t, \mathbf{Q}_t, \mathbf{R}_t)$  are independent of time, the model is called *stationary*.

### 8.2.2 Time Series Forecasting

The idea is to create a generative model of the data in terms of latent processes which capture different aspects of the signal.

**Local level model** The simplest latent process: 
$$\begin{cases} y_t = a_t + \epsilon_t^y, \epsilon_t^y \hookrightarrow \mathcal{N}(0, R) \\ a_t = a_{t-1} + \epsilon_t^a, \epsilon_t^a \hookrightarrow \mathcal{N}(0, Q) \end{cases}$$
 where the hidden state is just  $z_t = a_t$

**Local linear trend** Many time series exhibit linear trends upward or downward, at least locally. We can model this by letting the level  $a_t$  change by an amount  $b_t$  at each step: 
$$\begin{cases} y_t = a_t + \epsilon_t^y, \epsilon_t^y \hookrightarrow \mathcal{N}(0, R) \\ a_t = a_{t-1} + b_{t-1} + \epsilon_t^a, \epsilon_t^a \hookrightarrow \mathcal{N}(0, Q_a) \\ b_t = b_{t-1} + \epsilon_t^b, \epsilon_t^b \hookrightarrow \mathcal{N}(0, Q_b) \end{cases}$$

**Seasonality** This can be modeled by adding a latent process consisting of a series offset terms  $c_t$  which sum to zero (on average) over a complete cycle of  $S$  steps: 
$$c_t = -\sum_{s=1}^{S-1} c_{t-s} + \epsilon_t^c, \epsilon_t^c \hookrightarrow \mathcal{N}(0, Q_c)$$

**ARMA(Auto-Regressive Moving-Average) models** 
$$x_t = \sum_{i=1}^p \alpha_i x_{t-i} + \sum_{j=1}^q \beta_j w_{t-j} + v_t$$
 where  $v_t$  and  $w_t$  follow a  $\mathcal{N}(0, 1)$

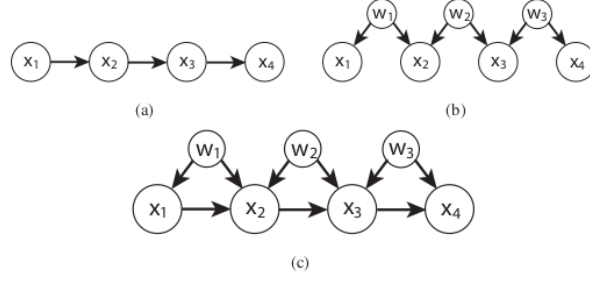


Figure 8.3: (a)  $\rightarrow$   $AR(1)$  (b)  $\rightarrow$   $MA(1)$  and (c)  $\rightarrow$   $ARMA(1,1)$

The structural approach to time series is often easier to understand than the ARMA approach. In addition it allows the parameters to evolve over time, which makes the models more adaptive to non-stationary.

### 8.2.3 Inference in LG-SSM (Linear Gaussian - State Space Models)

**Kalman filtering algorithm** It is Kalman filter is an [algorithm for exact Bayesian filtering for linear-Gaussian state space models](#) that sequentially computes  $\mathbb{P}(\mathbf{z}_t | \mathbf{y}_{1:t})$  for each  $t$ .

#### Prediction step

$$\begin{aligned} \mathbb{P}(\mathbf{z}_t | \mathbf{y}_{1:t-1}, \mathbf{u}_{1:t}) &= \int \mathcal{N}(\mathbf{z}_t | \mathbf{A}_t \mathbf{z}_{t-1} + \mathbf{B}_t \mathbf{u}_t, \mathbf{Q}_t) \mathcal{N}(\mathbf{z}_{t-1} | \boldsymbol{\mu}_{t-1}, \boldsymbol{\Sigma}_{t-1}) d\mathbf{z}_{t-1} \\ &= \mathcal{N}(\mathbf{z}_t | \boldsymbol{\mu}_{t|t-1}, \boldsymbol{\Sigma}_{t|t-1}) \\ \boldsymbol{\mu}_{t|t-1} &\triangleq \mathbf{A}_t \boldsymbol{\mu}_{t-1} + \mathbf{B}_t \mathbf{u}_t \\ \boldsymbol{\Sigma}_{t|t-1} &\triangleq \mathbf{A}_t \boldsymbol{\Sigma}_{t-1} \mathbf{A}_t^T + \mathbf{Q}_t \end{aligned}$$

**Measurement step** Can be computed using Bayes rule:

$$\mathbb{P}(\mathbf{z}_t | \mathbf{y}_t, \mathbf{y}_{1:t-1}, \mathbf{u}_{1:t}) \propto \mathbb{P}(\mathbf{y}_t | \mathbf{z}_t, \mathbf{u}_{1:t}) \mathbb{P}(\mathbf{z}_t | \mathbf{y}_{1:t-1}, \mathbf{u}_{1:t}).$$

$$\text{We show that this is given by: } \begin{cases} \mathbb{P}(\mathbf{z}_t | \mathbf{y}_{1:t}, \mathbf{u}_t) &= \mathcal{N}(\mathbf{z}_t | \boldsymbol{\mu}_t, \boldsymbol{\Sigma}_t) \\ \boldsymbol{\mu}_t &= \boldsymbol{\mu}_{t|t-1} + \mathbf{K}_t \mathbf{r}_t \\ \boldsymbol{\Sigma}_t &= (\mathbf{I} - \mathbf{K}_t \mathbf{C}_t) \boldsymbol{\Sigma}_{t|t-1} \end{cases} \quad \text{where } \mathbf{r}_t \text{ is the residual given by the}$$

$$\text{difference between our predicted observation and the actual observation: } \begin{cases} \mathbf{r}_t \triangleq \mathbf{y}_t - \hat{\mathbf{y}}_t \\ \hat{\mathbf{y}}_t \triangleq \mathbb{E}(\mathbf{y}_t | \mathbf{y}_{1:t-1}, \mathbf{u}_{1:t}) = \mathbf{C}_t \boldsymbol{\mu}_{t|t-1} + \mathbf{D}_t \mathbf{u}_t \end{cases}$$

and  $\mathbf{K}_t$  is the Kalman gain matrix given by  $\mathbf{K}_t \triangleq \boldsymbol{\Sigma}_{t|t-1} \mathbf{C}_t^T \mathbf{S}_t^{-1}$  where  $\mathbf{S}_t \triangleq \text{Cov}(\mathbf{r}_t | \mathbf{y}_{1:t-1}, \mathbf{u}_{1:t})$

**The Kalman smoothing algorithm** In offline setting we can wait until all the data has arrived and then compute  $\mathbb{P}(\mathbf{z}_t | \mathbf{y}_{1:T})$ . By conditioning on past and future data our uncertainty will be significantly reduced.

**Comparison to the forwards-backwards algorithm for HMMs** It turns out that we can rewrite the Kalman smoother in a modified form which makes it more similar to forwards-backwards for HMMs.

## Chapter 9

# Markov Random Field

### 9.0.1 Basics

**Definition** Similar to Bayesian Network in its representation dependencies except that MRF are undirected and may be cyclic.

#### Conditional independence properties

- **Global Markov** property: for a sets of nodes  $A, B$  and  $C$ ,  $\mathbf{x}_A \perp_G \mathbf{x}_B | \mathbf{x}_C$  iff  $C$  separates  $A$  from  $B$  in the graph  $G$ .  
Meaning that when we remove the nodes of  $C$  there is no way to connect any nodes in  $A$  to any nodes in  $B$
- **Undirected local Markov** property: consider the set of nodes rendering a given node  $N$  conditionally independent of all other nodes this is the **Markov blanket** denoted  $mb(N) = N \perp \mathcal{V} \setminus cl(N) | mb(N)$ , with  $cl(N) \triangleq mb(N) \cup \{N\}$  is the *closure* of node  $N$ . One can show that in a UGM a node's Markov blanket is its set of immediate neighbors.
- **Pairwise Markov** property: two nodes are conditionally independent given the rest if there is no direct edge between them:  $s \perp N | \mathcal{V} \setminus \{s, N\} \Leftrightarrow G_{sN} = 0$

#### Parametrization of MRFs

**Hammersley-Clifford** A positive distribution  $p(\mathbf{y}) > 0$  satisfies the CI properties of an undirected graph  $G$  iff  $p$  can be represented as a product of factors:

$$p(\mathbf{y}|\boldsymbol{\theta}) = \frac{1}{Z(\boldsymbol{\theta})} \prod_{c \in \mathcal{C}} \psi_c(\mathbf{y}_c | \boldsymbol{\theta}_c)$$

where  $\mathcal{C}$  is the set of all (maximal) cliques of  $G$ , and  $Z(\boldsymbol{\theta})$  is the **partition function** given by  $Z(\boldsymbol{\theta}) \triangleq$

$$\sum_{\mathbf{x}} \prod_{c \in \mathcal{C}} \psi_c(\mathbf{y}_c | \boldsymbol{\theta}_c)$$

**Connection with statistical physics** There is a model known as the **Gibbs distribution** which can be written as follows:  $p(\mathbf{y}|\boldsymbol{\theta}) = \frac{1}{Z(\boldsymbol{\theta})} \exp(E(\mathbf{y}_c | \boldsymbol{\theta}_c))$  where  $E(\mathbf{y}_c) > 0$  is the energy associated with the variables in clique  $c$ . We can convert this to UGM by defining  $\psi_c(\mathbf{y}_c | \boldsymbol{\theta}_c) = \exp(-E(\mathbf{y}_c | \boldsymbol{\theta}_c))$

**Representing potential functions** We define the **log potentials as a linear function of the parameters**:  $\log(\psi_c(\mathbf{y}_c)) \triangleq \boldsymbol{\phi}_c(\mathbf{y}_c)^T \boldsymbol{\theta}_c$  where  $\boldsymbol{\phi}_c(\mathbf{x}_c)$  is a feature vector derived from the values of the variables  $\mathbf{y}_c$ .

The resulting log probability has the form:  $\log(p(\mathbf{y}|\boldsymbol{\theta})) \triangleq \sum_c \boldsymbol{\phi}_c(\mathbf{y}_c)^T \boldsymbol{\theta}_c - Z(\boldsymbol{\theta})$  this also known as a maximum entropy

**Learning** TO DO

**Conditional Markov Random Field** TO DO

# Part III

## Classical Learning



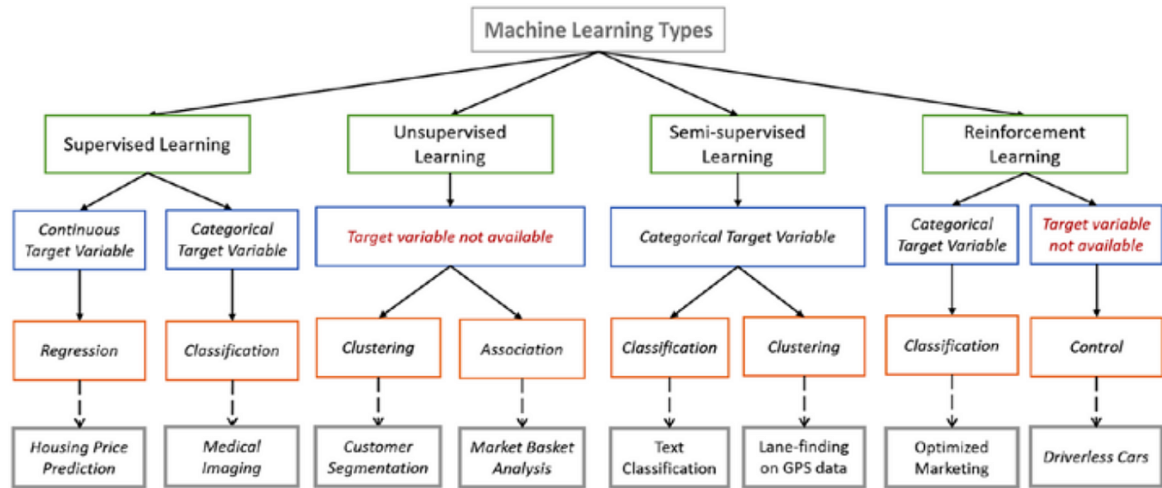


Figure 9.1: Types of machine learning methods

# Chapter 10

## Supervised Learning

With the *generative* approach we create a joint model of the form  $\mathbb{P}(y, \mathbf{x})$ , and then to condition on  $\mathbf{x}$ , thereby deriving  $\mathbb{P}(y|\mathbf{x})$ .

Alternatively, *fitting directly a model of the form  $\mathbb{P}(y|\mathbf{x})$*  is a *discriminative* approach.

- *Easy to fit*: generative classifiers is usually very easy
- *Fit classes separately?*: in a generative classifier we estimate the parameters of each class conditional density independently
- *Handle missing features easily?*: in a generative classifier there is a natural way to handle missing data:  $\mathbb{P}(\mathbf{x}_i, r_i | \boldsymbol{\theta}, \boldsymbol{\phi}) = \mathbb{P}(r_i | \mathbf{x}_i, \boldsymbol{\phi}) \mathbb{P}(\mathbf{x}_i | \boldsymbol{\theta})$  where  $\boldsymbol{\phi}$  are the parameters controlling whether the item is observed or not. Missing completely at random (MCAR):  $\mathbb{P}(r_i | \mathbf{x}_i, \boldsymbol{\phi}) = \mathbb{P}(r_i | \mathbf{x}_i^0 | \boldsymbol{\phi})$ , missing at random (MAR):  $\mathbb{P}(r_i | \mathbf{x}_i, \boldsymbol{\phi}) = \mathbb{P}(r_i | \mathbf{x}_i^0, \boldsymbol{\phi})$
- *Can handle unlabeled training data?*: Fairly easy in using generative models
- *Symmetric in inputs and outputs?*: We can run a generative model "backward" and infer probable inputs given the output by computing  $\mathbb{P}(\mathbf{x}|y)$
- *Can handle feature preprocessing?*: discriminative methods allow to preprocess the input in arbitrary ways, by replacing  $\mathbf{x}$  with  $\phi(\mathbf{x})$
- *Well-calibrated probabilities?*: discriminative models are usually better calibrated in terms of their probability estimates.

### 10.1 Classification

#### 10.1.1 Naive Bayes classifiers

**Purpose** *Classifying vectors of discrete-valuated features  $\mathbf{x} \in \llbracket 1, K \rrbracket^D$* , where  $K$  is the number of values that each feature is able to take, and  $D$  the number of features.

#### Assumptions

- *Features are conditionally independent given the class label*:  $\forall (j, j') \in \llbracket 1, D \rrbracket^2, j \neq j' \Rightarrow \mathbf{x}_j \perp \mathbf{x}_{j'} | y = c$
- *Independence of the observations*.

**Theory** As a *generative* model, meaning of the form:  $\mathbb{P}(y = c | \mathbf{x}, \boldsymbol{\theta}) \propto \mathbb{P}(\mathbf{x} | y = c, \boldsymbol{\theta}) \mathbb{P}(y = c | \boldsymbol{\theta})$ . The key of such models is the possibility to *specify a suitable form for the class-conditional density  $\mathbb{P}(\mathbf{x} | y = c, \boldsymbol{\theta})$*  which defines what kind of data we expect to see in each class. And with the independence assumption we have:

$$\mathbb{P}(\mathbf{x} | y = c, \boldsymbol{\theta}) = \prod_{j=1}^D \mathbb{P}(x_j | y = c, \boldsymbol{\theta}_{jc})$$

with all  $\mathbb{P}(x_j|y = c, \theta_{jc})$  being able to follow a *normal*, *bernoulli* or *multinoulli* distribution. Training a NBC consists in computing the MLE or the MAP estimate for the parameters.

For a single observation  $\mathbb{P}(\mathbf{x}_i, y_i|\boldsymbol{\theta}) = \mathbb{P}(y_i|\boldsymbol{\pi}) \prod_j \mathbb{P}(x_{ij}|\boldsymbol{\theta}_j) = \prod_c \pi_c^{\mathbb{1}(y_i=c)} \prod_j \prod_c \mathbb{P}(x_{ij}|\boldsymbol{\theta}_{jc})^{\mathbb{1}(y_i=c)}$

Hence the *log-likelihood*:  $\log(\mathcal{D}|\boldsymbol{\theta}) = \sum_{c=1}^C N_c \log(\pi_c) + \sum_{j=1}^D \sum_{c=1}^C \sum_{i:y_i=c} \log(\mathbb{P}(x_{ij}|\boldsymbol{\theta}_{jc}))$

By optimizing the above equation we are able to find the  $(\boldsymbol{\theta}_{jc})_{\substack{1 \leq j \leq D \\ 1 \leq c \leq C}}$  and we can then use them to predict the output of an observation  $\mathbf{x}$  as:  $\mathbb{P}(y = c|\mathbf{x}, \mathcal{D}) \propto \mathbb{P}(y = c|\mathcal{D}) \prod_{j=1}^D \mathbb{P}(x_j|y = c, \mathcal{D})$

## Strengths

- Simple model, for  $C$  classes and  $D$  features, and hence *relatively immune to overfitting*

## Weaknesses

- *Unaccuracy* because of the strong independence assumption

**Relationships with other methods** *Logistic Regression*: for discrete inputs *Naive Bayesian Classifiers* form a generative-discriminant pair with *Multinomial Logistic Regression*: each NBC can be considered a way of fitting a probability model that optimizes the joint likelihood  $\mathbb{P}(C, \mathbf{x})$ , while *Multinomial Logistic Regression* fits the same probability to optimize the conditional  $\mathbb{P}(C|\mathbf{x})$

## Examples of application

- Classifying documents using bag of words
- Determining the gender of a person, based on measured features

### 10.1.2 Linear/Quadratic Discriminant Analysis

**Purpose** It consists in defining the class conditional densities in a generative classifier:  $\mathbb{P}(\mathbf{x}|y = c, \boldsymbol{\theta}) = \mathcal{N}(\mathbf{x}, \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)$

As for a generative classifier we have the following equation:

$$\mathbb{P}(y = c|\mathbf{x}, \boldsymbol{\theta}) = \frac{\overbrace{\mathbb{P}(\mathbf{x}|y = c, \boldsymbol{\theta})}^{\text{class-conditional density}} \overbrace{\mathbb{P}(y = c|\boldsymbol{\theta})}^{\text{class prior}}}{\sum_{c'} \mathbb{P}(y = c'|\boldsymbol{\theta}) \mathbb{P}(\mathbf{x}|y = c', \boldsymbol{\theta})}$$

## Assumptions

- Features are normally distributed in each class grouping variable.
- *Homoscedasticity* for LDA: variances *among group* variables are the same across levels of predictors.
- *Independence of the observations*.

## Theory of Quadratic Discriminant Analysis

$$\mathbb{P}(y = c|\mathbf{x}, \boldsymbol{\theta}) = \frac{|2\pi \det(\boldsymbol{\Sigma}_c)|^{-\frac{1}{2}} \exp\left(-\frac{1}{2}[\mathbf{x} - \boldsymbol{\mu}_c]^T \boldsymbol{\Sigma}_c^{-1}[\mathbf{x} - \boldsymbol{\mu}_c]\right) \pi_c}{\sum_{c'} |2\pi \det(\boldsymbol{\Sigma}_{c'})|^{-\frac{1}{2}} \exp\left(-\frac{1}{2}[\mathbf{x} - \boldsymbol{\mu}_{c'}]^T \boldsymbol{\Sigma}_{c'}^{-1}[\mathbf{x} - \boldsymbol{\mu}_{c'}]\right) \pi_{c'}}$$

The threshold of this results will be a quadratic function of  $\mathbf{x}$ .

## Theory of Linear Discriminant Analysis

**LDA** Same equation than above but this time,  $\forall c \in \llbracket 1, C \rrbracket \Sigma_c = \Sigma$ , then quadratic term  $\mathbf{x}^T \Sigma^{-1} \mathbf{x}$  will cancel out from numerator and denominator. Also Then by considering the above cancellation and the fact that  $\Sigma$  is diagonal as well as the evidence is considered as a constant, we have:

$$\begin{aligned} \mathbb{P}(y = c | \mathbf{x}, \boldsymbol{\theta}) &\propto \exp(\log(\pi_c) + \boldsymbol{\mu}_c^T \Sigma^{-1} \mathbf{x}) \\ &= \exp(\boldsymbol{\beta}_c^T \mathbf{x} + \gamma_c) \end{aligned}$$

Note also that we have exactly:  $\mathbb{P}(y = c | \mathbf{x}, \boldsymbol{\theta}) = \frac{e^{\boldsymbol{\beta}_c^T \mathbf{x} + \gamma_c}}{\sum_{c'} e^{\boldsymbol{\beta}_{c'}^T \mathbf{x} + \gamma_{c'}}} = S(\boldsymbol{\eta})_c$ . With  $\boldsymbol{\eta} = (\boldsymbol{\beta}_c^T \mathbf{x} + \gamma_c)_{1 \leq c \leq C}$

We recognize the *softmax* function.

**RDA** Regularized Discriminant Analysis, assuming the covariance matrix is shared across the classes as in LDA, we perform MAP estimation of  $\boldsymbol{\Sigma} \hookrightarrow \text{InverseWishart}(\text{diag}(\hat{\boldsymbol{\Sigma}}_{mle}), \nu_0)$  then:

$$\hat{\boldsymbol{\Sigma}} = \lambda \text{diag}(\hat{\boldsymbol{\Sigma}}_{mle}) + (1 - \lambda) \hat{\boldsymbol{\Sigma}}_{mle}$$

where  $\lambda$  controls the amount of regularization which is related to the strength of the prior  $\nu_0$

### MLE for discriminant analysis

$$\log(\mathbb{P}(\mathcal{D} | \boldsymbol{\theta})) = \sum_{i=1}^n \sum_{c=1}^C \mathbb{1}_{\{y_i=c\}} \log(\pi_c) + \sum_{c=1}^C \sum_{i: y_i=c} \log(\mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c))$$

And we find 
$$\begin{cases} \hat{\boldsymbol{\mu}}_c &= \frac{1}{N_c} \sum_{i: y_i=c} \mathbf{x}_i \\ \hat{\boldsymbol{\Sigma}}_c &= \frac{1}{N_c} (\mathbf{x}_i - \hat{\boldsymbol{\mu}}_c)(\mathbf{x}_i - \hat{\boldsymbol{\mu}}_c)^T \end{cases}$$

### Strategies for preventing overfitting

- use a diagonal covariance matrix for each class
- use a full/diagonal covariance matrix [shared across the classes](#)
- use a [full covariance matrix but impose a prior and then integrate it out](#)
- [project the data into a low dimensional subspace and fit the Gaussian there](#)

### Strengths

- more common MVN(Multivariate Normal) model which is the most widely used joint probability density function for continuous variables.
- good simplicity-efficiently trade-off

### Weaknesses

- Multicollinearity: predictive power can decrease with an increased correlation between predictor variables.

### Relationships with other methods

- ANOVA
- Logistic Regression
- PCA: look for linear combination of variables which best explain the model
- Nearest Shrunken Centroids

## Examples of application

- bankruptcy prediction
- dimension reduction for face recognition

### 10.1.3 Nearest Shrunk Centroids Classifier

**Purpose** The basic idea is to perform MAP estimation for diagonal LDA with a sparsity-promoting (Laplace) prior

#### Assumptions

**Theory** Define the class-specific feature mean  $\mu_{cj}$  in terms of the class-independent feature mean  $m_j$  and a class-specific offset  $\Delta_{cj}$ . Thus:

$$\mu_{jc} = m_j + \Delta_{cj}$$

We will then put a prior on the  $\Delta_{cj}$  terms to encourage them to be strictly zero and compute MAP estimate.

If for feature  $j \forall c \in [1, C] \Delta_{cj} = 0$  then feature  $j$  will no play role in the classification decision, since  $\mu_{cj}$  will be independent of  $c$ .

Thus feature that are not discriminative are automatically ignored.

#### Strengths

- Sparsity  $\rightarrow$  more interpretable model

#### Weaknesses

#### Relationships with other methods

## Examples of application

- Gene expression classification

### 10.1.4 Fisher's Linear Discriminant Analysis (FLDA)

**Purpose** An alternative way to the above discriminant analysis is to reduce the dimensionality of the features  $\mathbf{x} \in \mathbb{R}^D$  and then fit a Multivariate Normal to the resulting low-dimensional features  $\mathbf{z} \in \mathbb{R}^L$ . PCA would not be a good idea because it is unsupervised and the low-dimensional resulting features would not be optimal for the classification problem.

A better option would be to find a matrix  $\mathbf{W}$  such that the low-dimensional data can be classified as well as possible using a Gaussian class-conditional density model, this is the FLDA.

#### Assumptions

- Gaussian class-conditional: reasonable since we are computing linear combination of potentially non-Gaussian features.

**Theory** For 2 classes Let us define 
$$\begin{cases} \mu_1 = \frac{1}{N_1} \sum_{i:y_i=1} \mathbf{x}_i \\ \mu_2 = \frac{1}{N_2} \sum_{i:y_i=2} \mathbf{x}_i \end{cases}, \text{ and } m_k = \mathbf{w}^T \mu_k \text{ being the projection of each}$$

mean onto the line  $\mathbf{w}$ . Also let  $z_i = \mathbf{w}^T \mathbf{x}_i$  be the projection of the data onto the line  $\mathbf{w}$  and finally  $s_k^2 = \sum_{i:y_i=k} (z_i - m_k)^2$ .

The goal is to find  $\mathbf{w}$  such that we maximize the distance between the means,  $m_1 - m_2$ , while also ensuring the projected clusters are "tight":

$$\mathbf{J}(\mathbf{w}) = \frac{(m_2 - m_1)^2}{s_1^2 + s_2^2} = \frac{\mathbf{w}^T \mathbf{S}_B \mathbf{w}}{\mathbf{w}^T \mathbf{S}_W \mathbf{w}}$$

with the *between-class scatter matrix*:  $\mathbf{S}_B = (\mu_2 - \mu_1)(\mu_2 - \mu_1)^T$  and *within-class scatter matrix*:  $\mathbf{S}_W = \sum_{i:y_i=1} (\mathbf{x}_i - \mu_1)(\mathbf{x}_i - \mu_1)^T + \sum_{i:y_i=2} (\mathbf{x}_i - \mu_2)(\mathbf{x}_i - \mu_2)^T$ . One can show that  $\mathbf{J}(\mathbf{w})$  is maximized when  $\mathbf{S}_B \mathbf{w} = \lambda \mathbf{S}_W \mathbf{w}$  where  $\lambda = \frac{\mathbf{w}^T \mathbf{S}_B \mathbf{w}}{\mathbf{w}^T \mathbf{S}_W \mathbf{w}}$ . If  $\mathbf{S}_W$  is invertible we can convert it to a regular eigenvalue problem:  $\mathbf{S}_W^{-1} \mathbf{S}_B \mathbf{w} = \lambda \mathbf{w}$

### Strengths

- Classification in taking in account the response label.

### Weaknesses

- FLDA is restricted to using  $L \leq C - 1$  dimensions, regardless of  $D$ .

### Relationships with other methods

- Linear Discriminant Analysis
- PCA
- ANOVA

### Examples of application

- Speech recognition

## 10.1.5 Logistic Regression

**Purpose** For binary classification.

### Assumptions

- Independence

**Theory** The data distribution is modelled by :  $\mathbb{P}(y|\mathbf{x}) = \text{Bernoulli}(y|\sigma(\mathbf{w}^T \mathbf{x}))$

With  $\sigma$  being the *sigmoid* function, such that  $\sigma = \begin{cases} \mathbb{R} \rightarrow [0, 1] \\ x \mapsto \frac{e^x}{1 + e^x} \end{cases}$

### Maximum Likelihood Estimator

$$\begin{aligned} NLL(\mathbf{w}) &= - \sum_{i=1}^N \log \left( \hat{y}_i^{\mathbb{1}_{\{y_i=1\}}} [1 - \hat{y}_i]^{\mathbb{1}_{\{y_i=0\}}} \right) \\ &= - \sum_{i=1}^N y_i \log(\hat{y}_i) + [1 - y_i] \log(1 - \hat{y}_i) \end{aligned}$$

This called *cross-entropy*

### Strengths

### Weaknesses

### Relationships with other methods

## Examples of application

## 10.2 Regression

### 10.2.1 Linear Regression

#### Purpose

#### Assumptions

#### Theory

**General** It is a model for which the data distribution (likelihood) is described by:

$$\mathbb{P}(y|\mathbf{x}, \boldsymbol{\theta}) = \mathcal{N}(y|\mathbf{w}^T \phi(\mathbf{x}), \sigma^2)$$

with  $\phi$  that can be a non-linear function, in this case we talk about *basis function expansion*.  
To estimate the parameters we can use the *maximum likelihood estimation*:  $\hat{\boldsymbol{\theta}} \triangleq \arg \max_{\boldsymbol{\theta}} \log(\mathbb{P}(\mathcal{D}|\boldsymbol{\theta}))$ .  
For computational purpose it is better to consider the minimization of the *Negative Log Likelihood* (NLL):

$$\begin{aligned} NLL(\boldsymbol{\theta}) &\triangleq -\log(p(\mathcal{D}|\boldsymbol{\theta})) \\ &= -\sum_{i=1}^n \log(\mathbb{P}(y_i|\mathbf{x}_i, \boldsymbol{\theta})) \\ &= -\sum_{i=1}^n \log\left(\left[\frac{1}{2\pi\sigma^2}\right]^{\frac{1}{2}} \exp\left(-\frac{1}{2\sigma^2} [y_i - \mathbf{w}^T \mathbf{x}_i]^2\right)\right) \\ &= \frac{n}{2} \log(2\pi\sigma^2) + \frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \mathbf{w}^T \mathbf{x}_i)^2 \\ &= \frac{n}{2} \log(2\pi\sigma^2) + \frac{1}{2\sigma^2} RSS(\mathbf{w}) \\ &= \frac{n}{2} \log(2\pi\sigma^2) + \frac{1}{2\sigma^2} \|\boldsymbol{\epsilon}\|_2^2 \end{aligned}$$

As the *MLE* for  $\mathbf{w}$  is the one minimizing the *RSS* then this method is known as *least square*.

**Derivation of the MLE** it is better to use a matrix-vector representation.

$NLL(\mathbf{w}) = \frac{1}{2} (y - \mathbf{X}\mathbf{w})^T (y - \mathbf{X}\mathbf{w}) = \frac{1}{2} \mathbf{w}^T (\mathbf{X}^T \mathbf{X}) \mathbf{w} - \mathbf{w}^T (\mathbf{X}^T \mathbf{y})$  Note that  $\mathbf{X}^T \mathbf{X}$  is the *sum of squares matrix*. Then **gradient**,  $g(\mathbf{w}) = \mathbf{X}^T \mathbf{X} \mathbf{w} - \mathbf{X}^T \mathbf{y}$  that we have to equate to zero to get  $\mathbf{X}^T \mathbf{X} \mathbf{w} = \mathbf{X}^T \mathbf{y}$  to conclude that:

$$\hat{\mathbf{w}}_{OLS} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

**Robust Linear Regression** It is very common to model the noise in regression models using a *Gaussian distribution*, meaning  $\boldsymbol{\epsilon}_i = y_i - \mathbf{w}^T \mathbf{x}_i \hookrightarrow \mathcal{I}, \sigma^{\epsilon}$ . One way to achieve *robustness* against *outliers* is to replace the Gaussian distribution for the response variable with a distribution having **heavy tails**.

| Likelihood | Prior    | Name                 |
|------------|----------|----------------------|
| Gaussian   | Uniform  | <i>Least Squares</i> |
| Gaussian   | Gaussian | <i>Ridge</i>         |
| Gaussian   | Laplace  | <i>Lasso</i>         |

**Ridge** encourages parameters to be small by using a zero-mean Gaussian prior:  $\mathbb{P}(\mathbf{w}) = \prod_{j=1}^D \mathcal{N}(\omega_j | 0, \tau^2)$ ,

where  $\frac{1}{\tau^2}$  controls the strength of the prior.

The corresponding *MAP* estimation problem becomes:  $\arg \max_{\mathbf{w}} \sum_{i=1}^n \log(\mathcal{N}(y_i | \omega_0 + \mathbf{w}^T \mathbf{x}_i, \sigma^2)) + \sum_{j=1}^D \log(\mathcal{N}(\omega_j | 0, \tau^2))$ .

After some calculus and with where  $\lambda \triangleq \frac{\sigma^2}{\tau^2}$  we deduce that:

$$\hat{\mathbf{w}}_{Ridge} = (\lambda \mathbf{I}_D + \mathbf{X}^T \mathbf{X})^{-1} \mathbf{X} \mathbf{y}$$

Advantages of Ridge regression on OLS regression:

- $(\lambda \mathbf{I}_D + \mathbf{X}^T \mathbf{X})$  is much better conditioned, and hence more likely to be invertible, than  $\mathbf{X}^T \mathbf{X}$  at least for suitable large  $\lambda$
- if we follow a *Singular Value Decomposition*  $\mathbf{X} = \mathbf{U} \mathbf{S} \mathbf{V}^T$  we find that  $\hat{\mathbf{y}} = \mathbf{X} \hat{\mathbf{w}}_{Ridge} = \sum_{j=1}^D \mathbf{u}_j \frac{\sigma_j^2}{\sigma_j^2 + \lambda} \mathbf{u}_j^T \mathbf{y}$

with  $(\sigma_j)_{1 \leq j \leq D}$  the singular value of  $\mathbf{X}$  whereas for OLS we have  $\hat{\mathbf{y}} = \mathbf{X} \hat{\mathbf{w}}_{OLS} = \sum_{j=1}^D \mathbf{u}_j \mathbf{u}_j^T \mathbf{y}$ . Meaning that with Ridge if  $\sigma_j^2$  is small compared to  $\lambda$  then direction  $\mathbf{u}_j$  will not have much effect on the prediction. In term of predictive accuracy *Ridge* regression is more interesting than *PCA* regression.

**Lasso (Least Absolute Shrinkage and Absolute Selection Operator)**  $\hat{\mathbf{w}}_{Lasso} = \text{sign}(\hat{\mathbf{w}}_{OLS}) \left[ |\hat{\mathbf{w}}_{OLS}| - \frac{\lambda}{2} \right]$

**Elastic-Net** In practice Elastic-Net often performs best, since it provides a good combination of sparsity and regularization.

### Strengths

- Simple
- Customizable to achieve robustness

### Weaknesses

- Not very powerful for non-linear data

### Relationships with other methods

- Ridge Regression has similitude with PCA

### Examples of application

#### 10.2.2 Generalized Linear Models (GLMs)

Models in which the **output density is in the exponential family** and in which **the mean parameters are a linear combination of the inputs**, passed through a possibly nonlinear function such as the logistic function.

Exponential family:  $\mathbb{P}(\mathbf{x} | \boldsymbol{\theta}) = \frac{1}{Z(\boldsymbol{\theta})} h(\mathbf{x}) e^{\boldsymbol{\theta}^T \phi(\mathbf{x})} = h(\mathbf{x}) e^{\boldsymbol{\theta}^T \phi(\mathbf{x}) - A(\boldsymbol{\theta})}$  where  $\begin{cases} Z(\boldsymbol{\theta}) = \sum_{\mathbf{x} \in \mathcal{X}^m} h(\mathbf{x}) e^{\boldsymbol{\theta}^T \phi(\mathbf{x})} \\ A(\boldsymbol{\theta}) = \log(Z(\boldsymbol{\theta})) \end{cases}$

with the *natural parameters*  $\boldsymbol{\theta}$ , the vector of *sufficient statistics*  $\phi(\mathbf{x})$ , the *partition function*  $Z(\boldsymbol{\theta})$ , the *log partition function* or *cumulant function*  $A(\boldsymbol{\theta})$  and the *scaling constant*  $h(\mathbf{x})$ .

The name *cumulant function* comes from the property of the exponential family being that derivatives of the log partition function can be used to generate cumulant of the sufficient statistics (the first and second cumulants of a distribution being its mean and variance).



**Purpose** We have the following data distribution:  $\mathbb{P}(y_i|\theta, \sigma^2) = \exp\left(\frac{y_i\theta - A(\theta)}{\sigma^2} + c(y_i, \sigma^2)\right)$  with  $c$  a normalization constant.

Let's consider an invertible mapping  $\Phi$  such that  $\theta = \Phi(\mu)$  with  $\mu$  being the mean parameter and  $\theta$  the natural parameter.

We have as well  $\mu = \Phi^{-1}(\theta) = A'(\theta)$ .

We are free to choose any link function as long as the inverse has an appropriate range.

| Distribution                 | Link function   | Natural parameter $\theta = \Phi(\mu)$        | Mean parameter $\mu = \Phi^{-1}(\theta) = \mathbb{E}(y)$ |
|------------------------------|-----------------|-----------------------------------------------|----------------------------------------------------------|
| $\mathcal{N}(\mu, \sigma^2)$ | <i>identity</i> | $\theta = \mu$                                | $\mu = \theta$                                           |
| $\mathcal{B}(n, \mu)$        | <i>logit</i>    | $\theta = \log\left(\frac{\mu}{1-\mu}\right)$ | $\mu = \text{sigm}(\theta)$                              |
| $\text{Poisson}(\mu)$        | <i>log</i>      | $\theta = \log(\mu)$                          | $\mu = e^\theta$                                         |

## Assumptions

## Theory

## Strengths

- GLMs can be fit using methods like gradient descent.

## Weaknesses

## Relationships with other methods

## Examples of application

### 10.2.3 Learning to rank

**Purpose** Modeling a function being able to rank a set of items. Suppose we have a query  $q$  and a set of documents  $(d_i)_{1 \leq i \leq n}$  and we would like to sort these documents in decreasing order of relevance.

## Assumptions

**Theory** Let us consider a document  $d$  and a query  $q$ , a standard way to measure the relevance between the both is to use  $\text{similitude}(q, d) \triangleq \mathbb{P}(q|d) = \prod_{i=1}^n \mathbb{P}(q_i|d)$  with  $q_i$  being the  $i^{\text{th}}$  word or term of  $q$ .

**Pointwise approach** for binary relevance labels, we can follow a standard binary classification scheme to estimate  $\mathbb{P}(y=1|\mathbf{x}(q, d))$ , in the case of ordered relevancy labels we can use an *ordinal regression to predict the rating*  $\mathbb{P}(y=r|\mathbf{x}(q, d))$ .

However this method does not take into account the location of each document in the list.

**Pairwise approach** to check the relative relevance of two items rather than absolute relevance. We can model this kind of data using a binary classifier of the form  $\mathbb{P}(y_{jk}|\mathbf{x}_j, \mathbf{x}_k) = \sigma(f(\mathbf{x}_j) - f(\mathbf{x}_k))$  where  $f$  is a scoring function, often taken to be linear:  $f(\mathbf{x}) = \mathbf{w}^T \mathbf{x}$ .

**Listwise approach** we now consider methods looking at the entire list of items at the same time. We can define a total order on a list by specifying a permutation of its indices:  $\pi$ . To model the uncertainty about  $\pi$  we can use the *Plackett-Luce* distribution.

$\mathbb{P}(\pi|\mathbf{s}) = \prod_{j=1}^m \frac{s_j}{\sum_{u=j}^m s_u}$  where  $s_j = s(\pi^{-1}(j))$ ,

the score of the document ranked at the  $j^{\text{th}}$  position.

## Strengths

## Weaknesses

## Relationships with other methods

## Examples of application

- *information retrieval* return a list of the top  $k$  more relevant documents, depending on a given query

### 10.2.4 Supervised PCA

**Purpose** Also called *Bayesian factor regression* this model take in account  $y_i$  when learning the low dimension embedding.

## Assumptions

$$\text{Theory} \quad \begin{cases} \mathbb{P}(z_i) = \mathcal{N}(\mathbf{0}, \mathbf{I}_L) \\ \mathbb{P}(y_i|z_i) = \mathcal{N}(w_y^T z_i + \mu_y, \sigma_y^2) \\ \mathbb{P}(x_i|z_i) = \mathcal{N}(W_x^T z_i + \mu_x, \sigma_x^2 \mathbf{I}_D) \end{cases}$$

The basic idea compressing  $x_i$  to predict  $y_i$  can be formulated using information theory, in particular we might want to find an encoding distribution  $\mathbb{P}(z|x)$  such that we minimize  $\mathbb{I}(X; Z) - \beta \mathbb{I}(X; Y)$ . Where  $\beta \geq 0$ , the *information bottleneck* is some parameter controlling the trade-off between compression and predictive accuracy.

## Strengths

## Weaknesses

## Relationships with other methods

- PCA

## Examples of application

- predict the movies that you would like knowing who your friends are as well as the rating from other users

### 10.2.5 Partial Least Squares

**Purpose** The key idea is to allow some of the (co)variance in the input features to be explained by its own subspace  $z_i^x$  and to let the remaining of subspace  $z_i^s$  be shared between input and output.

## Assumptions

$$\text{Theory} \quad \begin{cases} \mathbb{P}(z_i) = \mathcal{N}(z_i^s | \mathbf{0}, \mathbf{I}_{L_s}) \mathcal{N}(z_i^x | \mathbf{0}, \mathbf{I}_{L_x}) \\ \mathbb{P}(y_i|z_i) = \mathcal{N}(W_y z_i^s + \mu_y, \sigma^2 \mathbf{I}_{D_y}) \\ \mathbb{P}(x_i|z_i) = \mathcal{N}(W_x z_i^s + B_x z_i^x + \mu_x, \sigma^2 \mathbf{I}_{D_x}) \end{cases}$$

We should choose  $L$  large enough so that the shared subspace does not capture covariate specific variations.

## Strengths

## Weaknesses

## Relationships with other methods

## Examples of application

## 10.3 Classification and Regression

### 10.3.1 Multitask

**Purpose** To fit many related classification or regression models at the same time, as it is often reasonable to assume the input-output mappings is similar across these different models.

#### Assumptions

- Input/Output mapping

#### Theory

**Hierarchical Bayes for multi-task learning** Let consider  $y_{ij}$  the response of the  $i^{th}$  item in group  $j$  with  $(i, j) \in \llbracket 1, N_j \rrbracket \times \llbracket 1, J \rrbracket$ .

Although some groups may have lot of data, there is often a long tail where the majority of groups have little data. Thus we can't reliably fit each model separately but we don't want to use the same model for all the groups. As a compromise, [we can fit a separate model for each group but encourage the model parameters to be similar across the groups](#). More precisely:

$$\begin{cases} \mathbb{E}(y_{ij} | \mathbf{x}_{ij}) = g(\mathbf{x}_{ij}^T \beta_j) \\ \beta_j \hookrightarrow \mathcal{N}(\beta_*, \sigma_j^2 \mathbf{I}) \\ \beta_* \hookrightarrow \mathcal{N}(\mu, \sigma_*^2 \mathbf{I}) \end{cases}$$

$\beta_j$  is a coefficient vector associated with the group  $j$ .

Also groups with small sample size borrow statistical strength from the groups with larger sample size because the  $\beta_j$ 's are correlated via the latent common parents  $\beta_*$ .

The term  $\sigma_j^2$  controls how much group  $j$  depends on the common parents and the  $\sigma_*^2$  term controls the strength of the overall prior.

#### Strengths

- Performance improvement

#### Weaknesses

#### Relationships with other methods

#### Examples of application

- Determine the test scores for a given student in a given school without using as many models as school count
- Personal spam filtering:

### 10.3.2 Mixture models

**Purpose** It is the [simplest form of Latent Variable Models \(LVMs\)](#) is when  $z_i \in \llbracket 1, K \rrbracket$ .

Two mains application of mixture models:

- use them as *black-box* density model,  $p(\mathbf{x}_i)$ , useful for [data compression](#), [outlier detection](#) and [creating generative classifiers](#)
- use for [clustering](#)

#### Assumptions

**Theory** We use a **discrete prior**  $p(z_i) = \text{Cat}(\boldsymbol{\pi})$  and the likelihood  $p(\mathbf{x}_i|z_i = k)$ , finally the **mixture model** is :

$$\mathbb{P}(\mathbf{x}_i|\boldsymbol{\theta}) = \sum_{k=1}^K \pi_k \mathbb{P}(\mathbf{x}_i|z_i = k, \boldsymbol{\theta}) = \sum_{k=1}^K \pi_k \mathbb{P}_k(\mathbf{x}_i|\boldsymbol{\theta})$$

where  $\mathbb{P}_k$  is the  $k$ 'th *base distribution*.

**Mixtures of Gaussian** Each base distribution in the mixture is a multivariate Gaussian with mean  $\mu_k$  and covariance matrix  $\boldsymbol{\Sigma}_k$ :  $\mathbb{P}(\mathbf{x}_i|\boldsymbol{\theta}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_i|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$

Given a sufficiently large number of mixture components a **Gaussian Mixture Models (GMMs)** can be used to approximate any density defined on  $\mathbb{R}^D$ .

**Mixtures of multinoullis** can be used to define density models on data consisting of a  $D$ -dimensional bit vectors:  $\mathbb{P}(\mathbf{x}_i|z_i = k, \boldsymbol{\theta}) = \prod_{j=1}^D \text{Ber}(x_{ij}|\mu_{jk}) = \prod_{j=1}^D \mu_{jk}^{x_{ij}} (1 - \mu_{jk})^{1-x_{ij}}$  where  $\mu_{jk}$  is the probability that bit  $j$  turns on in cluster  $k$ .

**Mixture models for clustering** We first fit the mixture model and then compute  $\mathbb{P}(z_i = k|\mathbf{x}_i, \boldsymbol{\theta})$  representing the posterior probability that point  $i$  belongs to cluster  $k$ . This known as the *responsibility* of cluster  $k$  for point  $i$  and can be computed using Bayes:

$$r_{ik} \triangleq \mathbb{P}(z_i = k|\mathbf{x}_i, \boldsymbol{\theta}) = \frac{\mathbb{P}(z_i = k|\boldsymbol{\theta}) \mathbb{P}(\mathbf{x}_i|z_i = k, \boldsymbol{\theta})}{\sum_{k'=1}^K \mathbb{P}(z_i = k'|\boldsymbol{\theta}) \mathbb{P}(\mathbf{x}_i|z_i = k', \boldsymbol{\theta})}$$

**Mixture of experts** are build from a discriminative perspective, they relies on the idea that a good model can be achieved in **using multiple different linear method each applying to a different part of the input space**.

We can model this by allowing the mixing weights and the mixture densities to be input-dependent:

$$\begin{cases} \mathbb{P}(y_i|\mathbf{x}_i, z_i = k, \boldsymbol{\theta}) = \mathcal{N}(y_i|\mathbf{w}_k^T \mathbf{x}_i, \sigma_k^2) \\ \mathbb{P}(z_i|\mathbf{x}_i, \boldsymbol{\theta}) = \text{Cat}(z_i|S(\mathbf{V}^T \mathbf{x}_i)) \end{cases}$$

Useful in solving inverse problems, the ones in which we have to invert a many-to-one mapping, for example in robotics where the location of the end effector (hand)  $\mathbf{y}$  is uniquely determined by the joint angles of the motors,  $\mathbf{x}$ .

The overall posterior is then

$$\mathbb{P}(y_i|\mathbf{x}_i, \boldsymbol{\theta}) = \sum_k \mathbb{P}(z_i = k|\mathbf{x}_i, \boldsymbol{\theta}) \mathbb{P}(y_i|z_i = k, \mathbf{x}_i, \boldsymbol{\theta})$$

This model can be easily generalized to discrete data using the exponential family. The model is usually fit with *Expectation-Maximization* technique.

## Strengths

## Weaknesses

- Use of a single latent variable to generate the observation

## Relationships with other methods

## Examples of application

### 10.3.3 ARD: Automatic Relevance Determination

**Purpose** To get [greater sparsity](#)

**Assumptions**

**Theory** Relying on [empirical Bayes](#), whereby we integrate out  $\mathbf{w}$  and maximize the marginal likelihood  $\tau$ .

**Linear Regression** Let's consider weight precisions by  $\alpha_j = \frac{1}{\tau_j^2}$  and the measurement precision by  $\beta = \frac{1}{\sigma^2}$ :

$$\begin{cases} \mathbb{P}(y|\mathbf{x}, \mathbf{w}, \beta) = \mathcal{N}\left(y|\mathbf{w}^T\mathbf{x}, \frac{1}{\beta}\right) \\ \mathbb{P}(\mathbf{w}) = \mathcal{N}(\mathbf{w}|0, \mathbf{A}^{-1}) \end{cases}$$

where  $\mathbf{A} = \text{diag}(\boldsymbol{\alpha})$  To regularize the problem we may put a conjugate prior on each precision  $\alpha_j \hookrightarrow \text{Gamma}(a, b)$  and  $\beta \hookrightarrow \text{Gamma}(c, d)$  When performing EB point estimation will use the improper prior  $a = b = c = d = 0$  which results in [maximal sparsity](#).

**Sparsity** If  $\hat{\alpha}_j \approx 0$  we find  $\hat{w}_j \approx \hat{w}_j^{mle}$  since the Gaussian prior shrinking  $w_j$  towards 0 has zero precision. However if we find that  [\$\hat{\alpha}\_j \rightarrow \infty\$  then the prior is very confident that  \$w\_j = 0\$  and hence that feature  \$j\$  is "irrelevant".](#)

**Logistic Regression** uses iteratively reweighted  $l_1$  algorithm.

**Strengths**

- Resilient against uninformative features

**Weaknesses**

**Relationships with other methods**

**Examples of application**

### 10.3.4 Sparse coding

**Purpose** Sparse priors for unsupervised learning, here we relax the constraint that  $\mathbf{W}$  is orthogonal.

**Assumptions**

**Theory**

**Strengths**

**Weaknesses**

**Relationships with other methods**

**Examples of application**

### 10.3.5 Support Vector Machines (SVMs)

**Purpose** The combination of the *kernel trick* plus a modified loss function allowing *sparsity*, meaning that the prediction will only depend on a subset of the training data called *support vectors*. The overall process is known as a *Support Vector Machine*.

Because of sparsity encoding in the loss function instead of in the prior, kernel encoding through a trick instead of being an explicit part of the model, **SVMs do not provide probabilistic outputs**.

**SVMs for regression** with *kernalized ridge regression* the solution  $\mathbf{w}$  depends on all the training inputs. We should then use a variant of *Huber loss* function: the *epsilon insensitive loss function*:

$$L_{\epsilon}(y - \hat{y}) = \begin{cases} 0 & \Leftarrow |y - \hat{y}| < \epsilon \\ |y - \hat{y}| - \epsilon & \Leftarrow |y - \hat{y}| \geq \epsilon \end{cases}$$

meaning that any point lying inside an  $\epsilon$ -tube around the prediction is not penalized:  $J = C \sum_{i=1}^n L_{\epsilon}(y_i - \hat{y}_i) + \frac{1}{2} \|\mathbf{w}\|^2$ , with  $C = \frac{1}{\lambda}$  is a *regularization constant*.

It can be shown that **optimal solution has the form**  $\hat{\mathbf{w}} = \sum_i \alpha_i \mathbf{x}_i$ , where  $\alpha_i \geq 0$ . It turns out that  $\alpha$  is **sparse** as we don't care about errors which are smaller than  $\epsilon$ . The  $\mathbf{x}_i$  for which  $\alpha_i > 0$  are the **support vectors**.

Then we have  $\hat{y}(\mathbf{x}) = \hat{w}_0 + \hat{\mathbf{w}}^T \mathbf{x} = \hat{w}_0 + \sum_i \alpha_i \mathbf{x}_i^T \mathbf{x} = \sum_i \alpha_i k(\mathbf{x}, \mathbf{x}_i)$

**Classification** we consider now the *hinge loss*:

$$L_{\text{hinge}}(y, \eta) = \max(0, 1 - y\eta) = (1 - y\eta)_+$$

where  $\eta = f(\mathbf{x})$  is our 'confidence' in choosing label  $y = 1$  however it does not need to have any probabilistic semantics.

The overall objective has the form  $\min_{\mathbf{w}: w_0} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n (1 - y_i f(\mathbf{x}_i))_+$ . Same principle as in regression,

but this time  $\hat{y}(\mathbf{x}) = \text{sgn}(f(\mathbf{x})) = \text{sgn}(\hat{w}_0 + \hat{\mathbf{w}}^T \mathbf{x}) = \text{sgn}\left(\hat{w}_0 + \sum_{i=1}^n \alpha_i k(\mathbf{x}_i, \mathbf{x})\right)$

**The large margin principle** our goal is to derive a *discriminative function*  $f(x)$  which will be linear in the feature space implied by the choice of kernel. Hence:  $\mathbf{x} = \mathbf{x}_{\perp} + r \frac{\mathbf{w}}{\|\mathbf{w}\|}$  where  $r$  is the distance of  $\mathbf{x}$  from the decision boundary whose normal vector is  $\mathbf{w}$ , and  $\mathbf{x}_{\perp}$  is the orthogonal projection of  $\mathbf{x}$  onto this boundary. Hence  $f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + w_0 = (\mathbf{w}^T \mathbf{x}_{\perp} + w_0) + r \frac{\mathbf{w}^T \mathbf{w}}{\|\mathbf{w}\|}$ . As  $f(\mathbf{x}_{\perp}) = 0$ ,  $\mathbf{w}^T \mathbf{x}_{\perp} + w_0 = 0$ . Hence  $f(\mathbf{x}) = r \frac{\mathbf{w}^T \mathbf{w}}{\|\mathbf{w}\|}$ . Finally  $r = \frac{f(\mathbf{x})}{\|\mathbf{w}\|}$ , the distance that we would like to make as large as possible, in order to clearly separate the input.

**Regularization parameter  $C$**  it controls the number of errors we are willing to tolerate on the training set, it is chosen by cross-validation, an efficient way to chose  $C$  is to develop a path following algorithm in the spirit of *ARS*.

**Assumptions**

**Theory**

**Strengths**

- computational advantages over probabilistic model
- *kernel trick*  $\rightarrow$  **prevent underfitting**: ensuring that the feature vector is sufficiently rich that a linear classifier can separate

- *sparsity & large margin principles* → *prevent overfitting*: ensure that we do not use all the basis functions

### Weaknesses

- issues for multi-class classification due to the non-probabilistic aspect of the model: output scores are not on a calibrate scale

### Relationships with other methods

### Examples of application

## 10.3.6 MODEL COMPARISON

### Comparison of discriminative kernel methods

- *L1VM*:  $l_1$ -regularized vector machine
- *L2VM*:  $l_2$ -regularized vector machine
- *SVM*: Support Vector Machine
- *RVM*: Relevance Vector Machine
- *GP*: Gaussian Process
- *speed* → use RVM
- *well-calibrated probabilistic outputs* → GP

the only circumstances under which using an SVM seems sensible is the structured output case, where likelihood-based methods can be slow.

## 10.3.7 Gaussian Processes

**Purpose** It defines a prior over functions, which can be converted into a posterior over functions once we have seen some data.

They *can be thought of as a Bayesian alternative to the kernel methods*. In Python we can use the library *GPflow*

### Assumptions

### Theory

**Regression** Let the prior on the regression function be GP denoted by:

$$\begin{cases} f(\mathbf{x}) & \hookrightarrow GP(m(\mathbf{x}), k(\mathbf{x}, \mathbf{x}')) \\ m(\mathbf{x}) & = \mathbb{E}(f(\mathbf{x})) \text{ the mean function} \\ k(\mathbf{x}, \mathbf{x}') & = \mathbb{E}\left([f(\mathbf{x}) - m(\mathbf{x})]^T [f(\mathbf{x}') - m(\mathbf{x}')]\right) \text{ kernel or covariance function} \end{cases}$$

For any finite set of points, this process defines a joint Gaussian:  $\mathbb{P}(\mathbf{f}|\mathbf{X}) = \mathcal{N}(\mathbf{f}|\boldsymbol{\mu}, \mathbf{K})$ .

Where  $\mathbf{K}$  is defined by  $k_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$  and  $\boldsymbol{\mu} = (m(\mathbf{x}_i))_{1 \leq i \leq n}$

Let's consider the case where we predict using noise-free observation. Suppose we observe a *training observation set*  $\mathcal{D} = \{(\mathbf{x}_i, f_i)\}_{1 \leq i \leq n}$ , where  $f_i = f(\mathbf{x}_i)$  is the noise-free observation of the function evaluated at  $\mathbf{x}_i$ . Given a test set  $\mathbf{X}_*$  of size  $n_* \times D$  we want to predict the function output  $\mathbf{f}_*$ . The Gaussian Process will act as an *interpolator* of the training data.

$$\begin{bmatrix} \mathbf{f} \\ \mathbf{f}_* \end{bmatrix} \hookrightarrow \mathcal{N}\left(\begin{bmatrix} \boldsymbol{\mu} \\ \boldsymbol{\mu}_* \end{bmatrix}, \begin{bmatrix} \mathbf{K} & \mathbf{K}_* \\ \mathbf{K}_*^T & \mathbf{K}_{**} \end{bmatrix}\right)$$

where  $\mathbf{K} = k(\mathbf{X}, \mathbf{X})$ ,  $\mathbf{K}_* = k(\mathbf{X}, \mathbf{X}_*)$  and  $\mathbf{K}_{**} = k(\mathbf{X}_*, \mathbf{X}_*)$ . By the standard rules for conditioning

Gaussians, the posterior has the following form:

$$\begin{cases} \mathbb{P}(\mathbf{f}_* | \mathbf{X}_*, \mathbf{X}, \mathbf{f}) &= \mathcal{N}(\mathbf{f}_* | \boldsymbol{\mu}_*, \boldsymbol{\Sigma}_*) \\ \boldsymbol{\mu}_* &= \boldsymbol{\mu}(\mathbf{X}_*) + \mathbf{K}_*^T \mathbf{K}^{-1} (\mathbf{f} - \boldsymbol{\mu}(\mathbf{X})) \\ \boldsymbol{\Sigma}_* &= \mathbf{K}_{**} - \mathbf{K}_*^T \mathbf{K}^{-1} \mathbf{K}_* \end{cases}$$

**Strengths**

**Weaknesses**

**Relationships with other methods**

**Examples of application**

- noise-free GP regression is a computationally cheap proxy for the behavior of a complex simulator such as a weather forecasting program.

### 10.3.8 Classification And Regression Trees (CART)

**Purpose** Learning useful features directly from the input data

**Assumptions**

**Theory**

$$f(\mathbf{x}) = \mathbb{E}(y|\mathbf{x}) = \sum_{m=1}^M w_m \mathbb{1}_{\{\mathbf{x} \in R_m\}} = \sum_{m=1}^M w_m \phi(\mathbf{x}; \mathbf{v}_m)$$

where  $R_m$  is the  $m^{th}$  region  $w_m$  is the mean response in this region and  $\mathbf{v}_m$  encodes the choice of variable to split on and the threshold value on the path from the root to the  $m^{th}$  leaf.

**Growing a tree**

$$(j^*, t^*) = \arg \min_{j \in [1, D]} \min_{t \in \mathcal{T}_j} \text{cost}(\{\mathbf{x}_i, y_i : x_{ij} \leq t\}) + \text{cost}(\{\mathbf{x}_i, y_i : x_{ij} > t\})$$

where the set of possible thresholds  $\mathcal{T}_j$  for feature  $j$  can be obtained by sorting the unique values of  $x_{ij}$

**Cost**

- *Regression*:  $\text{cost}(\mathcal{D}) = \sum_{i \in \mathcal{D}} (y_i - \bar{y})^2$
- *Classification*: knowing that  $\hat{\pi}_c = \frac{1}{|\mathcal{D}|} \sum_{i \in \mathcal{D}} \mathbb{1}_{\{y_i = c\}}$ 
  - *Misclassification Rate*:  $\frac{1}{|\mathcal{D}|} \sum_{i \in \mathcal{D}} \mathbb{1}_{\{y_i \neq \hat{y}\}}$
  - *Entropy*  $\mathbb{H}(\hat{\pi}_c) = - \sum_{c=1}^C \hat{\pi}_c \log(\hat{\pi}_c)$
  - *Gini index*:  $\sum_{c=1}^C \hat{\pi}_c (1 - \hat{\pi}_c)$

Cross-entropy and Gini measures are very similar and more sensitive to changes in class probability than is the misclassification rate, finally they favor *pure nodes*



**Pruning** As stopping growing the tree to prevent overfitting would provoke to miss making any splits because of feature on its own has little predictive power.  
[Growing a "full" tree then performing \*pruning\* is better.](#)

**Random Forest** to **reduce the variance of an estimate**, we can train  $M$  different trees on **different subsets of the data**, randomly chosen with replacement and compute:

$$f(\mathbf{x}) = \sum_{m=1}^M \frac{1}{M} f_m(\mathbf{x})$$

where  $f_m$  is the  $m^{th}$  tree. This called **bagging (bootstrap aggregating)**  
 Simply re-running the sample learning algorithm on different subsets of the data can result in highly correlated predictors.

The **random forest** technique tries to decorrelate the base learners by learning trees based on a randomly chosen subset of input variables as well as a randomly chosen subset of data cases (observations).

### Strengths

- easy to interpret
- handle mixed discrete and continuous input
- insensitive to monotone transformation of the inputs (because the splits are based on ranking the data points)
- automatic variable selection
- relatively robust to outliers
- scale well to large datasets
- can be modified to handle missing inputs

### Weaknesses

- less accurate than other kinds of model (partly due to the greedy nature of the tree construction)
- unstable: small changes to the input data can have large effect on the structure of the tree (due to hierarchical nature of the tree-growing process)
- causing errors at the top can affect the rest of the tree

### Relationships with other methods

### Examples of application

#### 10.3.9 Generalized Additive models

**Purpose** To create a nonlinear model with multiple inputs:  $f(\mathbf{x}) = \alpha + \sum_{j=1}^D f_j(x_j)$   
 $f(\mathbf{x})$  can be mapped to  $\mathbb{P}(y|\mathbf{x})$  using a [link function](#) as in Generalized Linear Models.

### Assumptions

### Theory

**Smoothing splines** The loss function in regression setting:

$$J(\alpha, (f_j)_{1 \leq j \leq D}) = \sum_{i=1}^n \left( y_i - \alpha - \sum_{j=1}^D f_j(x_{ij}) \right)^2 + \sum_{j=1}^D \lambda_j \int f_j''(t_j)^2 dt_j$$

**Backfitting** As  $\alpha$  is not uniquely identifiable since we can always add or subtract constants to any of the  $f_j$ , the convention is  $\forall j \in \llbracket 1, D \rrbracket$ ,  $\sum_{i=1}^n f_j(x_{ij}) = 0$ , then  $\hat{\alpha} = \frac{1}{n} \sum_{i=1}^n y_i$ . Finally to fit the model:

1.  $\hat{f}_j \leftarrow \text{smoother} \left( \left\{ y_i - \sum_{k \neq j} \hat{f}_k(x_{ik}) \right\}_{1 \leq i \leq n} \right)$
2.  $\hat{f}_j \leftarrow \hat{f}_j - \frac{1}{n} \sum_{k \neq j} \hat{f}_k(x_{ik})$ , ensuring the output is zero mean

where  $\lambda_j$  is the strength of the regularizer for  $f_j$

**Multivariate Adaptive Regression Splines (MARS)** We create an ANOVA decomposition:

$$f(\mathbf{x}) = \beta_0 + \sum_{j=1}^D f_j(x_j) + \sum_{j,k} f_{jk}(x_j, x_k) + \sum_{j,k,l} f_{j,k,l}(x_j, x_k, x_l) + \dots$$

**Strengths**

**Weaknesses**

**Relationships with other methods**

**Examples of application**

### 10.3.10 Multi-layer Perceptron

**Purpose** It is a [series of logistic regression models stacked on top of each other](#), with the final layer being another logistic regression or a linear regression model.

**Assumptions**

**Theory** For a three layers regression problem the model has the form: 
$$\begin{cases} \mathbb{P}(y|\mathbf{x}, \boldsymbol{\theta}) &= \mathcal{N}(y|\mathbf{w}^T \mathbf{z}(\mathbf{x}), \sigma^2) \\ \mathbf{z}(\mathbf{x}) &= g(\mathbf{V}\mathbf{x}) \end{cases}$$

where [g is a non-linear activation function](#) (usually the logistic function),  $\mathbf{V}$  is the weight matrix from the input to the hidden nodes and [w is the weight vector from hidden nodes to the output](#).

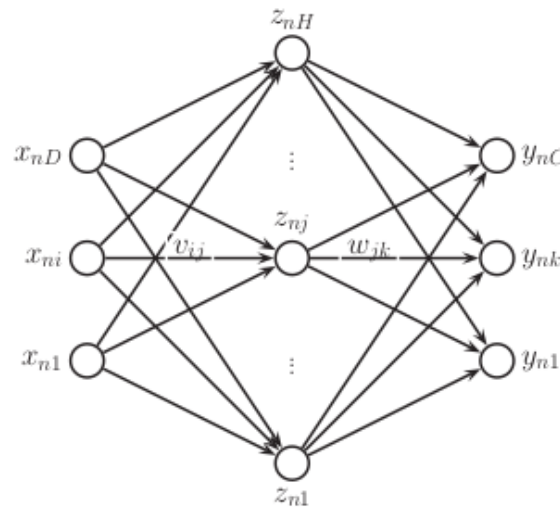


Figure 10.1: caption

**Convolutional Neural Networks** It is an MLP in which the hidden units have local receptive fields, and in which the weights are *tied* or *shared* across the image in order to reduce the number of parameters. Intuitively the effect of such spatial parameter tying is that any useful features that are "discovered" in some portion of the image can be re-used everywhere else without having to be independently learned. The resulting network then exhibits *translation invariance* meaning it can classify patterns no matter where they occur inside the input image.

**Strengths**

**Weaknesses**

**Relationships with other methods**

**Examples of application**

## 10.4 Model Selection

### 10.4.1 Bayesian Variable Selection

**Purpose** Let  $\gamma_j : \begin{cases} \gamma_j = 1 \Leftarrow \text{feature } j \text{ is relevant} \\ \gamma_j = 0 \Leftarrow \text{otherwise} \end{cases}$  our goal is to compute the posterior over models:

$$\mathbb{P}(\gamma|\mathcal{D}) = \frac{e^{-f(\gamma)}}{\sum_{\gamma'} e^{-f(\gamma')}}$$

where the cost function is defined by  $f(\gamma) \triangleq -[\log(\mathbb{P}(\mathcal{D}|\gamma)) + \log(\mathbb{P}(\gamma))]$

**Assumptions**

**Theory**

**Spike and slab model** remember that posterior is given by  $\mathbb{P}(\gamma|\mathcal{D}) \propto \mathbb{P}(\mathcal{D}|\gamma) \mathbb{P}(\gamma)$ .

It is common to use the following prior  $\mathbb{P}(\gamma) = \prod_{j=1}^D \text{Ber}(\gamma_j|\pi_0) = \pi_0^{\|\gamma\|_0} (1 - \pi_0)^{D - \|\gamma\|_0}$ , where  $\pi_0$  is the probability that a feature is relevant.

The likelihood is defined as follows:  $\mathbb{P}(\mathcal{D}|\mathbf{X}, \gamma) = \int \int \mathbb{P}(\mathbf{y}|\mathbf{X}, \mathbf{w}, \gamma) \mathbb{P}(\mathbf{w}|\gamma, \sigma^2) \mathbb{P}(\sigma^2) d\mathbf{w} d\sigma^2$

Consider the prior  $\mathbb{P}(\mathbf{w}|\gamma, \sigma^2)$  in standardizing the inputs, a reasonable prior is  $\mathcal{N}(0, \sigma^2 \sigma_w^2)$ , where  $\sigma_w^2$  controls how big we expect the coefficients associated with the relevant variables to be, which is scaled

by the overall noise. We can summarize this prior as follows:  $\mathbb{P}(\mathbf{w}_j|\sigma^2, \gamma_j) \begin{cases} \delta_0(w_j) \Leftarrow \gamma_j = 0 \\ \mathcal{N}(w_j|0, \sigma^2 \sigma_w^2) \Leftarrow \gamma_j = 1 \end{cases}$

the first term is a "spike" at the origin, as  $\sigma_w^2 \rightarrow +\infty$  the distribution  $\mathbb{P}(w_j|\gamma_j = 1)$  approaches a uniform distribution which can be thought of as a "slab".

$$\text{Bernoulli-Gaussian model} \quad \text{we have} \quad \begin{cases} \mathbb{P}(y_i|\mathbf{x}_i, \mathbf{w}, \gamma, \sigma^2) = \mathcal{N}\left(\sum_j \gamma_j w_j x_{ij}, \sigma^2\right) \\ \mathbb{P}(\gamma_j) = \text{Ber}(\pi_0) \\ \mathbb{P}(w_j) = \mathcal{N}(0, \sigma_w^2) \end{cases} \quad \text{we can think}$$

of the  $\gamma_j$  as a masking out the weights  $w_j$ . Unlike the spike and slab model we do not integrate the irrelevant coefficients, they always exists.

One interesting aspect of this model is that it can be used to derive objective function that is widely used in the non-Bayesian subset selection literature.

**Algorithms** assuming that we want to find the MAP model.

- Single best replacement: at each step, we define a neighborhood of the current model to be all models than can be reached by flipping a single bit of  $\gamma$
- Orthogonal least squares: we start from an empty set of variables and we add the best feature  $j^* = \arg \min_{j \notin \gamma_t} \min_w \|\mathbf{y} - \mathbf{X}_{\gamma_t \cup j} \mathbf{w}\|^2$
- Orthogonal matching pursuits: as Orthogonal least squares is somewhat expensive, a simplification is to freeze the current weight and then pick the next feature to add by solving  $j^* = \arg \min_{j \notin \gamma_t} \min_{\beta} \|\mathbf{y} - \mathbf{X} \mathbf{w}_t - \beta \mathbf{x}_{.j}\|^2$ . And  $\beta = \frac{\mathbf{x}_{.j}^T \mathbf{r}_t}{\|\mathbf{x}_{.j}\|^2}$ , where  $\mathbf{r} = \mathbf{y} - \mathbf{X} \mathbf{w}_t$
- Backward selection: starts with all variables in the model and deletes the
- Bayesian matching pursuits: similar to OMP except it uses a Bayesian marginal likelihood scoring criterion instead of a least square objective.

**Strengths**

**Weaknesses**

**Relationships with other methods**

**Examples of application**

### 10.4.2 Least Angle Regression Shrinkage

**Purpose** Instead of updating only one variable at a time, inducing slow converging, we can use Active set methods that update many variables at a time. They add or remove a few variables at a time so they can take a long time if they started far from the solution.

Suppose  $\mathcal{A}_k$  is the active set of variables at the beginning of the  $k^{th}$  step and let  $\beta_{\mathcal{A}_k}$  be the coefficient vector for these variables at this step, there will be  $k - 1$  nonzero values and the one just entered will be zero.

If  $\mathbf{r}_k = \mathbf{y} - \mathbf{X}_{\mathcal{A}_k} \beta_{\mathcal{A}_k}$  is the current residual, then the direction for this step is:

$$\delta_k = (\mathbf{X}_{\mathcal{A}_k}^T \mathbf{X}_{\mathcal{A}_k})^{-1} \mathbf{X}_{\mathcal{A}_k}^T \mathbf{r}_k$$

The name “least angle” arises from a geometrical interpretation of this process;  $\mathbf{u}_k$  makes the smallest (and equal) angle with each of the predictors in  $\mathcal{A}_k$

**Assumptions**

**Theory** Active methods exploit the fact that one can quickly compute  $\hat{\mathbf{w}}(\lambda_k)$  from  $\hat{\lambda}(\lambda_{k-1})$  if  $\lambda_k \approx \lambda_{k-1}$  this is known as **warm starting**.

#### Least Angle Regression

1. Standardize the predictors to have mean zero and unit norm.  
Start with the residual  $\mathbf{r} = \mathbf{y} - \bar{\mathbf{y}}$ , and for all  $j \in \llbracket 1, p \rrbracket$ ,  $\beta_j = 0$
2. Find the predictor  $x_j$  most correlated with  $\mathbf{r}$
3. For  $j \in \llbracket 1, p \rrbracket$ 
  - Move  $\beta_j$  from 0 to its least-squares coefficient  $\frac{\langle \mathbf{x}_j | \mathbf{r} \rangle}{\langle \mathbf{x}_j | \mathbf{x}_j \rangle}$ , then recompute  $\mathbf{r}$ . Find the predictor  $x_k$  most correlated with the new  $\mathbf{r}$
  - Move  $\beta_k$  from 0 to its least-squares coefficient  $\frac{\langle \mathbf{x}_k | \mathbf{r} \rangle}{\langle \mathbf{x}_k | \mathbf{x}_k \rangle}$ , then recompute  $\mathbf{r}$ . Find the predictor  $x_l$  most correlated with the new  $\mathbf{r} \dots$

- Move  $\beta_j$  and  $\beta_k$  in the direction defined by their joint least squares coefficient of the current residual on  $(x_j, x_k)$ , until some other competitor  $x_l$  has as much correlation with the current residual.
4. After  $\min(N-1, p)$  steps, we arrive at the full least-squares solution.

### Least Angle Regression: Lasso Modification

- 4a If a non-zero coefficient hits zero, drop its variable from the active set of variables and recompute the current joint least squares direction

### Strengths

- Numerically efficient when  $p \gg n$
- As fast as forward selection and has the same order of complexity as OLS
- Produces a full piecewise linear solution path
- Stable
- Easily modified to produce for other estimators like the Lasso

### Weaknesses

- Because LARS is based upon iterative refitting of the residuals, it would appear to be especially sensitive to the effect noise.

### Relationships with other methods

- Least Squares

### Examples of application

## 10.5 Regularization

### 10.5.1 $l_2$ regularization

#### Purpose

- Prevent overfitting
- Improve condition number

**Assumptions** We assume  $\mathbb{P}(\mathbf{w}) = \prod_{j=1}^D \mathcal{N}(w_j | 0, \tau^2)$ , where  $\frac{1}{\tau^2}$  controls the strength of the prior!

**Theory** The corresponding MAP estimation problem is :

$$\arg \max_{\mathbf{w}} \sum_{i=1}^n \log(\mathcal{N}(y_i | \mathbf{w}_0 + \mathbf{w}^T \mathbf{x}_i, \sigma^2)) + \sum_{j=1}^D \log(\mathcal{N}(w_j | 0, \tau^2))$$

We can rewrite this as

$$\frac{1}{n} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 + \lambda \|\mathbf{w}\|_2^2$$

The optimal solution is

$$\mathbf{w}_{ridge} = (\lambda \mathbf{I}_D + \mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

## Relationships with other methods

**Connection with PCA** Let  $\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^T$  be the *SVD* of  $\mathbf{X}$ .  $\hat{\mathbf{w}}_{ridge} = \mathbf{V}(\mathbf{S}^2 + \lambda\mathbf{I})^{-1}\mathbf{S}\mathbf{U}^T\mathbf{y}$   
Hence

$$\begin{aligned}\hat{\mathbf{y}} &= \mathbf{X}\hat{\mathbf{w}}_{ridge} \\ &= \mathbf{U}\mathbf{S}\mathbf{V}^T\mathbf{V}(\mathbf{S}^2 + \lambda\mathbf{I})^{-1}\mathbf{S}\mathbf{U}^T\mathbf{y} \\ &= \mathbf{U}\tilde{\mathbf{S}}\mathbf{U}^T\mathbf{y} \\ &= \sum_{j=1}^p \mathbf{u}_j \tilde{S}_{jj} \mathbf{u}_j^T \mathbf{y}\end{aligned}$$

where  $\tilde{S}_{jj} \triangleq [\mathbf{S}(\mathbf{S}^2 + \lambda\mathbf{I})^{-1}\mathbf{S}]_{jj} = \frac{\sigma_j^2}{\sigma_j^2 + \lambda}$  and  $\sigma_j$  are the singular values of  $\mathbf{X}$ . Therefore

$$\hat{\mathbf{y}} = \mathbf{X}\hat{\mathbf{w}}_{ridge} = \sum_{j=1}^p \mathbf{u}_j \frac{\sigma_j^2}{\sigma_j^2 + \lambda} \mathbf{u}_j^T \mathbf{y}$$

Whereas in ordinary least squares we have:  $\hat{\mathbf{y}} = \mathbf{X}\hat{\mathbf{w}}_{ols} = (\mathbf{U}\mathbf{S}\mathbf{V}^T)(\mathbf{V}\mathbf{S}^{-1}\mathbf{U}^T\mathbf{y}) = \mathbf{U}\mathbf{U}^T\mathbf{y} = \sum_{j=1}^p \mathbf{u}_j \mathbf{u}_j^T \mathbf{y}$

## 10.5.2 $l_1$ regularization

### Purpose

**Assumptions** We assume  $\mathbb{P}(\mathbf{w}|\lambda) = \prod_{j=1}^D \text{Lap}(w_j|0, \frac{1}{\lambda}) \propto \prod_{j=1}^D e^{-\lambda|w_j|}$

**Theory** For penalized negative log likelihood has the form:  $f(\mathbf{w}) = -\log(\mathbb{P}(\mathcal{D}|\mathbf{w})) - \log(\mathbb{P}(\mathbf{w}|\lambda)) = NLL(\mathbf{w}) + \lambda \|\mathbf{w}\|_1$ .

Geometrically we understand that as we relax the constraint we grow  $l_1$  ball until it meets the objective, the corners of the ball are more likely to intersect the ellipse than one of the sides especially in high dimensions because the corners "stick out". The corners correspond to sparse solutions which lie on the coordinate axes. By contrast when we grow the  $l_2$  ball it can intersect the objective at any point, there are no "corners" so there is no preference for sparsity.

### Strengths

### Weaknesses

- can give quite different results if the data is slightly perturbed

## Relationships with other methods

### Examples of application

## 10.5.3 Noise Robustness

**Purpose** It can be much more powerful than shrinking the parameters

### Assumptions

### Theory

### Strengths

Weaknesses

Relationships with other methods

Examples of application

### 10.5.4 Regularization path

**Purpose** As we increase  $\lambda$ , the solution vector  $\hat{\mathbf{w}}(\lambda)$  will tend to get sparser, although not necessary monotonically. For each feature  $j$  we can plot the values  $\hat{w}_j(\lambda)$  vs  $\lambda$  which is known as *regularization path*

Assumptions

Theory

Strengths

Weaknesses

Relationships with other methods

Examples of application

### 10.5.5 Coordinate Descent

**Purpose** These algorithms exploit the fact that one can quickly compute  $\hat{\mathbf{w}}(\lambda_k)$  from  $\hat{\mathbf{w}}(\lambda_{k-1})$  if  $\lambda_k \approx \lambda_{k-1}$  this is known as *warm starting*.  
Useful when it is hard to optimize all the variables simultaneously

Assumptions

Theory

**Aim of Coordinate descent** We solve for the  $j^{th}$  coefficient with all the others held fixed:

$$w_j^* = \arg \min_z f(\mathbf{w} + z\mathbf{e}_j) - f(\mathbf{w}) \quad \text{with } \mathbf{e}_j \text{ is the } j\text{'th unit vector.}$$

**Algorithm for Lasso**

1. Initialise  $\mathbf{w} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}$
2. Repeat until convergence:  
for  $j \in \llbracket 1, D \rrbracket$

- $\alpha_j = 2 \sum_{i=1}^n x_{ij}^2$
- $c_j = 2 \sum_{i=1}^n x_{ij} (y_i - \mathbf{w}^T \mathbf{x}_i + w_j x_{ij})$
- $w_j = \text{soft} \left( \frac{c_j}{\alpha_j}, \frac{\lambda}{\alpha_j} \right)$

Strengths

- if each one-dimensional optimization problem can be solved analitically

Weaknesses

Relationships with other methods

Examples of application

## 10.6 Ensemble Learning

The overall purpose is to learn a weighted combination of base models of the form:

$$f(y|\mathbf{x}, \boldsymbol{\pi}) = \sum_{m \in \mathcal{M}} w_m f_m(y|\mathbf{x})$$

where  $w_m$  are tunable parameters

### 10.6.1 Boosting

**Purpose** It aims to [build a model with reduced bias and variance](#).

It is a greedy algorithm for fitting adaptive basis-function models  $f(\mathbf{x}) = w_0 + \sum_{m=1}^M w_m \phi_m(\mathbf{x})$ , where  $\phi_m$  are generated by an algorithm called **weak learner**. This algorithm works by applying the weak learner sequentially to weighted versions of the data, where more weight is given to examples that were misclassified by earlier rounds.

This weak learner can be any classification or regression algorithm, but [it is common to use CART model](#). [Boosting is very resistant to overfitting](#).

The goal of boosting is to solve the following optimization problem:

$$\min_f \sum_{i=1}^n L(y_i, f(\mathbf{x}_i))$$

**Assumptions** A [family of weak learners](#) (working slightly better than random guess) [can be transformed in a family of strong learners](#)

### Theory

**Forward stagewise additive modeling** For squared error loss the optimal estimate is given by:  $f^*(\mathbf{x}) = \arg \min_{f(\mathbf{x})} \mathbb{E}_{y|\mathbf{x}} ([y - f(\mathbf{x})]^2) = \mathbb{E}(y|\mathbf{x})$

For binary classification we can use the logloss which is a convex upper bound on 0-1 loss. One can show that  $f^*(\mathbf{x}) = \frac{1}{2} \log \left( \frac{\mathbb{P}(\hat{y} = 1|\mathbf{x})}{\mathbb{P}(\hat{y} = -1|\mathbf{x})} \right)$

1. initialise by defining  $f_0(\mathbf{x}) = \arg \min_{\gamma} \sum_{i=1}^n L(y_i, f(\mathbf{x}; \gamma))$ , for example if we use squared error we

can set  $f_0(\mathbf{x}) = \bar{y}$  and if we use log-loss or exponential loss we can set  $f_0 = \frac{1}{2} \log \left( \frac{\hat{\pi}}{1-\hat{\pi}} \right)$  where

$$\hat{\pi} = \frac{1}{n} \sum_{i=1}^n \mathbb{1}_{\{y_i=1\}}$$

2. at iteration  $m$  we compute  $(\beta_m, \gamma_m) = \arg \min_{\beta, \gamma} \sum_{i=1}^n L(y_i, f_{m-1}(\mathbf{x}_i) + \beta \phi(\mathbf{x}_i; \gamma))$  where  $\phi$  is generated by the *weak learner* algorithm.

3.  $f_m(\mathbf{x}) = f_{m-1}(\mathbf{x}) + \nu \beta_m \phi(\mathbf{x}_i; \gamma_m)$  where  $\nu \in [0, 1]$  is a step-size parameter.



| Name                    | Loss                                    | Derivate                            | $f^*$                                                   | Algorithm                |
|-------------------------|-----------------------------------------|-------------------------------------|---------------------------------------------------------|--------------------------|
| <i>Squared error</i>    | $\frac{1}{2} (y_i - f(\mathbf{x}_i))^2$ | $y_i - f(\mathbf{x}_i)$             | $\mathbb{E}(y \mathbf{x}_i)$                            | <i>L2Boosting</i>        |
| <i>Absolute error</i>   | $ y_i - f(\mathbf{x}_i) $               | $\text{sgn}(y_i - f(\mathbf{x}_i))$ | $\text{median}(y \mathbf{x}_i)$                         | <i>Gradient Boosting</i> |
| <i>Exponential loss</i> | $\exp(-y_i, f(\mathbf{x}_i))$           | $-y_i \exp(-y_i f(\mathbf{x}_i))$   | $\frac{1}{2} \log \left( \frac{\pi_i}{1-\pi_i} \right)$ | <i>AdaBoost</i>          |
| <i>Logloss</i>          | $\log(1 + e^{-y_i f(\mathbf{x}_i)})$    | $-y_i - \pi_i$                      | $\frac{1}{2} \log \left( \frac{\pi_i}{1-\pi_i} \right)$ | <i>LogitBoost</i>        |

where  $\pi = \sigma(2f(\mathbf{x}))$

**Strengths** - robust against overfitting - reduce bias and variance

**Weaknesses**

**Relationships with other methods**

**Examples of application**

### 10.6.2 Bayes Model Averaging (that is not an Ensemble Method)

**Purpose** Instead of picking the best model and then using this to make predictions we make a weighted average of the predictions made by each model:

**Assumptions**

**Theory** Let's compute:

$$\mathbb{P}(\{y|\mathbf{x}, \mathcal{D}\}) = \sum_{m \in \mathcal{M}} \mathbb{P}(\{y|\mathbf{x}, m, \mathcal{D}\}) \mathbb{P}(\{m|\mathcal{D}\})$$

Obviously averaging over all the models is computationally infeasible, then a simple approximation

**Strengths**

- Better results than using any single model

**Weaknesses**

**Relationships with other methods**

**Examples of application**

### 10.6.3 Stacking

**Purpose** Refers to learning a weighted combination of base models of the form:  $f(y|\mathbf{x}, \boldsymbol{\pi}) = \sum_{m \in \mathcal{M}} w_m f_m(y|\mathbf{x})$

where  $w_m$  are the tunable parameters.

**Assumptions**

**Theory** To estimate the weights we can use :

$$\hat{\mathbf{w}} = \arg \min_{\mathbf{w}} \sum_{i=1}^n L \left( y_i, \sum_{m=1}^M w_m \hat{f}_m^{-i}(\mathbf{x}) \right)$$

with  $\hat{f}_m^{-i}(\mathbf{x})$  being the predictor obtained by training on data excluding  $(\mathbf{x}_i, y_i)$ . It would allow to avoid overfitting that would produce in using simply  $f_m$ . This know as **stacking** or **stacked generalization**.

### Strengths

- Robustness when the "true" model is not in the model class.

### Weaknesses

### Relationships with other methods

### Examples of application

- analogy with neural networks:  $f_m$  representing the  $m^{th}$  hidden unit and  $w_m$  are the outputs layer weights
- analogy with boosting: where the weights on the base models are determined sequentially

## 10.6.4 Bootstrap Aggregating (Bagging)

**Purpose** It is designed to [improve the stability and accuracy](#) of machine learning algorithms.

### Assumptions

**Theory** Let's consider  $M$  different models  $(f_m)_{1 \leq m \leq M}$  that we will train on random different subsets of data (*bootstrap step*).

We then compute the ensemble:

$$f(\mathbf{x}) = \sum_{m=1}^M \frac{1}{M} f_m(\mathbf{x})$$

### Strengths

- Better results than using any single model

### Weaknesses

- Variance reduction not very efficient because of rerunning same learning algorithm on different subsets of the data can result in highly correlated

### Relationships with other methods

- Random forest: use bagging in randomly cutting the input space vertically (subsets of predictors) and horizontally (subset of observations)

### Examples of application

# Chapter 11

## Unsupervised Learning

### 11.1 Clustering

#### 11.1.1 Clustering in general

**Purpose** 2 kinds of inputs we might use, *similarity-based clustering* the input to the algorithm is an  $N \times N$  dissimilarity matrix. In *feature-based clustering* the input to the algorithm is an  $N \times D$  feature matrix or design matrix  $\mathbf{X}$ .

#### Assumptions

#### Theory

**Measuring dissimilarity**  $\Delta(\mathbf{x}_i, \mathbf{x}_{i'}) = \sum_{j=1}^D \Delta_j(x_{ij}, x_{i'j})$

- *Squared distance*  $\Delta_j(x_{ij}, x_{i'j}) = (x_{ij} - x_{i'j})^2$
- *correlation* (useful for time-series of real-valued data)  $\Delta_j(x_{ij}, x_{i'j}) = Cor(\mathbf{x}_i, \mathbf{x}_{i'})$
- *l1 distance*  $\Delta_j(x_{ij}, x_{i'j}) = |x_{ij} - x_{i'j}|$
- *ordinal variables* any dissimilarity functions for quantitative function
- *categorical distance*  $\Delta_j(x_{ij}, x_{i'j}) = \sum_{j=1}^D \mathbb{1}_{\{x_{ij} \neq x_{i'j}\}}$

#### Measuring dis(similarity)

**Evaluating the output of clustering methods** The validation of clustering structures is the most difficult and frustrating part of cluster analysis

**Purity**  $purity \triangleq \sum_i \frac{N_i}{N} p_i$

#### Rand index

#### Mutual information

#### Variation of information

#### Strengths

Weaknesses

Relationships with other methods

Examples of application

### 11.1.2 *K*-means algorithm

**Purpose** Consider a *Gaussian Mixture Models* in which we make the following assumptions  $\Sigma_k = \sigma^2 I_D$  is fixed and  $\pi_k = \frac{1}{K}$ , then only the cluster centers  $\mu_k \in \mathbb{R}^D$  have to be estimated.

Assumptions

**Theory** Now consider  $\mathbb{P}(z_i = k | \mathbf{x}_i, \boldsymbol{\theta}) \approx \mathbb{1}_{\{k=z_i^*\}}$ , where  $z_i^* = \arg \max_k \mathbb{P}(z_i = k | \mathbf{x}_i, \boldsymbol{\theta})$ .

As we are making a hard assignment of points to clusters, this is sometimes called *hard EM*. Since we assumed an equal spherical covariance matrix for each cluster we have  $z_i^* = \arg \min_k \|\mathbf{x}_i - \boldsymbol{\mu}_k\|_2^2$

The *M step* updates each cluster center by computing the mean of all points assigned to it:  $\boldsymbol{\mu}_k = \frac{1}{n_k} \sum_{i: z_i=k} \mathbf{x}_i$ .

Since K-means is not a proper EM algorithm it is not maximizing likelihood, instead it can be interpreted as a greedy algorithm minimizing a loss function related to data compression.

Strengths

Weaknesses

Relationships with other methods

- Gaussian Mixture

Examples of application

### 11.1.3 Vector Quantization

**Purpose** Suppose performing lossy compression of some real-valued vectors  $\mathbf{x}_i \in \mathbb{R}^D$ .

The basic idea is to replace each real-valued vector  $\mathbf{x}_i \in \mathbb{R}^D$  with a discrete symbol  $z_i \in \llbracket 1, K \rrbracket$  being an index into a *codebook* of  $K$  prototypes  $\boldsymbol{\mu}_k \in \mathbb{R}^D$

**Theory** Each data vector is encoded by using the index of the most similar prototype where similarity is measured in terms of Euclidean distance:

$$\text{encode}(\mathbf{x}_i) = \arg \min_k \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$$

Here a cost function measuring the quality of the codebook by computing the *reconstruction error*:

$$J(\boldsymbol{\mu}, z | K, \mathbf{X}) \triangleq \frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K \|\mathbf{x}_i - \text{decode}(\text{encode}(\mathbf{x}_i))\|^2 = \frac{1}{N} \sum_{i=1}^N \|\mathbf{x}_i - \boldsymbol{\mu}_{z_i}\|^2$$

Assumptions

Strengths

Weaknesses

Relationships with other methods

## Examples of application

### 11.1.4 Factor Analysis

**Purpose** Getting a bigger representation power through latent variables than Mixture models in using real-valued latent variable  $\mathbf{z}_i \in \mathbb{R}^L$ .

It can be thought of as a way of specifying a joint density model on  $\mathbf{x}$  using a small number of parameters.

#### Assumptions

**Theory** Let's consider  $\begin{cases} \mathbb{P}(\mathbf{z}_i) = \mathcal{N}(\mathbf{z}_i | \boldsymbol{\mu}_0, \boldsymbol{\Sigma}_0) \\ \mathbb{P}(\mathbf{x}_i | \mathbf{z}_i, \boldsymbol{\theta}) = \mathcal{N}(\mathbf{W}\mathbf{z}_i + \boldsymbol{\mu}, \boldsymbol{\Phi}) \end{cases}$  where  $\mathbf{W}$  is a  $D \times L$  matrix known as *factor loading matrix* and  $\boldsymbol{\Phi}$  is a  $D \times D$  covariance matrix, which is diagonal as we aim to force  $\mathbf{z}_i$  to explain correlation.

### 11.1.5 Mixture of Factor Analyzers

**Purpose** While Factor Analysis assumes that data lives on a low dimensional *linear* manifold, in reality most data is better modeled by some form of low dimensional *curved* manifold.

We can approximate a curved manifold by a piecewise linear manifold

#### Assumptions

**Theory** Let the  $k^{th}$  linear subspace of dimensionality  $L_k$  be represented by  $\mathbf{W}_k$ . Suppose we have a latent indicator  $q_i \in \llbracket 1, k \rrbracket$  specifying which subspace we should use to generate the data. Here the model:

$$\begin{cases} \mathbb{P}(\mathbf{x}_i | \mathbf{z}_i, q_i = k, \boldsymbol{\theta}) &= \mathcal{N}(\mathbf{x}_i | \mathbf{W}_k \mathbf{z}_i, \boldsymbol{\Psi}) \\ \mathbb{P}(\mathbf{z}_i | \boldsymbol{\theta}) &= \mathcal{N}(\mathbf{z}_i | \mathbf{0}, \mathbf{I}) \\ \mathbb{P}(q_i | \boldsymbol{\theta}) &= \text{Cat}(q_i | \boldsymbol{\pi}) \end{cases}$$

#### Strengths

- it is a good density model for high-dimensional real-valued data

#### Weaknesses

#### Relationships with other methods

## Examples of application

### 11.1.6 Canonical Correlation Analysis

**Purpose** It is a symmetric unsupervised version of PLS, it allows each view to have its own "private" subspace, but there is also a shared space.

#### Assumptions

**Theory** If we have 2 observed variables  $\mathbf{x}_i$  and  $\mathbf{y}_i$  then we have 3 latent variables:  $\mathbf{z}_i^s \in \mathbb{R}^{L_0}$  which is shared,  $\mathbf{z}_i^x \in \mathbb{R}^{L_x}$  and  $\mathbf{z}_i^y \in \mathbb{R}^{L_y}$  which are private :

$$\begin{cases} \mathbb{P}(\mathbf{z}_i) &= \mathcal{N}(\mathbf{z}_i^s | \mathbf{0}, \mathbf{I}_{L_s}) \mathcal{N}(\mathbf{z}_i^x | \mathbf{0}, \mathbf{I}_{L_x}) \mathcal{N}(\mathbf{z}_i^y | \mathbf{0}, \mathbf{I}_{L_y}) \\ \mathbb{P}(\mathbf{x}_i | \mathbf{z}_i) &= \mathcal{N}(\mathbf{x}_i | \mathbf{B}_x \mathbf{z}_i^s + \mathbf{W}_x \mathbf{z}_i^x + \boldsymbol{\mu}_x, \sigma^2 \mathbf{I}_{D_x}) \\ \mathbb{P}(\mathbf{y}_i | \mathbf{z}_i) &= \mathcal{N}(\mathbf{y}_i | \mathbf{B}_y \mathbf{z}_i^s + \mathbf{W}_y \mathbf{z}_i^y + \boldsymbol{\mu}_y, \sigma^2 \mathbf{I}_{D_y}) \end{cases}$$

#### Strengths

Weaknesses

Relationships with other methods

Examples of application

### 11.1.7 Independent Component Analysis (ICA)

**Purpose** When we want to deconvolve mixed signals into their constituent parts.

Unlike PCA, we relax the Gaussian assumption about latent variable, now the distribution can be any non-Gaussian distribution. The reason the Gaussian distribution is disallowed as a source prior in ICA is that it does not permit unique recovery of the sources.

Assumptions

**Theory**  $\mathbb{P}(\mathbf{z}_t) = \prod_{j=1}^L \mathbb{P}_j(z_{tj})$

Strengths

Weaknesses

Relationships with other methods

Examples of application

### 11.1.8 DBSCAN: Density-Based Spatial Clustering of Applications with Noise

**Purpose** Given a set of points in some space  $(p_i)_{1 \leq i \leq n} \in \mathcal{S}^n$ , it groups together points that are closely packed together, making then as outliers points that lie alone in low density regions.

Assumptions

Theory

**Definition** Let  $\epsilon \in \mathbb{R}$  the radius of a neighborhood with respect to some points,  $\rho_{min}$  a threshold of point count. Consider 2 points  $p$  and  $q$ .

- $\#\{m : \|m - p\| < \epsilon\} > \rho_{min} \Rightarrow p$  is **core point**
- $\begin{cases} p \text{ is a core point} \\ \|q - p\| < \epsilon \end{cases} \Rightarrow q$  is **directly reachable from**  $p$
- Let  $n \in \mathbb{N}$ .  $\exists (m_i)_{1 \leq i \leq n} \in \mathcal{S}^n$ ,  $\begin{cases} (m_1 = p) \wedge (m_n = q) \\ \forall i \in \llbracket 1, n-1 \rrbracket \ m_{i+1} \text{ is directly reachable from } m_i \end{cases} \Rightarrow q$  is **reachable from**  $p$
- Points that are not reachable from a core point are considered as **outliers**
- If there is a point  $m$  from which 2 points  $p$  and  $q$  are reachable from  $m$  then  $p$  and  $q$  are **density-connected**

### Algorithm

1. For each point  $m$ , count the number of points at a distance lower than  $\epsilon$  from  $m$
2. Identify the core points, the ones having a neighborhood count greater than  $\rho_{min}$
3. Delimit clusters with the reachable property of the different core points.
4. For each non-core points assign it to the closer cluster being at a distance lower than  $\epsilon$ , if no cluster is found consider this point as noise.

### Strengths

- Number of cluster does not need to be specified
- Clusters are arbitrary-shaped
- Robust to outliers

### Weaknesses

- For a given border point that is reachable by multiple clusters, the cluster with which it will be associated with depends on the database order.
- Due to euclidean distance to measure distance between 2 points, DBSCAN can suffer from "curse of dimensionality"
- Cannot cluster well data sets with large differences in densities, since  $\rho_{min}$  and  $\epsilon$  would not be set appropriately for each cluster.
- If the data is not well understood, choosing meaningful  $\rho_{min}$  and  $\epsilon$  is difficult.

### Relationships with other methods

### Examples of application

#### 11.1.9 Affinity Propagation

**Purpose** The idea is that each point must choose another data point as its exemplar or centroid some data points will choose themselves as centroids and this will automatically determine the number of clusters

### Assumptions

### Theory

### Strengths

### Weaknesses

### Relationships with other methods

### Examples of application

#### 11.1.10 Dirichlet Process

### Purpose

### Assumptions

### Theory

Strengths

Weaknesses

Relationships with other methods

Examples of application

#### **11.1.11 Spectral Clustering**

Purpose

Assumptions

Theory

Strengths

Weaknesses

Relationships with other methods

Examples of application

#### **11.1.12 Agglomerative Clustering**

Purpose

Assumptions

Theory

Strengths

Weaknesses

Relationships with other methods

Examples of application

#### **11.1.13 Bayesian Hierarchical Clustering**

Purpose

Assumptions

Theory

Strengths

Weaknesses

Relationships with other methods

Examples of application



## 11.2 Association

### 11.2.1 Directed graphical models (Bayes nets)

**Purpose** Relevant to **compactly represent the joint distribution**  $\mathbb{P}(\mathbf{x}|\boldsymbol{\theta})$  and then *infer* one set of variables given another, and how *learn* the parameters of this distribution.

#### Assumptions

- Conditional independence

#### Theory

**Graph terminology** let's consider a *graph*  $G = (\mathcal{V}, \mathcal{E})$  consisting of a set of *vertices* or nodes  $\mathcal{V} = v_{1 \leq v \leq V}$  and *edges*,  $\mathcal{E} = \{(s, t) : s, t \in \mathcal{V}^2\}$

- *Cycle*: series a cycle to be a series of nodes such that we can get back to where we started by following edges.
- *DAG*: Directed Acyclic Graph is a directed graph without cycles.
- *Tree*: Undirected graph without cycles.
- *Forest*: set of trees

A **Directed Graphical Model (DGM)** are more commonly known as *Bayesian Networks*.

**Inference** In partitioning the data into *visible variables*  $\mathbf{x}_v$  and *hidden variables*  $\mathbf{x}_h$ , inference refers to computing the posterior distribution of the unknown given the knows:

$$\mathbb{P}(\mathbf{x}_h | \mathbf{x}_v, \boldsymbol{\theta}) = \frac{p(\mathbf{x}_h, \mathbf{x}_v | \boldsymbol{\theta})}{p(\mathbf{x}_v | \boldsymbol{\theta})} = \frac{p(\mathbf{x}_h, \mathbf{x}_v | \boldsymbol{\theta})}{\sum_{\mathbf{x}'_h} p(\mathbf{x}'_h, \mathbf{x}_v | \boldsymbol{\theta})}$$

**Learning** The aim is to compute a MAP estimate of the parameters given data:  $\hat{\boldsymbol{\theta}} = \arg \max_{\boldsymbol{\theta}} \sum_{i=1}^n \log(x_{i,v} | \boldsymbol{\theta}) + \log(\mathbb{P}(\boldsymbol{\theta}))$ .

In adopting a Bayesian view, the parameter are unknown variables and should also be inferred, thus to a Bayesian there is no distinction between inference and learning.

#### Strengths

#### Weaknesses

#### Relationships with other methods

#### Examples of application

### 11.2.2 Kernels

**Purpose** **Measuring the similarity between 2 objects** that does not require preprocessing them into feature vector format.

#### Assumptions

## Theory

- *Radial Basis Function (RBF)*, or Gaussian kernel:  $k(\mathbf{x}, \mathbf{x}') = e^{\frac{1}{2}(\mathbf{x}-\mathbf{x}')^T \Sigma^{-1}(\mathbf{x}-\mathbf{x}')}$
- *Mercer* kernel: kernel for which the *Gram matrix*  $K = \begin{pmatrix} k(\mathbf{x}_1, \mathbf{x}_1) & \cdots & k(\mathbf{x}_1, \mathbf{x}_n) \\ \vdots & & \vdots \\ k(\mathbf{x}_n, \mathbf{x}_1) & \cdots & k(\mathbf{x}_n, \mathbf{x}_n) \end{pmatrix}$  is positive definite
- *Linear* kernel:  $k(\mathbf{x}, \mathbf{x}') = \mathbf{x}^T \mathbf{x}'$ , useful if original data is already high dimensional and features individually informative
- *Matern* kernels:  $k(r) = \frac{2^{1-\nu}}{\Gamma(\nu)} \left( \frac{\sqrt{2\nu}r}{l} \right)^\nu K_\nu \left( \frac{\sqrt{2\nu}r}{l} \right)$  where  $r = \|\mathbf{x} - \mathbf{x}'\|$ ,  $\nu > 0$ ,  $l > 0$  and  $K_\nu$  a modified Bessel function.
- *Probability product* kernels:  $k(\mathbf{x}_i, \mathbf{x}_j) = \int \mathbb{P}(\mathbf{x}|\mathbf{x}_i)^\rho \mathbb{P}(\mathbf{x}|\mathbf{x}_j)^\rho d\mathbf{x}$  where  $\rho > 0$ , relevant for a probabilistic generative model.
- *Fisher* kernels:  $k(\mathbf{x}, \mathbf{x}') = \mathbf{g}(\mathbf{x})^T \mathbf{F}^{-1} \mathbf{g}(\mathbf{x}')^T$  where  $\mathbf{g}$  is the gradient of the log-likelihood and  $\mathbf{F}$  is the Fisher information matrix.
- *Epanechnikov* kernel:  $k(x) \triangleq \frac{3}{4}(1-x^2)\mathbb{1}_{\{|x| \leq 1\}}$ , compact support (useful for efficiently reasons since one can use fast nearest neighbor methods to evaluate density) but not continuous derivatives at the boundaries of its support
- *Tri-cube* kernel:  $k(x) \triangleq \frac{3}{4}(1-|x|^3)^3\mathbb{1}_{\{|x| \leq 1\}}$ , having compact support and two continuous derivatives at the boundary of its support.

**Kernel machines** is a Generalized Linear Model where the input feature vector has the form  $\phi(\mathbf{x}) = (k(\mathbf{x}, \boldsymbol{\mu}_k))_{1 \leq k \leq K}$  where  $\boldsymbol{\mu}_k$  being the  $k^{th}$  centroid. We can use any of the sparsity-promoting priors for  $\mathbf{w}$  to efficiently select a subset of the training exemplars, it is sparse vector machine. We can get even greater sparsity by using *ARD/SBL* resulting in a method called the *Relevance Vector Machine* (RVM).

**Kernel trick** Rather than defining our feature vector in terms of kernels,  $\phi(\mathbf{x}) = (k(\mathbf{x}, \mathbf{x}_i))_{1 \leq i \leq n}$ , we can instead work with the original feature vectors  $\mathbf{x}$  but modify algorithm so that we replace all inner product  $\langle \mathbf{x} | \mathbf{x}' \rangle$  with a call to the kernel function  $k(\mathbf{x}, \mathbf{x}')$ .

### Kernalized model

- *Nearest Neighbor (Classification)*: in using  $\|\mathbf{x}_i - \mathbf{x}_{i'}\|_2^2 = \langle \mathbf{x}_i | \mathbf{x}_i \rangle + \langle \mathbf{x}_{i'} | \mathbf{x}_{i'} \rangle - 2\langle \mathbf{x}_i | \mathbf{x}_{i'} \rangle$
- *K-medoids (Clustering)*: similar to K-means, but instead of representing each cluster's centroid by the mean of all data vectors assigned to this cluster, we make each centroid be one of the data vectors themselves. When we update the centroids, we look at each object that belong to the cluster and measure the sum of its distance to all others in the same cluster with:  $m_k = \arg \min_{i: z_i=k} \sum_{i': z_{i'}=k} \|\mathbf{x}_i - \mathbf{x}_{i'}\|_2^2$  where  $z_i = \arg \min_k d(i, m_k)$
- *Ridge Regression*: using the matrix inversion lemme,  $\mathbf{w}_{Ridge} = \mathbf{X}_T (\mathbf{X} \mathbf{X}^T + \lambda \mathbf{I}_n)^{-1} \mathbf{y}$ , we can partially kernalize this by replacing  $\mathbf{X} \mathbf{X}^T$  with the *Gram matrix*. Let's define dual variables  $\boldsymbol{\alpha} \triangleq (\mathbf{K} + \lambda \mathbf{I}_n) \mathbf{y}$  then we rewrite the primal variables  $\mathbf{w} = \mathbf{X}^T \boldsymbol{\alpha} = \sum_{i=1}^n \alpha_i \mathbf{x}_i^T \mathbf{x}_i = \sum_{i=1}^n \alpha_i k(\mathbf{x}_i, \mathbf{x})$
- *PCA*: from the eigenvectors of the inner product matrix  $\mathbf{X} \mathbf{X}^T$ , allowing to produce a nonlinear embedding using kernel trick. The mathematical demonstration is long and not important.

**Kernel Density Estimation (KDE)** Instead of estimating  $\mu_k$  we can allocate one cluster center per data point so  $\mu_i = \mathbf{x}_i$  then the model becomes:  $\mathbb{P}(\mathbf{x}|\mathcal{D}) = \frac{1}{n} \sum_{i=1}^n \mathcal{N}(\mathbf{x}|\mathbf{x}_i, \sigma^2 \mathbf{I})$  That we can generalize with:

$$\hat{p}(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^n k_h(\mathbf{x} - \mathbf{x}_i)$$

This is called *Parzen window density estimator* or *Kernel Density Estimator*.

**Strengths**

**Weaknesses**

**Relationships with other methods**

**Examples of application**

- comparing documents

### 11.2.3 Latent Dirichlet Allocation

**Purpose**

**Assumptions**

**Theory**

**Strengths**

**Weaknesses**

**Relationships with other methods**

**Examples of application**

## 11.3 Dimensionality Reduction

### 11.3.1 Principal Components Analysis

**Purpose** Consider the *FA* model where we constrain  $\Phi = \sigma^2 \mathbf{I}$  and  $\mathbf{W}$  to be orthonormal. It can be shown that when  $\sigma^2$  tends to 0, this model reduces to classical PCA, otherwise it will be *Probabilistic PCA*.

**Assumptions**

**Theory** Suppose we want to find an orthogonal set of  $L$  linear basis vectors  $\mathbf{w}_j \in \mathbb{R}^D$  and the corresponding scores  $\mathbf{z}_i \in \mathbb{R}^L$  such that we minimize the average reconstruction error:

$$J(\mathbf{W}, \mathbf{Z}) = \frac{1}{n} \sum_{i=1}^n \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2$$

where  $\hat{\mathbf{x}}_i = \mathbf{W} \mathbf{z}_i$  subject to the constraint that  $\mathbf{W}$  is orthonormal.

**Model selection** We can plot the *scree plot* :  $E(\mathcal{D}_{train}, L)$  vs  $L$ , with  $E$  being the error reconstruction. A related quantity is the fraction of variance explained defined as  $F(\mathcal{D}_{train}, L) = \frac{\sum_{j=1}^L \lambda_j}{\sum_{j'=1}^{L_{max}} \lambda_{j'}}$

### Strengths

- *PCA* is the best low rank approximation to the data

### Weaknesses

- It is not a proper generative model of the data, in providing more latent dimensions it will be able to better approximate the test data

### Relationships with other methods

### Examples of application

## 11.3.2 PCA for categorical data

**Purpose** Useful when data is categorical rather than real-valued

### Assumptions

**Theory** Let  $\mathbf{X} \in \mathcal{M}_{np}([1, c])$  and we assume that each  $x_{ij}$  is generated from a latent variable  $\mathbf{z}_i \in \mathbb{R}^L$ .

$$\begin{cases} \mathbb{P}(\mathbf{z}_i) = \mathcal{N}(\mathbf{0}, \mathbf{I}) \\ \mathbb{P}(x_{ij} | \mathbf{z}_i, \boldsymbol{\theta}) = \prod_{j=1}^p \text{Cat}(x_{ij} | \mathcal{S}(\mathbf{W}_j^T \mathbf{z}_i + \mathbf{w}_{0j})) \end{cases} \quad \text{where } \mathbf{W} \in \mathbb{R}^{L \times M} \text{ is the factor loading matrix for}$$

response  $j$  and  $\mathbf{w}_{0j} \in \mathbb{R}^M$  is the offset term for response  $j$ . To fit the model in using a modified version of EM: infer a Gaussian approximation to the posterior  $\mathbb{P}(\mathbf{z}_i | \mathbf{x}_i, \boldsymbol{\theta})$  in the E step and then to maximize  $\boldsymbol{\theta} = (\mathbf{W}_j, \mathbf{w}_{0j})_{1 \leq j \leq p}$  in M step.

### Strengths

### Weaknesses

### Relationships with other methods

### Examples of application

## Chapter 12

# Semi-supervised Learning

## Chapter 13

# Reinforcement Learning

# Chapter 14

## Optimization methods

### 14.1 Optimization of loss functions

#### 14.1.1 Gradient Descent

**Purpose** Simplest algorithm for unconstrained optimization.

**Theory**

**Principle**

$$\theta_{k+1} = \theta_k - \eta_k g_k$$

where  $\eta_k$  is the *step size* or *learning rate*. For a given function to optimize  $f : \mathbb{R}^p \rightarrow \mathbb{R}$  being 2 times derivable with *Taylor's theorem*:  $f(\theta + \eta d) \approx f(\theta) + \eta g^T d$  where  $d$  is our descent directions.

If  $\eta$  is chosen small enough then  $f(\theta + \eta d) < f(\theta)$  since the gradient will be negative. But we don't want to choose the step size  $\eta$  to small, so let's pick  $\eta$  to minimize  $\phi(\eta) = f(\theta_k + \eta d_k)$  this is called *line minimization* or *line search*

**Zig-zag behavior** exact line search satisfies  $\eta_k = \arg \min_{\eta > 0} \phi(\eta)$ . A necessary condition for the optimum is  $\phi'(\eta) = 0$ , by the chain rule  $\phi'(\eta) = g^T d$  where  $g = f'(\theta, \eta d)$  is the gradient at the end of the step. So we either have  $g = 0$  meaning that we found a stationary point or  $g \perp d$  meaning that exact search stops at a point where the local gradient is perpendicular to the search direction.

**Weaknesses**

- setting the *step size*: too large steps can induce failing to converge, too small can as well fail to converge because of too long time

#### 14.1.2 Newton's method

**Purpose** Takes the curvature of the space, the Hessian matrix, into account, it is belong to the family of *second order* optimization methods

**Theory** It consists to update:

$$\theta_{k+1} = \theta_k - \eta_k H_k^{-1} g_k$$

Consider making a second-order Taylor series approximation of  $f(\theta)$  around  $\theta_k$ :  $f_{quad}(\theta) = f_k + g_k^T(\theta - \theta_k) + \frac{1}{2}(\theta - \theta_k)^T H_k (\theta - \theta_k)$ .

That can be rewrite as

$$f_{quad}(\theta) = \theta^T A \theta + b^T \theta + c \quad \begin{cases} A &= \frac{1}{2} H_k \\ b &= g_k - H_k \theta_k \\ c &= f_k - g_k^T \theta_k + \frac{1}{2} \theta_k^T H_k \theta_k \end{cases}$$

The minimum of  $f_{quad}$  is at  $\theta = -\frac{1}{2}A^{-1}b = \theta_k - H_k^{-1}g_k$ . Thus the Newton step  $d_k = -H_k^{-1}g_k$  is what should be added to  $\theta_k$  to minimize the second order approximation of around  $\theta_k$ .

### Strengths

- Faster than usual gradient descent

## 14.1.3 Online learning and stochastic optimization

**Purpose** Instead of minimizing regret with respect to the past, we want to minimize expected loss in the future.

**Theory**  $f(\theta) = \mathbb{E}(f(\theta, z))$  where the expectation is taken over future data.

One way to optimize this *stochastic objective* such as the above equation, is to perform the update  $\theta_{k+1} = \text{proj}_{\Theta}(\theta_k - \eta_k g_k)$  where  $\text{proj}_{\mathcal{V}}(v) = \arg \min_{w \in \mathcal{V}} \|w - v\|^2$  that is the projection of vector  $v$  onto space  $\mathcal{V}$ .

In practice it is usually better to randomly permute the data and sample without replacement, and then to repeat. A single such pass over the entire data set is called an *epoch*.

In this offline cases, it is often better to compute the gradient of a *mini-batch* of  $B$  data cases. If  $B = 1$  this is standard Stochastic Gradient Descent, and if  $B = n$  this is standard Gradient Descent typically  $B \approx 100$  is used.

### Strengths

- online learning, useful for streaming data
- main way to train large model on large dataset
- as the number of examples in the training set approaches infinity the model will eventually converge to its best possible test errors before SGD has sampled every example in the training set.

## 14.1.4 EM Algorithm

**Purpose** Gradient-based optimizer used to find a local minimum of the *Negative Log Likelihood* can be stuck with the imposed constraint like positive definite covariance matrices, mixing weights having to sum to one.

In a few words *Expectation Maximization (EM)* which alternates between inferring the missing values given the parameters (*E step*), and then optimizing the parameters given the filled in data (*M step*).

The goal is to maximize the log likelihood of the observed data:

$$l(\theta) = \sum_{i=1}^n \log(\mathbb{P}(x_i|\theta)) = \sum_{i=1}^n \log \left( \sum_{z_i} \mathbb{P}(x_i, z_i|\theta) \right)$$

since the log cannot be pushed inside the sum, it is difficult to optimize, EM gets around this problem in defining the *complete data log likelihood* to be  $l_c(\theta) \triangleq \sum_{i=1}^n \log(\mathbb{P}(x_i, z_i|\theta))$ . But it cannot be computed since  $z_i$  is unknown. So let's define the **expected complete data log likelihood**:

$$Q(\theta, \theta^{t-1}) = \mathbb{E}(l_c(\theta) | \mathcal{D}, \theta^{t-1})$$

where  $t$  is the current iteration number,  $Q$  is called the **auxiliary function**. The goal of the *E step* is to compute  $Q(\theta, \theta^{t-1})$ , in the *M step* we optimize the  $Q$  function.

### Assumptions

### Theory

### Strengths



## Weaknesses

## Relationships with other methods

## Examples of application

### 14.1.5 Backpropagation Algorithm

**Purpose** Useful for Neural Network's loss function is non-convex function of its parameters. It is common to use first-order online methods such as stochastic gradient descent.

## Assumptions

**Theory** Let's assume a model with just one hidden layer, it is helpful to distinguish the pre- and post-synaptic values of a neuron, that is before and after applying the non-linearity. Then  $\mathbf{x}_n$ , the  $n^{th}$  input,  $\mathbf{a}_n = \mathbf{V}\mathbf{x}_n$  be the pre-synaptic hidden layer and  $\mathbf{z}_n = g(\mathbf{a}_n)$  be the pro-synaptic hidden layer where  $g$  is some transfer function. More succinctly:

$$\mathbf{x}_n \xrightarrow{\mathbf{V}} \mathbf{a}_n \xrightarrow{g} \mathbf{z}_n \xrightarrow{\mathbf{W}} \mathbf{b}_n \xrightarrow{h} \hat{\mathbf{y}}$$

$\theta = (\mathbf{V}, \mathbf{W})$  the first and second layer weight matrices.

With  $K$  outputs the negative linear loss is given by

- *regression*:  $J(\theta) = -\sum_n \sum_k (\hat{y}_{nk}(\theta) - y_{nk})$
- *classification*:  $J(\theta) = -\sum_n \sum_k y_{nk} \log(\hat{y}_{nk}(\theta))$

Our task is to compute  $\Delta_\theta J$ , we will derive this for each  $n$  separately, the overall gradient is obtained by summing over  $n$

1. *output layer*:  $\nabla_{\mathbf{w}_k} J_n = \frac{\partial J_n}{\partial b_{nk}} \nabla_{\mathbf{w}_k} = \frac{\partial J_n}{\partial b_{nk}} \mathbf{z}_n$  since  $b_{nk} = \mathbf{w}_k^T \mathbf{z}_n$
2. considering  $J_n$  as a squared penalty:  $\frac{\partial J_n}{\partial b_{nk}} \triangleq \sigma_{nk}^w = (\hat{y}_{nk} - y_{nk})$  being the error signal, so the overall gradient  $\nabla_{\mathbf{w}_k} J_n = \sigma_{nk}^w \mathbf{z}_n$
3. *input layer*:  $\nabla_{v_j} J_n = \frac{\partial J_n}{\partial a_{nj}} \nabla_{v_j} a_{nj} \triangleq \delta_{nj}^v \mathbf{x}_n$

## Strengths

## Weaknesses

## Relationships with other methods

## Examples of application

## 14.2 Variational Inference

### 14.2.1 Basics

Suppose  $p^*(\mathbf{x})$  is our true but intractable distribution and  $q(\mathbf{x})$  is some approximation chosen from some tractable family, such as a multivariate Gaussian.

In using:

$$\mathbb{KL}(p^*||q) = \sum_{\mathbf{x}} q(\mathbf{x}) \log \left( \frac{q(\mathbf{x})}{p^*(\mathbf{x})} \right)$$

**Mean field method** We assume the posterior is fully factorized approximation of the form:  $q(\mathbf{x}) = \prod_i q_i(\mathbf{x}_i)$  The goal is to solve the optimization problem  $\min \mathbb{KL}(q||p)$

### 14.2.2 Variational Bayes

When we want to infer the parameters themselves, if we make a fully factorized approximation  $\mathbb{P}(\boldsymbol{\theta}|\mathcal{D}) \approx \prod_k q(\boldsymbol{\theta}_k)$

### 14.2.3 Variational Bayes EM

TO COMPLETE

The proposal distribution

### 14.2.4 Gibbs sampling

TO COMPLETE

## 14.3 Numerical Computation

### 14.3.1 Numerical Computation

**Poor conditioning** Conditioning refers to how rapidly a function changes with respect to small changes in its inputs.

$$\max_{i,j} \left| \frac{\lambda_i}{\lambda_j} \right|$$

### 14.3.2 Jacobian and Hessian matrices

**Jacobian** If we have a function  $f : \mathbb{R}^m \rightarrow \mathbb{R}^n$ , then the Jacobian matrix  $\mathbf{J} \in \mathbb{R}^{n \times m}$  such that

$$J_{i,j} = \frac{\partial}{\partial x_j} f(\mathbf{x}_i)$$

**Hessian**

### 14.3.3 Optimization technique

**Gradient-Based Optimization**

Part IV

Deep Learning

# Chapter 15

## Optimization

### 15.1 Back-propagation

#### 15.1.1 Purpose

The [back-propagation](#) algorithm allows the information from the cost to then flow backwards through the network [in order to compute the gradient](#).

Actually it refers only to the [method for computing the gradient](#), while [another algorithm such as stochastic gradient descent](#) is used to perform learning using this gradient.

#### 15.1.2 Modeling

**Chain rule of calculus** Let  $(\mathbf{x}, \mathbf{y}) \in \mathbb{R}^m \times \mathbb{R}^n$  and  $z = f(\mathbf{y})$  with  $f : \mathbb{R}^n \rightarrow \mathbb{R}$  we have  $\nabla_{\mathbf{x}} z = \begin{bmatrix} \frac{dz}{dx_1} \\ \vdots \\ \frac{dz}{dx_m} \end{bmatrix}$ ,

$$\nabla_{\mathbf{y}} z = \begin{bmatrix} \frac{dz}{dy_1} \\ \vdots \\ \frac{dz}{dy_n} \end{bmatrix} \text{ and } \frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{dy_1}{x_1} & \dots & \frac{dy_1}{x_m} \\ \vdots & \ddots & \vdots \\ \frac{dy_n}{x_1} & \dots & \frac{dy_n}{x_m} \end{bmatrix} \text{ we have :}$$

$$\nabla_{\mathbf{x}} z = \left( \frac{\partial \mathbf{y}}{\partial \mathbf{x}} \right)^T \nabla_{\mathbf{y}} z$$

**Back-propagation in fully connected MLP** Consider a network of depth  $l$ , and the matrices of the model  $(\mathbf{W}_j)_{1 \leq j \leq l}$  and the bias parameters  $(\mathbf{b}_j)_{1 \leq j \leq l}$  then  $\mathbf{x}$  the input vector and  $\mathbf{y}$  the target vector.

**Forward propagation** Consider a  $l-1$  hidden layer model, assume that we use an unique activation function  $f$ :

1.  $\mathbf{h}_0 = \mathbf{x}$  consider the input as the first layer
2. for  $k \in \llbracket 1, l \rrbracket$ 
  - $\mathbf{a}_k = \mathbf{b}_k + \mathbf{W}_k \mathbf{h}_{k-1}$  vector containing all the resulting weighted sum and bias of inputs  $k$
  - $\mathbf{h}_k = f(\mathbf{a}_k)$  vector containing all the neuron outputs of the considered layer by applying the activation function
3.  $\hat{\mathbf{y}} = \mathbf{h}_l$  output layer
4.  $J = L(\hat{\mathbf{y}}, \mathbf{y}) + \lambda \Omega(\theta)$  finally compute loss function with a potential regulation term

## Backward

1.  $\mathbf{g} \leftarrow \nabla_{\hat{\mathbf{y}}} J = \nabla_{\hat{\mathbf{y}}} L(\hat{\mathbf{y}}, \mathbf{y})$  initial gradient vector
2. for  $k \in \llbracket l-1, 1 \rrbracket$ 
  - $\mathbf{g} \leftarrow \nabla_{\mathbf{a}_k} J = \mathbf{g} \odot f'(\mathbf{a}_k)$  compute derivative with respect to  $\mathbf{a}_k$ , Hadamard product is used to imitate the rule:  $(h \circ f)(x)' = h'(f(x)) f'(x)$
  - $\nabla_{\mathbf{b}_k} J = \mathbf{g} + \lambda \nabla_{\mathbf{b}_k} \Omega(\boldsymbol{\theta})$
  - $\nabla_{\mathbf{w}_k} J = \mathbf{g} \mathbf{h}_{k-1}^T + \lambda \nabla_{\mathbf{w}_k} \Omega(\boldsymbol{\theta})$  # TO COMPLETE
  - $\mathbf{g} \leftarrow \nabla_{\mathbf{h}_{k-1}} J = \mathbf{W}_k^T \mathbf{g}$

# Chapter 16

## Regularization

### 16.1 Norm penalties

We denote the regularized objective function by:

$$\tilde{J}(\boldsymbol{\theta}, \mathbf{X}, \mathbf{y}) = J(\boldsymbol{\theta}, \mathbf{X}, \mathbf{y}) + \alpha \Omega(\boldsymbol{\theta}) \text{ with } \begin{cases} \alpha \in [0, \infty[ \text{ hyper-parameters weighting the impact of norm penalty} \\ \Omega \text{ relative to standard objective function } J \end{cases}$$

#### 16.1.1 Data Augmentation

#### 16.1.2 Drop out

**Purpose** It provides an inexpensive approximation to [training and evaluating a bagged ensemble of exponentially many neural networks](#).

**Theory** It [trains the ensemble consisting of all sub-networks that can be formed by removing non-output units](#) from an underlying base network.

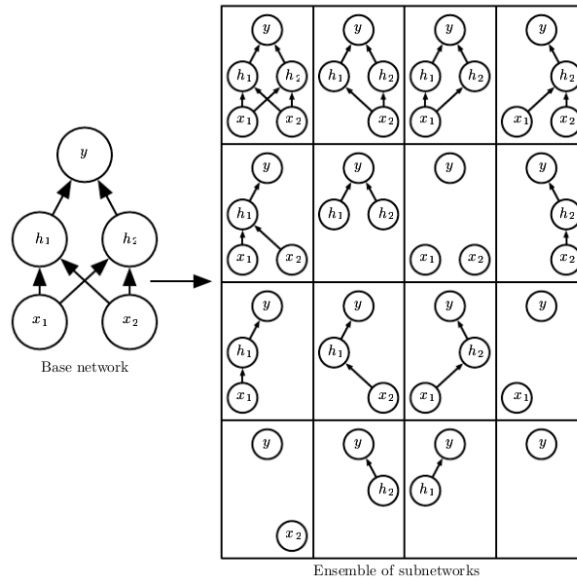


Figure 16.1: Drop out illustration

# Chapter 17

## Types of Neural Networks

### 17.1 Convolutional Neural Networks

#### 17.1.1 Components of CNN

There are [neural networks using convolution](#) operation instead of general matrix multiplication [in at least one of their layers](#).

1. Apply several convolution in parallel to produce a set of activation functions
2. The previous activation functions are run through nonlinear activation function (*detector stage*)
3. Modify the output of the layer further with **pooling functions**

#### Convolution

**Convolution operation** [allows to reduce the noise coming from observation by considering multiple observations and averaging them](#). However we want to have the ability to give more weight to some observations, the more recent ones for example. Let's assume we observe [a signal  \$x\(t\)\$  \(input\)](#) and we have a [weighting function  \$w\(a\)\$  \(kernel\)](#) where [a is the age of a measurement](#) the resulting signal issued from convolution (*feature map*) is:

$$s(t) = (x * w)(t) = \int x(a)w(t-a)da$$

or in matrix convention

$$S(n, p) = (I * K)(n, p) = \sum_i \sum_j I(i, j)K(n-i, p-j) = \sum_i \sum_j I(n-i, p-j)K(i, j) = (K * I)(n, p)$$

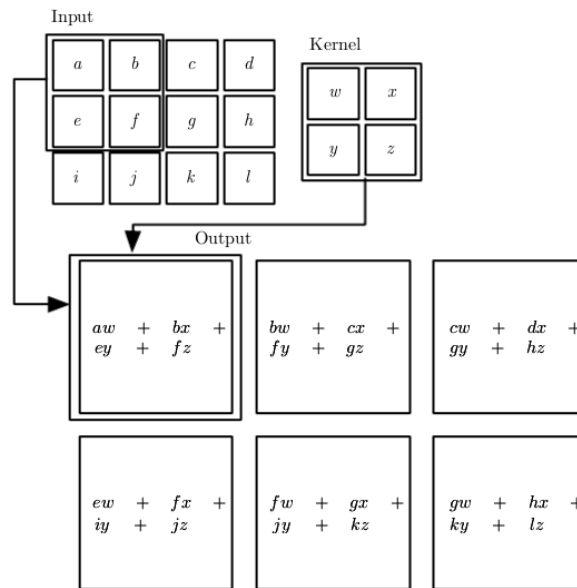


Figure 17.1: 2-D convolution without kernel-flipping

### Motivation

- **sparse interactions** (due to reduced size of kernel compare with the input one)
- **parameter sharing** (rather than learning a separate set of parameters, for every location we learn only one set, that are the kernel's ones)
- **equi-variant representations:**  $f(x)$  is equi-variant to a function  $g$  if  $f(g(x)) = g(f(x))$ . For example if  $g$  is any function that translates the input then the convolution is equi-variant to  $g$ 
  - for time series, **convolution produces a sort of timeline showing when different features appear in the input.**  
By moving an event later in time in the input will induce that the same representation of the event will appear in the output
  - for images, **convolution creates a 2-D map of where certain features appear in the input.**  
By moving the object in the input, its representation will move the same amount in the output
- **ability to work with inputs of variable size**

### Pooling

**Purpose** of pooling is to **make representation become approximately invariant to small translations of the input**. Such a priority can be very useful when we are searching for the presence of some feature rather than their location.

It can be viewed as **adding an infinitely strong prior to the weight resulting in layer invariance to small translations**

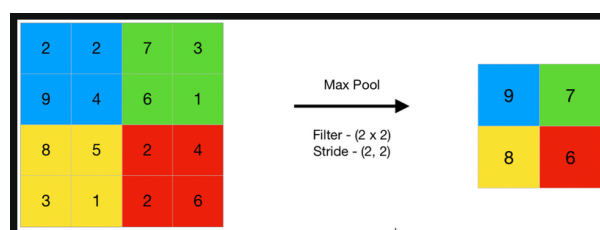


Figure 17.2: Max pooling



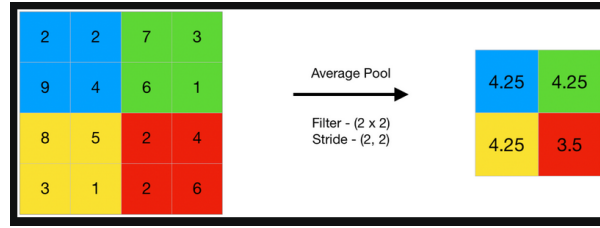


Figure 17.3: Max pooling

### Examples of pooling

## 17.2 Recurrent and Recursive Neural Networks

### 17.2.1 Recurrent Neural Networks

**Architecture** , let's consider a *computational graph* to compute the training loss of a recurrent neural network that maps an input sequence of  $\mathbf{L}$  to a corresponding sequence of output  $\mathbf{o}$  values. A loss  $\mathbf{L}$  measures how far each  $\mathbf{o}$  is from the corresponding training target  $\mathbf{y}$

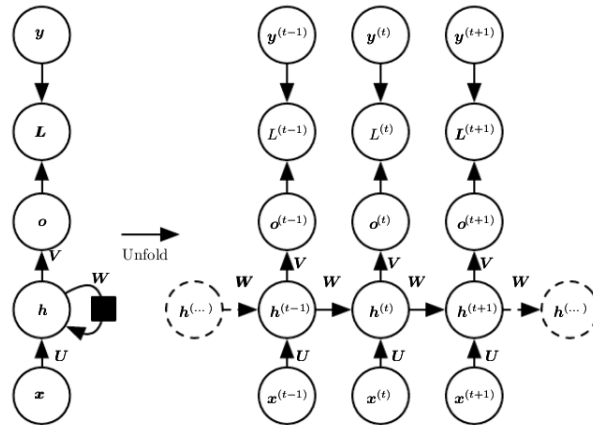


Figure 17.4: Recurrent Neural Networks

### 17.2.2 Encoder-Decoder Sequence-to-Sequence Architectures

It is composed of an encoder RNN that reads the input sequence and a decoder RNN that generates the output sequence.

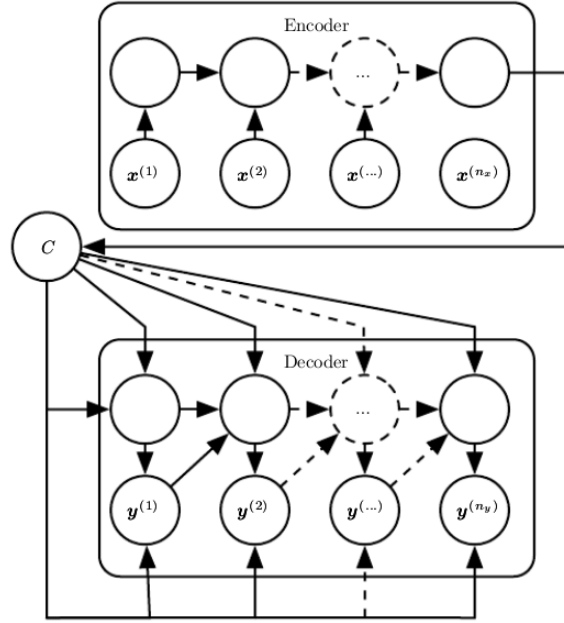


Figure 17.5: Encoder-Decoder (or Sequence-to-Sequence RNN) architecture

The final hidden state of the encoder RNN is used to compute a generally fixed-size context variable  $C$  representing a semantic summary of the input sequence, then is given as input to the decoder RNN.

### 17.2.3 The Challenge of Long-Term Dependencies

**The basic problem** is that gradients propagated over many stages tend to either vanish or explode. Consider a very simple recurrent neural network lacking a nonlinear function:  $\mathbf{h}_t = \mathbf{W}^T \mathbf{h}_{t-1}$ , if we admits an eigen-decomposition of the form  $\mathbf{W} = \mathbf{Q} \mathbf{\Lambda} \mathbf{Q}^T$  with orthogonal  $\mathbf{Q}$  the recurrence may be simplified further to  $\mathbf{h}_t = \mathbf{Q}^T \mathbf{\Lambda}^t \mathbf{Q} \mathbf{h}_0$ . The eigenvalues are raised to the power of  $t$  causing the eigenvalues with magnitude lesser than one to decay to zero and eigenvalues with magnitude greater than one to explode.

### 17.2.4 Leaky Units

**Purpose** dealing with long-term dependencies by designing a model operating at multiple time scales, allowing to let some parts of the model operating at fine-grained time scales to handle small details while other parts operate at coarse time scales and transfer information from the distant past to the present more efficiently.

**Adding Skip Connections through Time** consists in adding direct connections from variables in the distant past to variables in the present.

By introducing recurrent connections with a time-delay of  $d$  to mitigate, Gradients now diminish exponentially as a function of  $\frac{\tau}{d}$  rather than  $\tau$

**Leaky Units** Another way to obtain paths on which the product of derivatives is close to one is to have units with linear self-connections and a weight near one on these connections. When we accumulate a running average  $\mu_t$  of some value  $v_t$  by applying  $\mu_t \leftarrow \alpha \mu_{t-1} + (1 - \alpha) v_t$  the  $\alpha$  parameter is an example of linear self connection from  $\mu_{t-1}$  to  $\mu_t$ .

When  $\alpha$  is near one, the running average remembers information about the past for a long time, and when  $\alpha$  is near zero, information about the past is rapidly discarded.

## 17.2.5 The Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU)

**Purpose** going further with the idea of self-loops, by making the weight of the self-loop conditioned on the context rather than fixed.

Self-loops produce paths where the gradient can flow for long duration.

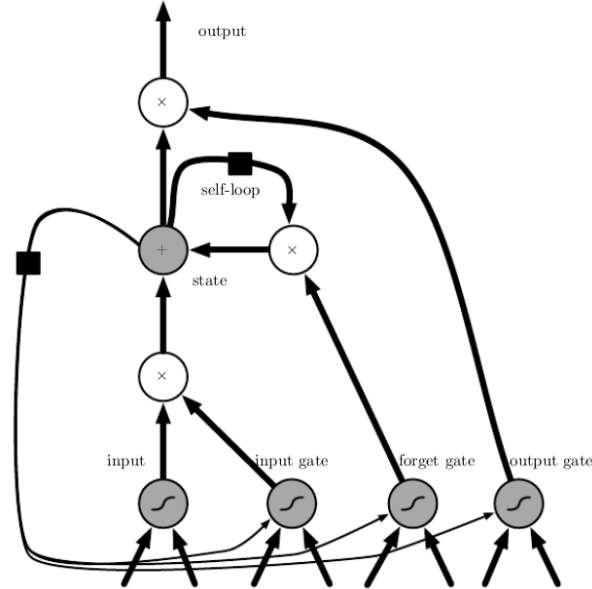


Figure 17.6: Block diagram of the LSTM recurrent neural network *cell*

Cells are connected recurrently to each other replacing the usual hidden units of ordinary recurrent networks.

**Theory** The **forget gate** unit  $f_i^{(t)}$  for time step  $t$  and cell  $i$  that sets the weight to a value between 0 and 1

$$f_i^t = \sigma \left( b_i^f + \sum_j U_{i,j}^f x_j^{(t)} + \sum_j W_{i,j}^f h_j^{(t-1)} \right)$$

with  $\mathbf{x}^{(t)}$  the current input vector,  $\mathbf{h}^t$  the current hidden layer vector containing the outputs of all the LSTM cells, and  $\mathbf{b}^f$ ,  $\mathbf{U}^f$ ,  $\mathbf{W}^f$  are respectively biases, input weights and recurrent weights for the forget gates.

**Part V**

**Overall Issues**

## 17.3 High Dimensions

### 17.3.1 Dimension Increase

### 17.3.2 Curse of dimensionality

**Purpose** When the dimensionality increases, the volume of the space increases so fast that the available data become sparse.

#### Main issues

**Combinatorics** Let's consider  $(d, k) \in \mathbb{N}^*$  and  $(x_i)_{1 \leq i \leq d} \in \llbracket 1, K \rrbracket^d$ , the range of possible values for the families  $(x_i)_{1 \leq i \leq d}$  is  $K^d$  which increases obviously exponentially.

# Bibliography

- [1] Omar Elgabry. *The Ultimate Guide to Data Cleaning*. 2019. URL: <https://towardsdatascience.com/the-ultimate-guide-to-data-cleaning-3969843991d4>.
- [2] Wikipedia contributors. *Moments (Mathematics)*. [Online; accessed 21-August-2023]. 2023. URL: [https://en.wikipedia.org/wiki/Moment\\_\(mathematics\)](https://en.wikipedia.org/wiki/Moment_(mathematics)).
- [3] Wikipedia contributors. *Probability*. [Online; accessed 20-August-2023]. 2023. URL: <https://en.wikipedia.org/wiki/Probability>.